```
            DeleteXTree(bt->lchild);
            DeleteXTree(bt->rchild);      //删除bt的左子树、右子树
            free(bt);                     //释放被删结点所占的存储空间
        }
    }
    //在二叉树上查找所有以x为元素值的结点，并删除以其为根的子树
    void Search(BiTree bt,ElemType x){
        BiTree Q[];                       //Q是存放二叉树结点指针的队列，容量足够大
        if(bt){
            if(bt->data==x){              //若根结点值为x，则删除整棵树
                DeleteXTree(bt);
                exit(0);
            }
            InitQueue(Q);
            EnQueue(Q,bt);
            while(!IsEmpty(Q)){
                DeQueue(Q,p);
                if(p->lchild)                         //若左子女非空
                    if(p->lchild->data==x){           //左子树符合则删除左子树
                        DeleteXTree(p->lchild);
                        p->lchild=NULL;
                    }                                 //父结点的左子女置空
                    else
                        EnQueue(Q,p->lchild);         //左子树入队列
                if(p->rchild)                         //若右子女非空
                    if(p->rchild->data==x){           //右子女符合则删除右子树
                        DeleteXTree(p->rchild);
                        p->rchild=NULL;
                    }                                 //父结点的右子女置空
                    else
                        EnQueue(Q,p->rchild);         //右子女入队列
            }
        }
    }
```

09.【解答】

算法思想：采用非递归后序遍历，最后访问根结点，访问到值为x的结点时，栈中所有元素均为该结点的祖先，依次出栈打印即可。算法实现如下：

```
    typedef struct{
        BiTree  t;
        int     tag;
    }stack;  //tag=0表示左子女被访问，tag=1表示右子女被访问
    void  Search(BiTree bt,ElemType x){
    //在二叉树bt中，查找值为x的结点，并打印其所有祖先
        stack s[];                                        //栈容量足够大
        top=0;
        while(bt!=NULL||top>0){
            while(bt!=NULL&&bt->data!=x){                 //结点入栈
                s[++top].t=bt;
                s[top].tag=0;
                bt=bt->lchild;                            //沿左分支向下
            }
            if(bt!=NULL&&bt->data==x){
                printf("所查结点的所有祖先结点的值为:\n");  //找到x
                for(i=1;i<=top;i++)
                printf("%d",s[i].t->data);                //输出祖先值后结束
```

```
            exit(1);
        }
        while(top!=0&&s[top].tag==1)
            top--;                                    //退栈（空遍历）
        if(top!=0){
            s[top].tag=1;
            bt=s[top].t->rchild;                      //沿右分支向下遍历
        }
    } //while(bt!=NULL||top>0)
}
```

因为查找的过程就是后序遍历的过程，所以使用的栈的深度不超过树的深度。

10.【解答】

后序遍历最后访问根结点，即在递归算法中，根是压在栈底的。本题要找 p 和 q 的最近公共祖先结点 r，不失一般性，设 p 在 q 的左边。算法思想：采用后序非递归算法，栈中存放二叉树结点的指针，当访问到某结点时，栈中所有元素均为该结点的祖先。后序遍历必然先遍历到结点 p，栈中元素均为 p 的祖先。先将栈复制到另一辅助栈中。继续遍历到结点 q 时，将栈中元素从栈顶开始逐个到辅助栈中去匹配，第一个匹配（相等）的元素就是结点 p 和 q 的最近公共祖先。算法实现如下：

```
typedef struct{
    BiTree t;
    int tag; //tag=0 表示左子女已被访问，tag=1 表示右子女已被访问
}stack;
stack s[],s1[];                                       //栈，容量足够大
BiTree Ancestor(BiTree ROOT,BiTNode *p,BiTNode *q){
//本算法求二叉树中 p 和 q 指向结点的最近公共结点
    top=0;bt=ROOT;
    while(bt!=NULL||top>0){
        while(bt!=NULL){
            s[++top].t=bt;
            s[top].tag=0;
            bt=bt->lchild;
            }                                         //沿左分支向下
        while(top!=0&&s[top].tag==1){
        //假定 p 在 q 的左侧，遇到 p 时，栈中元素均为 p 的祖先
            if(s[top].t==p){
                for(i=1;i<=top;i++)
                    s1[i]=s[i];
                    top1=top;
            }                                         //将栈 s 的元素转入辅助栈 s1 保存
            if(s[top].t==q)                           //找到 q 结点
                for(i=top;i>0;i--){//将栈中元素的树结点到 s1 中去匹配
                    for(j=top1;j>0;j--)
                        if(s1[j].t==s[i].t)
                            return  s[i].t; //p 和 q 的最近公共祖先已找到
                }
                top--;                                //退栈
        } //while
        if(top!=0){
            s[top].tag=1;
            bt=s[top].t->rchild;
            }                                         //沿右分支向下遍历
    } //while
    return  NULL;                                     //p 和 q 无公共祖先
}
```

11.【解答】

采用层次遍历的方法求出所有结点的层次，并将所有结点和对应的层次放在一个队列中。然后通过扫描队列求出各层的结点总数，最大的层结点总数即二叉树的宽度。算法实现如下：

```
typedef struct{
    BiTree data[MaxSize];          //保存队列中的结点指针
    int level[MaxSize];            //保存data中相同下标结点的层次
    int front,rear;
}Qu;
int BTWidth(BiTree b){
    BiTree p;
    int k,max,i,n;
    Qu.front=Qu.rear=-1;           //队列为空
    Qu.rear++;
    Qu.data[Qu.rear]=b;            //根结点指针入队
    Qu.level[Qu.rear]=1;           //根结点层次为1
    while(Qu.front<Qu.rear){
        Qu.front++;                //出队
        p=Qu.data[Qu.front];       //出队结点
        k=Qu.level[Qu.front];      //出队结点的层次
        if(p->lchild!=NULL){       //左孩子入队
            Qu.rear++;
            Qu.data[Qu.rear]=p->lchild;
            Qu.level[Qu.rear]=k+1;
        }
        if(p->rchild!=NULL){       //右孩子入队
            Qu.rear++;
            Qu.data[Qu.rear]=p->rchild;
            Qu.level[Qu.rear]=k+1;
        }
    }//while
    max=0;i=0;                     //max保存同一层最多的结点个数
    k=1;                           //k表示从第一层开始查找
    while(i<=Qu.rear){             //i扫描队中所有元素
        n=0;                       //n统计第k层的结点个数
        while(i<=Qu.rear&&Qu.level[i]==k){
            n++;
            i++;
        }
        k=Qu.level[i];
        if(n>max)   max=n;         //保存最大的n
    }
    return max;
}
```

注 意

本题队列中的结点，在出队后仍需要保留在队列中，以便求二叉树的宽度，所以设置的队列采用非环形队列，否则在出队后可能被其他结点覆盖，无法再求二叉树的宽度。

12.【解答】

对一般二叉树，仅根据先序或后序序列，不能确定另一个遍历序列。但对满二叉树，任意一个结点的左、右子树均含有相等的结点数，同时，先序序列的第一个结点作为后序序列的最后一个结点，由此得到将先序序列pre[l1..h1]转换为后序序列post[l2..h2]的递归模型如下：

f(pre,l1,h1,post,l2,h2)=不做任何事情　　h1<l1时

f(pre,l1,h1,post,l2, h2)=post[h2]=pre[l1]　　其他情况

取中间位置 half=(h1-l1)/2;

将 pre[l1+1,l1+half]左子树转换为 post[l2,l2+half-1],

即 f(pre,l1+1,l1+half,post,l2,l2+half-1);

将 pre[l1+half+1,h1]右子树转换为 post[l2+half,h2-1],

即 f(pre,l1+half+1,h1,post,l2+half,h2-1)。

其中，post[h2]=pre[l1]表示后序序列的最后一个结点（根结点）等于先序序列的第一个结点（根结点）。相应的算法实现如下：

```
void PreToPost(ElemType pre[],int l1,int h1,ElemType post[],int l2,int h2){
    int half;
    if(h1>=l1){
        post[h2]=pre[l1];
        half=(h1-l1)/2;
        PreToPost(pre,l1+1,l1+half,post,l2,l2+half-1);      //转换左子树
        PreToPost(pre,l1+half+1,h1,post,l2+half,h2-1);      //转换右子树
    }
}
```

例如，有以下代码：

```
ElemType *pre="ABCDEFG";
ElemType post[MaxSize];
PreToPost(pre,0,6,post,0,6);
printf("后序序列：");
for(int i=0;i<=6;i++)
    printf("%c",post[i]);
printf("\n");
```

执行结果如下：

```
后序序列：C D B F G E A
```

13.【解答】

通常使用的先序、中序和后序遍历对于叶结点的访问顺序都是从左到右，这里选择中序递归遍历。算法思想：设置前驱结点指针 pre，初始为空。第一个叶结点由指针 head 指向，遍历到叶结点时，就将它前驱的 rchild 指针指向它，最后一个叶结点的 rchild 为空。算法实现如下：

```
LinkedList head, pre=NULL;                          //全局变量
LinkedList InOrder(BiTree bt){
    if(bt){
        InOrder(bt->lchild);                        //中序遍历左子树
        if(bt->lchild==NULL&&bt->rchild==NULL)      //叶结点
            if(pre==NULL){
                head=bt;
                pre=bt;
            }                                       //处理第一个叶结点
            else{
                pre->rchild=bt;
                pre=bt;
            }                                       //将叶结点链入链表
        InOrder(bt->rchild);                        //中序遍历右子树
        pre->rchild=NULL;                           //设置链表尾
    }
    return head;
}
```

上述算法的时间复杂度为 $O(n)$，辅助变量使用 head 和 pre，栈空间复杂度为 $O(n)$。

14.【解答】

本题采用递归的思想求解，若 T_1 和 T_2 都是空树，则相似；若有一个为空另一个不空，则必然不相似；否则递归地比较它们的左、右子树是否相似。递归函数的定义如下：

1）f(T1,T2)=1；若 T1==T2==NULL。

2）f(T1,T2)=0；若 T1 和 T2 之一为 NULL，另一个不为 NULL。

3）f(T1,T2)=f(T1->lchild,T2->lchild)&&f(T1->rchild,T2->rchild)；若 T1 和 T2 均不为 NULL。

因此，算法实现如下：

```
int similar(BiTree T1,BiTree T2){
//采用递归的算法判断两棵二叉树是否相似
    int leftS,rightS;
    if(T1==NULL&&T2==NULL)          //两棵树皆空
        return 1;
    else if(T1==NULL||T2==NULL)     //只有一棵树为空
        return 0;
    else{                           //递归判断
        leftS=similar(T1->lchild,T2->lchild);
        rightS=similar(T1->rchild,T2->rchild);
        return leftS&&rightS;
    }
}
```

15.【解答】

二叉树的带权路径长度有两种常见的计算方法：①根据二叉树的带权路径长度的定义，二叉树的 WPL 值 = 树中全部叶结点的带权路径长度之和。②根据带权二叉树的性质，二叉树的 WPL 值 = 树中所有非叶结点的权值之和（记住该结论即可，不要求证明）。根据两种常见的计算方法，本题不难写出下列两种解法。

1）算法的基本设计思想。

① 本问题可采用递归算法实现。根据定义：

二叉树的 WPL 值 = 树中全部叶结点的带权路径长度之和

= 根结点左子树中全部叶结点的带权路径长度之和 +

根结点右子树中全部叶结点的带权路径长度之和

叶结点的带权路径长度 = 该结点的 weight 域的值×该结点的深度

设根结点的深度为 0，若某结点的深度为 d 时，则其孩子结点的深度为 $d+1$。

在递归遍历二叉树结点的过程中，若遍历到叶结点，则返回该结点的带权路径长度，否则返回其左右子树的带权路径长度之和。

② 若借用非叶结点的 weight 域保存其孩子结点中 weight 域值的和，则树的 WPL 等于树中所有非叶结点 weight 域值之和。

采用后序遍历策略，在遍历二叉树 T 时递归计算每个非叶结点的 weight 域的值，则树 T 的 WPL 等于根结点左子树的 WPL 加上右子树的 WPL，再加上根结点中 weight 域的值。在递归遍历二叉树结点的过程中，若遍历到叶结点，则 return 0 并且退出递归，否则递归计算其左右子树的 WPL 和自身结点的权值。

2）二叉树结点的数据类型定义如下。

```
typedef struct node{
    int weight;
    struct node *left,*right;
}BTree;
```

3）算法的代码如下。

①基于方法 1 的算法实现：

```
int WPL(BTree *root)                    //根据 WPL 的定义采用递归算法实现
{   return WPL1(root,0);
}
int WPL1(BTree *root,int d)             //d 为结点深度
{   if(root->left==NULL&&root->right==NULL)
    return(root->weight*d);
        else
            return (WPL1(root->left,d+1)+WPL1(root->right,d+1));
}
```

②基于方法 2 的算法实现：

```
int WPL(BTree *root)                    //基于递归的后序遍历算法实现
{   int w_l,w_r;
    if(root->left==NULL&&root->right==NULL)
        return 0;
    else
    {   w_l=WPL(root->left);            //计算左子树的 WPL
        w_r=WPL(root->right);           //计算右子树的 WPL
        root->weight=root->left->weight+root->right->weight;
                                        //填写非叶结点的 weight 域
        return (w_l+w_r+root->weight);  //返回 WPL 值
    }
}
```

注　意

上述两种算法为官方标准答案，当遍历到度为 1 的结点时，会传入空指针，导致空指针异常。但是，作为 408 考试的算法题，不要求考虑特殊的边界条件，只要算法思想正确，代码逻辑正确，即可得满分。因此，在复习过程中，无须花过多的时间抠代码的各种边界条件。

16.【解答】

1）算法的基本设计思想。

表达式树的中序序列加上必要的括号即等价的中缀表达式。可以基于二叉树的中序遍历策略得到所需的表达式。

表达式树中分支结点所对应的子表达式的计算次序，由该分支结点所处的位置决定。为得到正确的中缀表达式，需要在生成遍历序列的同时，在适当位置增加必要的括号。显然，表达式的最外层（对应根结点）和操作数（对应叶结点）不需要添加括号。

2）算法实现。

将二叉树的中序遍历递归算法稍加改造即可得本题的答案。除根结点和叶结点外，遍历到其他结点时在遍历其左子树之前加上左括号，遍历完右子树后加上右括号。

```
void BtreeToE(BTree *root){
    BtreeToExp(root,1);                 //根的高度为 1
}
void BtreeToExp( BTree *root,int deep){
    if(root==NULL) return;              //空结点返回
    else if(root->left==NULL&&root->right==NULL)   //若为叶结点
        printf("%s", root->data);       //输出操作数，不加括号
    else{
        if(deep>1) printf("(");         //若有子表达式则加一层括号
```

```
        BtreeToExp(root->left, deep+1);
        printf("%s", root->data);          //输出操作符
        BtreeToExp(root->right, deep+1);
        if(deep>1) printf(")");            //若有子表达式则加一层括号
    }
}
```

17.【解答1】

1）算法的基本设计思想。

对于采用顺序存储方式保存的二叉树，根结点保存在SqBiTNode[0]中；当某结点保存在SqBiTNode[i]中时，若有左孩子，则其值保存在SqBiTNode[2i+1]中；若有右孩子，则其值保存在SqBiTNode[2i+2]中；若有双亲结点，则其值保存在SqBiTNode[(i-1)/2]中。

二叉搜索树需要满足的条件是：任意一个结点值大于其左子树中的全部结点值，小于其右子树中的全部结点值。中序遍历二叉搜索树得到一个升序序列。

使用整型变量val记录中序遍历过程中已遍历结点的最大值，初值为一个负整数。若当前遍历的结点值小于或等于val，则算法返回false，否则，将val的值更新为当前结点的值。

2）算法实现。

```
#define false 0
#define true 1
typedef int bool;
bool judgeInOrderBST(SqBiTree bt,int k,int *val){//初始调用时k的值是0
    if(k<bt.ElemNum&&bt.SqBiTNode[k]!=-1){
        if(!judgeInOrderBST(bt,2*k+1,val)) return false;
        if(bt.SqBiTNode[k]<=*val) return false;
        *val=bt.SqBiTNode[k];
        if(!judgeInOrderBST(bt,2*k+2,val)) return false;
    }
    return true;
}
```

【解答2】

1）算法的基本设计思想。

对于采用顺序存储方式保存的二叉树，根结点保存在SqBiTNode[0]中；当某结点保存在SqBiTNode[i]中时，若有左孩子，则其值保存在SqBiTNode[2i+1]中；若有右孩子，则其值保存在SqBiTNode[2i+2]中；若有双亲结点，则其值保存在SqBiTNode[(i-1)/2]中。

二叉搜索树需要满足的条件是：任意一个结点值大于其左子树中的全部结点值，小于其右子树中的全部结点值。设置两个数组pmax和pmin。根据二叉搜索树的定义，SqBiTNode[i]中的值应该大于以SqBiTNode[2i+1]为根的子树中的最大值（保存在pmax[2i+1]中），小于以SqBiTNode[2i+2]为根的子树中的最小值（保存在pmin[2i+1]中）。初始时，用数组SqBiTNode中前ElemNum个元素的值对数组pmax和pmin初始化。

在数组SqBiTNode中从后向前扫描，扫描过程中逐一验证结点与子树之间是否满足上述的大小关系。

2）算法实现。

```
#define false 0
#define true 1
typedef int bool;
bool judgeBST(SqBiTree bt){
    int k,m,*pmin,*pmax;
    pmin=(int *)malloc(sizeof(int)*(bt.ElemNum));
    pmax=(int *)malloc(sizeof(int)*(bt.ElemNum));
    for(k=0;k<bt.ElemNum;k++)          //辅助数组初始化
```

```
            pmin[k]=pmax[k]=bt.SqBiTNode[k];
        for(k=bt.ElemNum-1;k>0;k--){      //从最后一个叶结点向根遍历
            if(bt.SqBiTNode[k]!=-1){
                m=(k-1)/2;                //双亲
                if(k%2==1&&bt.SqBiTNode[m]>pmax[k])          //其为左孩子
                    pmin[m]=pmin[k];
                else if(k%2==0&&bt.SqBiTNode[m]<pmin[k])     //其为右孩子
                    pmax[m]=pmax[k];
                else  return false;
            }
        }
        return true;
    }
```

5.4　树、森林

5.4.1　树的存储结构

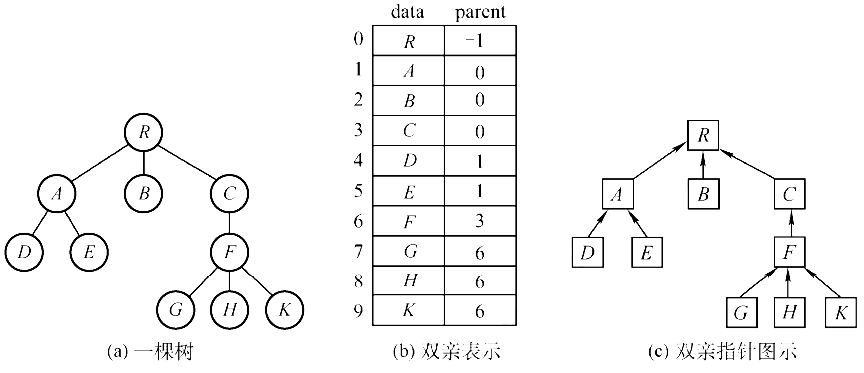
树的存储方式有多种，既可采用顺序存储结构，又可采用链式存储结构，但无论采用何种存储方式，都要求能唯一地反映树中各结点之间的逻辑关系，这里介绍 3 种常用的存储结构。

1．双亲表示法

这种存储结构采用一组连续空间来存储每个结点，同时在每个结点中增设一个伪指针，指示其双亲结点在数组中的位置。如图 5.20 所示，根结点下标为 0，其伪指针域为-1。

双亲表示法的存储结构描述如下：

```
#define MAX_TREE_SIZE 100          //树中最多结点数
typedef struct{                    //树的结点定义
    ElemType data;                 //数据元素
    int parent;                    //双亲位置域
}PTNode;
typedef struct{                    //树的类型定义
    PTNode nodes[MAX_TREE_SIZE];   //双亲表示
    int n;                         //结点数
}PTree;
```



(a) 一棵树　　(b) 双亲表示　　(c) 双亲指针图示

图 5.20　树的双亲表示法

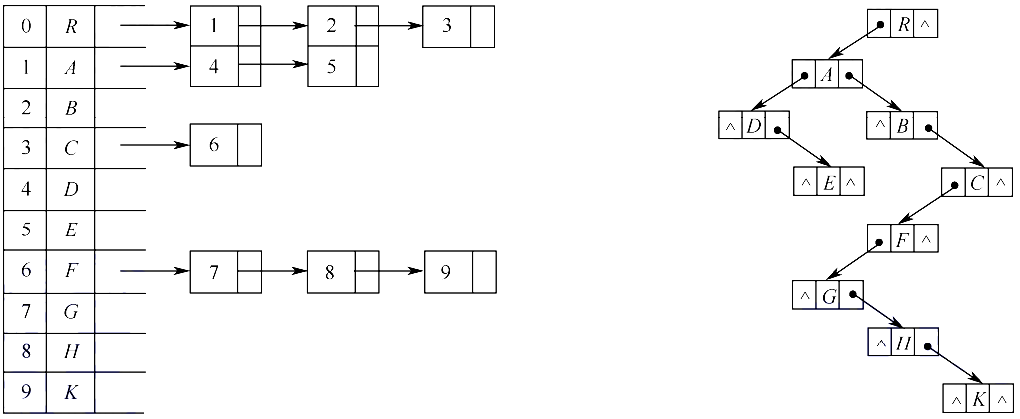
双亲表示法利用了每个结点（根结点除外）只有唯一双亲的性质，可以很快地得到每个结点的双亲结点，但求结点的孩子时则需要遍历整个结构。

注　意

区别树的顺序存储结构与二叉树的顺序存储结构。在树的顺序存储结构中，数组下标代表结点的编号，下标中所存的内容指示了结点之间的关系。而在二叉树的顺序存储结构中，数组下标既代表了结点的编号，又指示了二叉树中各结点之间的关系。当然，二叉树属于树，因此二叉树也可用树的存储结构来存储，但树却不都能用二叉树的存储结构来存储。

2．孩子表示法

孩子表示法是将每个结点的孩子结点视为一个线性表，且以单链表作为存储结构，则 n 个结点就有 n 个孩子链表（叶结点的孩子链表为空表）。而 n 个头指针又组成一个线性表，为便于查找，可采用顺序存储结构。图 5.21(a)是图 5.20(a)中的树的孩子表示法。



(a) 孩子表示法　　(b) 孩子兄弟表示法

图 5.21　树的孩子表示法和孩子兄弟表示法

与双亲表示法相反，孩子表示法寻找孩子的操作非常方便，而寻找双亲的操作则需要遍历 n 个结点中孩子链表指针域所指向的 n 个孩子链表。

3．孩子兄弟表示法

孩子兄弟表示法也称二叉树表示法，即以二叉链表作为树的存储结构。孩子兄弟表示法使每个结点包括三部分内容：结点值、指向结点第一个孩子结点的指针，以及指向结点下一个兄弟结点的指针（沿此域可以找到结点的所有兄弟结点），如图 5.21(b)所示。

孩子兄弟表示法的存储结构描述如下：

```
typedef struct CSNode{
    ElemType data;                                  //数据域
    struct CSNode *firstchild,*nextsibling;         //第一个孩子和右兄弟指针
}CSNode,*CSTree;
```

孩子兄弟表示法比较灵活，其最大的优点是可以方便地实现树转换为二叉树的操作，易于查找结点的孩子等，但缺点是从当前结点查找其双亲结点比较麻烦。若为每个结点增设一个 parent 域指向其父结点，则查找结点的父结点也很方便。

5.4.2　树、森林与二叉树的转换

二叉树和树都可以用二叉链表作为存储结构。从物理结构上看，树的孩子兄弟表示法与二叉树的二叉链表表示法是相同的，因此可以用同一存储结构的不同解释将一棵树转换为二叉树。

1．树转换为二叉树

命题追踪 ▶ 树和二叉树的转换及相关性质的推理（2009、2011）

树转换为二叉树的规则：每个结点的左指针指向它的第一个孩子，右指针指向它在树中的相邻右兄弟，这个规则也称*左孩子右兄弟*。根结点没有兄弟，因此树转换得到的二叉树没有右子树，如图 5.22 所示。

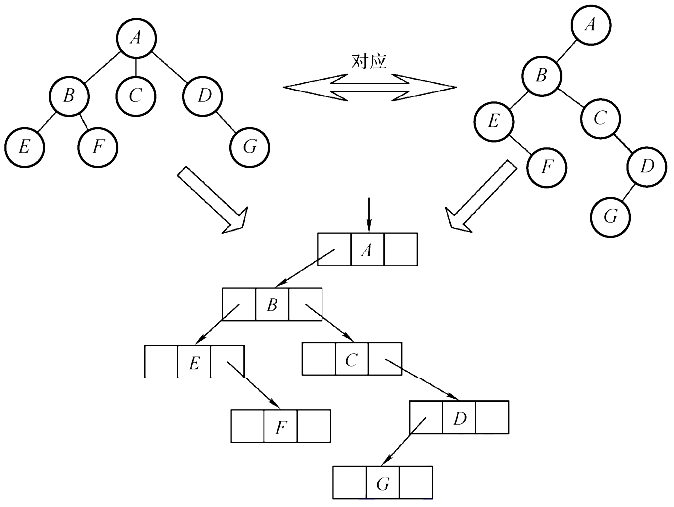


图 5.22　树与二叉树的对应关系

树转换为二叉树的画法：

1）在兄弟结点之间加一连线；

2）对每个结点，只保留它与第一个孩子的连线，而与其他孩子的连线全部抹掉；

3）以树根为轴心，顺时针旋转 45°。

2．森林转换为二叉树

命题追踪 ▶ 森林和二叉树的转换及相关性质的推理（2014）

将森林转换为二叉树的规则与树类似。先将森林中的每棵树转换为二叉树，由于任意一棵树对应的二叉树的右子树必空，森林中各棵树的根也可视为兄弟关系，将第二棵树对应的二叉树当作第一棵二叉树根的右子树……以此类推，就可以将森林转换为二叉树。

森林转换为二叉树的画法：

1）将森林中的每棵树转换成相应的二叉树；

2）每棵树的根也可视为兄弟关系，在每棵树的根之间加一根连线；

3）以第一棵树的根为轴心顺时针旋转 45°。

或者先在森林中每棵树的根之间加一根连线，然后再采用树转换为二叉树的方法。

3．二叉树转换为森林

命题追踪 ▶ 由遍历序列构造一棵二叉树并转换为对应的森林（2020、2021）

二叉树转换为森林的规则：若二叉树非空，则二叉树的根及其左子树为第一棵树的二叉树形式，所以将根的右链断开。二叉树根的右子树又可视为一个由除第一棵树外的森林转换后的二叉树，应用同样的方法，直到最后只剩一棵没有右子树的二叉树为止，最后将每棵二叉树依次转换

成树，就得到了原森林，如图 5.23 所示。二叉树转换为树或森林是唯一的。

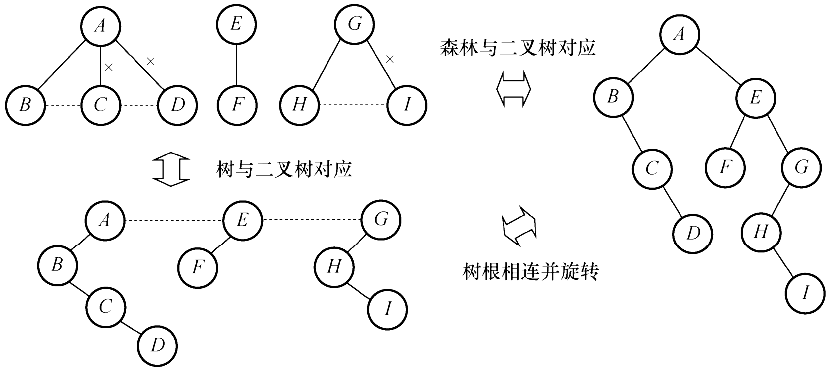


图 5.23　森林与二叉树的对应关系

5.4.3　树和森林的遍历

1．树的遍历

命题追踪 ▶ 树与二叉树遍历方法的对应关系（2019）

树的遍历是指用某种方式访问树中的每个结点，且仅访问一次。主要有两种方式：

1）先根遍历。若树非空，则按如下规则遍历：

- 先访问根结点。
- 再依次遍历根结点的每棵子树，遍历子树时仍遵循先根后子树的规则。

其遍历序列与这棵树相应二叉树的先序序列相同。

2）后根遍历。若树非空，则按如下规则遍历：

- 先依次遍历根结点的每棵子树，遍历子树时仍遵循先子树后根的规则。
- 再访问根结点。

其遍历序列与这棵树相应二叉树的中序序列相同。

图 5.22 的树的先根遍历序列为 *ABEFCDG*，后根遍历序列为 *EFBCGDA*。

另外，树也有层次遍历，与二叉树的层次遍历思想基本相同，即按层序依次访问各结点。

2．森林的遍历

按照森林和树相互递归的定义，可得到森林的两种遍历方法。

1）先序遍历森林。若森林为非空，则按如下规则遍历：

- 访问森林中第一棵树的根结点。
- 先序遍历第一棵树中根结点的子树森林。
- 先序遍历除去第一棵树之后剩余的树构成的森林。

2）中序遍历森林。森林为非空时，按如下规则遍历：

- 中序遍历森林中第一棵树的根结点的子树森林。
- 访问第一棵树的根结点。
- 中序遍历除去第一棵树之后剩余的树构成的森林。

图 5.23 的森林的先序遍历序列为 *ABCDEFGHI*，中序遍历序列为 *BCDAFEHIG*。

命题追踪 ▶ 森林与二叉树遍历方法的对应关系（2020）

当森林转换成二叉树时，其第一棵树的子树森林转换成左子树，剩余树的森林转换成右子树，

可知森林的先序和中序遍历即其对应二叉树的先序和中序遍历。

树和森林的遍历与二叉树的遍历关系见表 5.1。

表 5.1　树和森林的遍历与二叉树遍历的对应关系

树	森　林	二　叉　树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中序遍历

注　意

森林的遍历方法的命名按照严蔚敏老师的经典教材。有些教材也将森林的中序遍历称为后序遍历，称中序遍历是相对其二叉树而言的，称后序遍历是因为根确实是最后才访问。

5.4.4　本节试题精选

一、单项选择题

01. 下列关于树的说法中，正确的是（　）。

I. 对于有 n 个结点的二叉树，其高度为 $\log_2 n$

II. 完全二叉树中，若一个结点没有左孩子，则它必是叶结点

III. 高度为 h（$h>0$）的完全二叉树对应的森林所含的树的个数一定是 h

IV. 一棵树中的叶子数一定等于与其对应的二叉树的叶子数

A. I 和 III　　B. IV　　C. I 和 II　　D. II

02. 利用二叉链表存储森林时，根结点的右指针是（　）。

A. 指向最左兄弟　B. 指向最右兄弟　C. 一定为空　　D. 不一定为空

03. 设森林 F 中有 3 棵树，第 1、2、3 棵树的结点个数分别为 M_1, M_2 和 M_3，与森林 F 对应的二叉树根结点的右子树上的结点个数是（　）。

A. M_1　　B. M_1+M_2　　C. M_3　　D. M_2+M_3

04. 设森林 F 中有 4 棵树，第 1、2、3、4 棵树的结点数分别为 a、b、c 和 d，与森林 F 对应的二叉树的根结点的左子树上的结点数是（　）。

A. a　　B. $b+c+d$　　C. $a-1$　　D. $a+b+c$

05. 设森林 F 对应的二叉树为 B，它有 m 个结点，B 的根为 p，p 的右子树结点数为 n，森林 F 中第一棵树的结点数是（　）。

A. $m-n$　　B. $m-n-1$　　C. $n+1$　　D. 条件不足，无法确定

06. 设森林 F 对应的二叉树是一棵具有 16 个结点的完全二叉树，则森林 F 中树的数目和结点最多的树的结点数分别是（　）。

A. 2 和 8　　B. 2 和 9　　C. 4 和 8　　D. 4 和 9

07. 森林 $T=(T_1, T_2, \cdots, T_m)$ 转化为二叉树 BT 的过程为：若 $m=0$，则 BT 为空，若 $m\neq 0$，则（　）。

A. 将中间子树 T_{mid}（$\text{mid}=(1+m)/2$）的根作为 BT 的根；将$(T_1, T_2, \cdots, T_{\text{mid}-1})$转换为 BT 的左子树；将$(T_{\text{mid}+1}, \cdots, T_m)$转换为 BT 的右子树

B. 将子树 T_1 的根作为 BT 的根；将 T_1 的子树森林转换成 BT 的左子树；将$(T_2, T_3, \cdots, T_m)$转换成 BT 的右子树

C. 将子树 T_1 的根作为 BT 的根；将 T_1 的左子树森林转换成 BT 的左子树；将 T_1 的右子树森林转换为 BT 的右子树；其他以此类推

D. 将森林 T 的根作为 BT 的根；将$(T_1, T_2, \cdots, T_m)$转化为该根下的结点，得到一棵树，然后将这棵树再转化为二叉树 BT

08. 设 F 是一个森林，B 是由 F 变换来的二叉树。若 F 中有 n 个非终端结点，则 B 中右指针域为空的结点有（ ）个。

A. $n-1$　　B. n　　C. $n+1$　　D. $n+2$

09. 设某树的孩子兄弟链表示中共有 6 个空的左指针域、7 个空的右指针域，包括 5 个结点的左、右指针域都为空，则该树中叶结点的个数是（ ）。

A. 7　　B. 6　　C. 5　　D. 不能确定

10. 若 T_1 是由有序树 T 转换而来的二叉树，则 T 中结点的后根序列就是 T_1 中结点的（ ）序列。

A. 先序　　B. 中序　　C. 后序　　D. 层序

11. 某二叉树结点的中序序列为 $BDAECF$，后序序列为 $DBEFCA$，则该二叉树对应的森林包括（ ）棵树。

A. 1　　B. 2　　C. 3　　D. 4

12. 设 X 是树 T 中的一个非根结点，B 是 T 所对应的二叉树。在 B 中，X 是其双亲结点的右孩子，下列结论中正确的是（ ）。

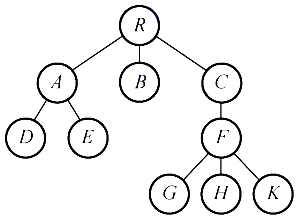
A. 在树 T 中，X 是其双亲结点的第一个孩子

B. 在树 T 中，X 一定无右边兄弟

C. 在树 T 中，X 一定是叶结点

D. 在树 T 中，X 一定有左边兄弟

13. 下图是一棵逻辑上的树 T，则在关于该树的存储结构的叙述中，错误的是（ ）。



A. 若 T 采用双亲表示法，则有 9 个指向双亲的指针

B. 若 T 采用孩子表示法，则在 T 中查找某个结点的孩子比双亲表示法更方便

C. 若 T 采用孩子兄弟表示法，则在 T 中查找某个结点的双亲的时间复杂度为 $O(1)$

D. 双亲表示法是顺序存储结构，孩子表示法和孩子兄弟表示法是链式存储结构

14. 在森林的二叉树表示中，结点 M 和结点 N 是同一父结点的左儿子和右儿子，则在该森林中（ ）。

A. M 和 N 有同一双亲　　B. M 和 N 可能无公共祖先

C. M 是 N 的儿子　　D. M 是 N 的左兄弟

15. 【2009 统考真题】将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 的父结点的父结点，则在原来的森林中，u 和 v 可能具有的关系是（ ）。

I. 父子关系　II. 兄弟关系　III. u 的父结点与 v 的父结点是兄弟关系

A. 只有 II　　B. I 和 II　　C. I 和 III　　D. I、II 和 III

16. 【2011 统考真题】已知一棵有 2011 个结点的树，其叶结点个数为 116，该树对应的二叉树中无右孩子的结点个数是（ ）。

A. 115　　B. 116　　C. 1895　　D. 1896

17. 【2014 统考真题】将森林 F 转换为对应的二叉树 T，F 中叶结点的个数等于（ ）。

A. T 中叶结点的个数　　B. T 中度为 1 的结点个数

C. T 中左孩子指针为空的结点个数　　D. T 中右孩子指针为空的结点个数

18.【2019 统考真题】若将一棵树 T 转化为对应的二叉树 BT，则下列对 BT 的遍历中，其遍历序列与 T 的后根遍历序列相同的是（　）。

A. 先序遍历　　B. 中序遍历

C. 后序遍历　　D. 按层遍历

19.【2020 统考真题】已知森林 F 及与之对应的二叉树 T，若 F 的先根遍历序列是 a, b, c, d, e, f，后根遍历序列是 b, a, d, f, e, c，则 T 的后序遍历序列是（　）。

A. b, a, d, f, e, c　　B. b, d, f, e, c, a

C. b, f, e, d, c, a　　D. f, e, d, c, b, a

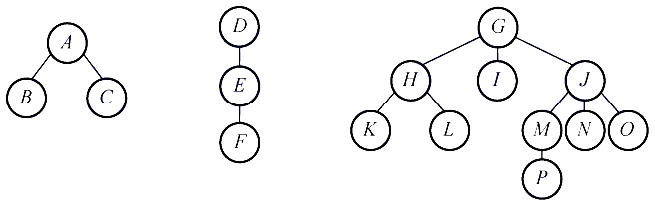
20.【2021 统考真题】某森林 F 对应的二叉树为 T，若 T 的先序遍历序列是 a, b, d, c, e, g, f，中序遍历序列是 b, d, a, e, g, c, f，则 F 中树的棵数是（　）。

A. 1　　B. 2　　C. 3　　D. 4

二、综合应用题

01. 给定一棵树的先根遍历序列和后根遍历序列，能否唯一确定一棵树？若能，请举例说明；若不能，请给出反例。

02. 将下面一个由 3 棵树组成的森林转换为二叉树。



03. 已知某二叉树的先序序列和中序序列分别为 *ABDEHCFIMGJKL* 和 *DBHEAIM FCGKLJ*，请画出这棵二叉树，并画出二叉树对应的森林。

04. 编程求以孩子兄弟表示法存储的森林的叶结点数。

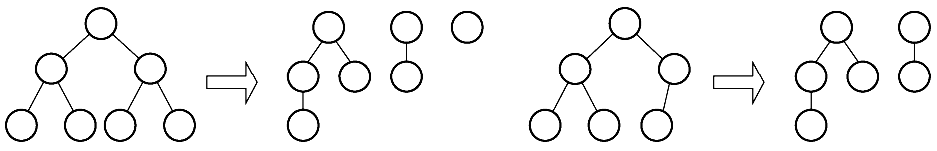
05. 以孩子兄弟链表为存储结构，请设计递归算法求树的深度。

5.4.5　答案与解析

一、单项选择题

01. D

若 n 个结点的二叉树是一棵单支树，则其高度为 n。完全二叉树中最多存在一个度为 1 的结点且只有左孩子，若不存在左孩子，则一定也不存在右孩子，因此必是叶结点，选项 II 正确。只有满二叉树才具有性质 III，如下图所示。



(a) 满二叉树转化为对应的森林　　(b) 非满二叉树转化为对应的森林

在树转换为二叉树时，若有几个叶结点具有共同的双亲，则转换成二叉树后只有一个叶结点（最右边的叶结点），如下图所示，选项 IV 错误。注意，若树中的任意两个叶结点都不存在相同

的双亲，则树中的叶子数才有可能与其对应的二叉树中的叶子数相等。

02．D

森林与二叉树具有对应关系，因此，我们存储森林时应先将森林转换成二叉树，转换的方法就是“左孩子右兄弟”，与树不同的是，若存在第二棵树，则二叉链表的根结点的右指针指向的是森林中的第二棵树的根结点。若此森林只有一棵树，则根结点的右指针为空。因此，右指针可能为空也可能不为空。

03．D

与树转换为二叉树不同，森林中的每棵树是独立的，因此先要将每棵树的根结点全部视为兄弟结点的关系。森林转换为二叉树后，树 2 作为树 1 的根结点的右子树，树 3 作为树 2 的根结点的右子树，因此森林 F 对应的二叉树根结点的右子树上的结点个数是 M_2+M_3。

04．C

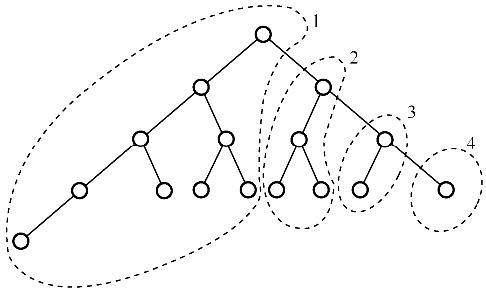
森林转换为二叉树后，二叉树的根结点为第 1 棵树的根结点，二叉树的根结点的左子树包含第 1 棵树的所有孩子，因此森林 F 对应的二叉树的根结点的左子树上的结点数是 $a-1$。

05．A

森林转换成二叉树时采用孩子兄弟表示法，根结点及其左子树为森林中的第一棵树。右子树为其他剩余的树。所以，第一棵树的结点个数为 $m-n$。

06．D

森林转换为二叉树后，二叉树的根结点及其左子树由第 1 棵树转换得到，二叉树的根结点的右子树由剩余的森林转换得到，以此类推，可以划分出第 2, 3,… 棵树的结点。具有 16 个结点的完全二叉树的形态如下图所示，沿二叉树的根结点往右下遍历，共有 4 个结点，可知森林中有 4 棵树，其中第 1 棵树的结点数最多，有 9 个。



07．B

将森林中每棵树的根结点视为兄弟结点的关系，再按照“左孩子右兄弟”的规则来进行转化。

08．C

根据森林与二叉树转换规则“左孩子右兄弟”。二叉树 B 中右指针域为空代表该结点没有兄弟结点。森林中每棵树的根结点从第二个开始依次连接到前一棵树的根的右孩子，因此最后一棵树的根结点的右指针为空。另外，每个非终端结点，其所有孩子结点在转换之后，最后一个孩子的右指针也为空，所以树 B 中右指针域为空的结点有 $n+1$ 个。

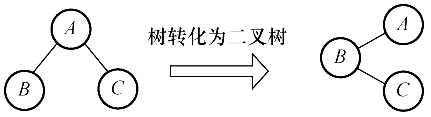
09．B

在树的孩子兄弟表示法中，若一个结点没有孩子（叶结点），则表现为该结点的左指针域为

空，因此本题答案为“6”。至于“5 个结点的左、右指针域都为空”，表示树中有 5 个结点既没有孩子又没有右兄弟，约束条件比题中的“求叶结点的个数”要求更严格。

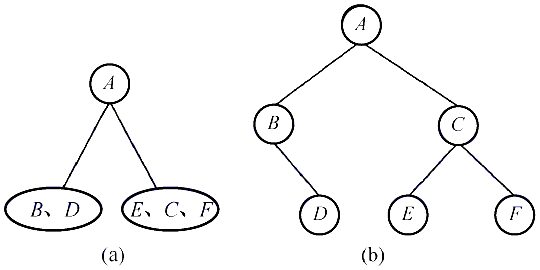
10．B

有序树 T 转换成二叉树 T_1 时，T 的后根序列是对应 T_1 的中序序列还是后序序列呢（显然树的后根序列不可能对应二叉树的先序序列和层序序列）？看下图所示的例子，在树 T 中，叶结点 B 应最先访问，在 T_1 中，B 的右兄弟 C 转换为它的右孩子，若对应 T_1 的后序序列，则 C 应在 B 的前面访问，所以 T 的后根序列不可能对应 T_1 的后序序列。



11．C

根据二叉树的前序序列和中序序列可以确定一棵二叉树。根据后序序列，A 是二叉树的根结点。根据中序序列，二叉树的形态如下图(a)所示。对于 A 的左子树，根据后序序列，B 比 D 后被访问，因此 B 必为 D 的父结点，又根据中序序列，D 是 B 的右儿子。对于 A 的右子树，同理可确定结点 E、C、F 的关系。此二叉树的形态如下图(b)所示。



再根据二叉树与森林的对应关系，森林中树的棵数即其对应二叉树（向右上旋转 45°后）的根结点 A 及其右兄弟数，或解释为：对应二叉树从根结点 A 开始不断往右孩子访问，所访问到的结点数。可知此森林中有 3 棵树，根结点分别为 A, C 和 F。

12．D

在二叉树 B 中，X 是其双亲的右孩子，因此在树 T 中，X 必是其双亲结点的右兄弟，换句话说，X 在树中必有左兄弟。

13．C

若 T 采用双亲表示法存储，则除根结点外，其余每个结点都有指向其双亲的指针，T 共有 10 个结点，于是有 9 个指向双亲的指针，选项 A 正确。若 T 采用孩子表示法存储，则每个结点的孩子被视为一个线性表，且以单链表作为存储结构，只要遍历该单链表，就能找到某个结点的所有孩子，而双亲表示法要寻找某个结点的孩子，就必须遍历整棵树，选项 B 正确。若 T 采用孩子兄弟表示法，则在 T 中查找某个结点的双亲也必须遍历整棵树，时间复杂度为 $O(n)$，选项 C 错误。选项 D 显然正确。

14．B

在森林的二叉树表示中，当 M 和 N 的父结点是二叉树根结点时，M 和 N 在不同的树上。因此 M 和 N 可能无公共祖先。

15．B

森林与二叉树的转换规则为“左孩子右兄弟”。在最后生成的二叉树中，父子关系在对应森林关系中可能是兄弟关系或者原本就是父子关系。

情形 I：若结点 v 是结点 u 的第二个孩子结点，转换时，结点 v 就变成结点 u 第一个孩子的右孩子，符合要求。情形 II：结点 u 和 v 是兄弟结点的关系，但二者之中还有一个兄弟结点 k，则转换后结点 v 就变为结点 k 的右孩子，而结点 k 则是结点 u 的右孩子，符合要求。

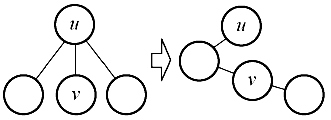


图 I

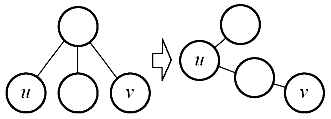


图 II

情形 III：若结点 u 的父结点与 v 的父结点是兄弟关系，则转换后，结点 u 和 v 分别在两者最左父结点的两棵子树中，不可能出现在同一条路径中。

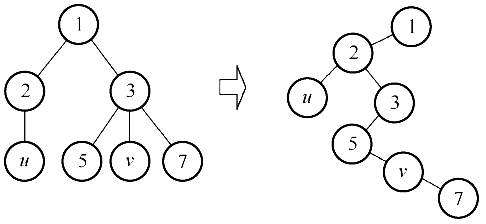
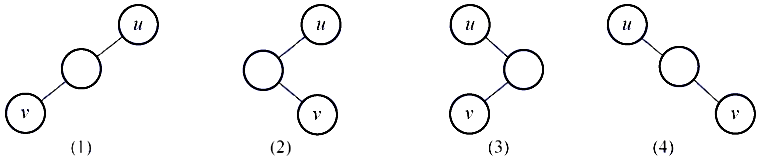


图 III

【另解】由题意可知 u 是 v 的父结点的父结点，如下图所示，有四种情况：

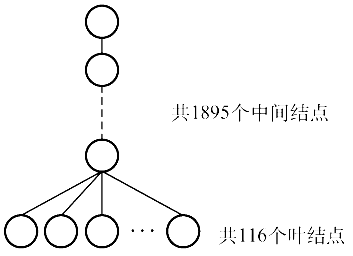


根据树与二叉树的转换规则，将这四种情况转换成树中结点的关系。（1）在原来的树中 u 是 v 的父结点的父结点；（2）在树中 u 是 v 的父结点；（3）在树中 u 是 v 的父结点的兄弟；（4）在树中 u 与 v 是兄弟关系。由此可知选项 I 和 II 正确。

16．D

树转换为二叉树时，树的每个分支结点的所有子结点中的最右子结点无右孩子，根结点转换后也没有右孩子，因此，对应二叉树中无右孩子的结点个数＝分支结点数＋1＝2011－116＋1＝1896。

通常本题应采用特殊法求解，设题意中的树是如下图所示的结构，则对应的二叉树中仅有前 115 个叶结点有右孩子，所以无右孩子的结点个数＝2011－115＝1896。

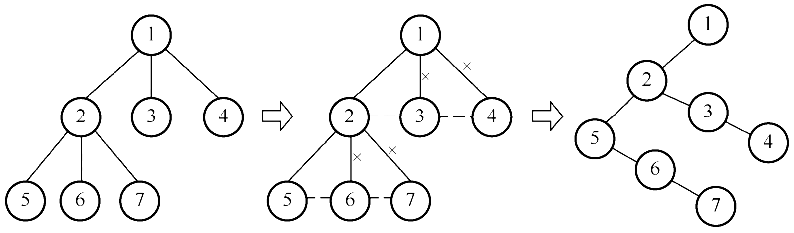


17．C

将森林转化为二叉树相当于用孩子兄弟表示法来表示森林。在变化过程中，原森林某结点的第一个孩子结点作为它的左子树，它的兄弟作为它的右子树。森林中的叶结点没有孩子结点，转化为二叉树时，该结点就没有左结点，因此 F 中叶结点的个数等于 T 中左孩子指针为空的结点个数。此题还可通过一些特例来排除选项 A、B 和 D。

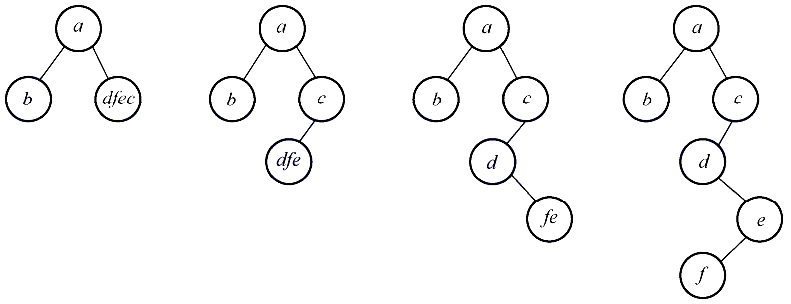
18．B

后根遍历树可分为两步：①从左到右访问双亲结点的每个孩子（转化为二叉树后，先访问根结点，再访问右子树）；②访问完所有孩子后再访问它们的双亲结点（转化为二叉树后，先访问左子树，再访问根结点），因此树的后根遍历序列与其相应二叉树的中序遍历序列相同。对于此类题，采用特殊值法求解通常会更便捷，左下图树 T 转换为二叉树 BT 的过程如下图所示，树的后序遍历序列显然和其相应二叉树的中序遍历序列相同，均为 5,6,7,2,3,4,1。



19．C

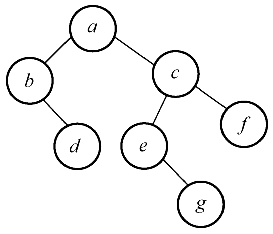
森林 F 的先根遍历序列对应于其二叉树 T 的先序遍历序列，森林 F 的后根遍历序列对应于其二叉树 T 的中序遍历序列。即 T 的先序遍历序列为 a, b, c, d, e, f，中序遍历序列为 b, a, d, f, e, c。根据二叉树 T 的先序序列和中序序列可以唯一确定它的结构，构造过程如下：



可以得到二叉树 T 的后序序列为 b, f, e, d, c, a。

20．C

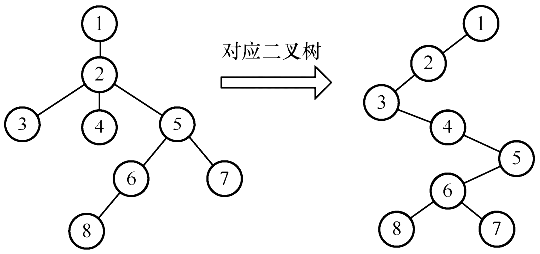
由二叉树 T 的先序序列和中序序列可以构造出 T，如下图所示。由森林转化成二叉树的规则可知，森林中每棵树的根结点以右子树的方式相连，所以 T 中的结点 a、c、f 为 F 中树的根结点，森林 F 中有 3 棵树。



二、综合应用题

01．【解答】

一棵树的先根遍历结果与其对应二叉树的先序遍历结果相同，树的后根遍历结果与其对应二叉树表示的中序遍历结果相同。二叉树的先序序列和中序序列能够唯一地确定这棵二叉树，因此，根据题目给出的条件，利用树的先根遍历序列和后根遍历序列能够唯一地确定这棵树。例如，对于下图所示的树，对应二叉树的先序序列为 1, 2, 3, 4, 5, 6, 8, 7，中序序列为 3, 4, 8, 6, 7, 5, 2, 1。原树的先根遍历序列为 1, 2, 3, 4, 5, 6, 8, 7，后根遍历序列为 3, 4, 8, 6, 7, 5, 2, 1。

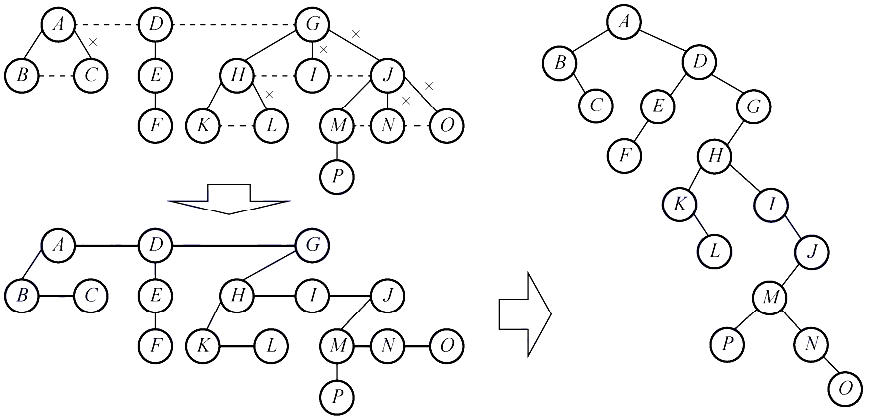


注 意

树的先根遍历、后根遍历与对应二叉树的前序遍历、中序遍历对应。

02.【解答】

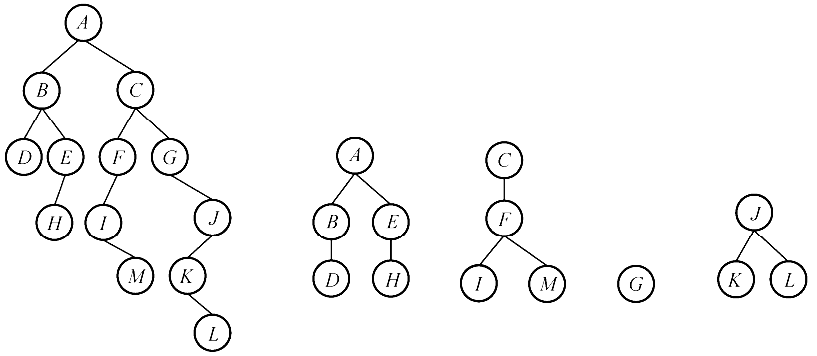
根据树与二叉树“左孩子右兄弟”的转换规则，将森林转换为二叉树的过程如下：①将每棵树的根结点也视为兄弟关系，在兄弟结点之间加一连线。②对每个结点，只保留它与第一个子结点的连线，与其他子结点的连线全部抹掉。③以树根为轴心，顺时针旋转 45°。结果如下图所示。



03.【解答】

知道二叉树的先序和中序遍历后，可以唯一确定这棵树的结构。然后把二叉树转换到树和森林的方式是，若结点 x 是双亲 y 的左孩子，则把 x 的右孩子、右孩子的右孩子……都与 y 用连线连起来，最后去掉所有双亲到右孩子的连线。

最后得到的二叉树及对应的森林如下图所示。



04.【解答】

当森林（树）以孩子兄弟表示法存储时，若结点没有孩子（`fch=NULL`），则它必是叶子，总的叶结点个数是孩子子树（`fch`）上的叶子数和兄弟子树（`nsib`）上的叶结点个数之和。

算法代码如下：

```
typedef struct node
{
    ElemType    data;                  //数据域
    struct node *fch,*nsib;            //孩子与兄弟域
}*Tree;
int  Leaves(Tree t){                   //计算以孩子兄弟表示法存储的森林的叶子数
    if(t==NULL)
        return  0;                     //树空返回 0
    if(t->fch==NULL)                   //若结点无孩子，则该结点必是叶子
        return  1+Leaves(t->nsib);     //返回叶结点和其兄弟子树中的叶结点数
    else                               //孩子子树和兄弟子树中叶子数之和
        return  Leaves(t->fch)+Leaves(t->nsib);
}
```

05.【解答】

由孩子兄弟链表表示的树，求高度的算法思想：采用递归算法，若树为空，高度为零；否则，高度为第一子女树高度加 1 和兄弟子树高度的大者。其非递归算法使用队列，逐层遍历树，取得树的高度。算法代码如下：

```
int  Height(CSTree bt){
//递归求以孩子兄弟链表表示的树的深度
    int hc,hs;
    if(bt==NULL)
        return  0;
    else{//否则，高度取子女高度+1 和兄弟子树高度的大者
        hc=Height(bt->firstchild);      //第一子女树高
        hs=Height(bt->nextsibling);     //兄弟树高
        if(hc+1>hs)
            return  hc+1;
        else
            return  hs;
    }
}
```

5.5　树与二叉树的应用

5.5.1　哈夫曼树和哈夫曼编码

1．哈夫曼树的定义

在介绍哈夫曼树之前，先介绍几个相关的概念：

在许多应用中，树中结点常常被赋予一个表示某种意义的数值，称为该结点的权。

从树的根到一个结点的路径长度与该结点上权值的乘积，称为该结点的带权路径长度。

树中所有叶结点的带权路径长度之和称为该树的带权路径长度，记为

$$\mathrm{WPL}=\sum_{i=1}^{n} w_i l_i$$

式中，w_i 是第 i 个叶结点所带的权值，l_i 是该叶结点到根结点的路径长度。

在含有 n 个带权叶结点的二叉树中，其中带权路径长度（WPL）最小的二叉树称为哈夫曼树，也称最优二叉树。例如，图 5.24 中的 3 棵二叉树都有 4 个叶结点 a, b, c, d，分别带权 7, 5, 2, 4，它们的带权路径长度分别为

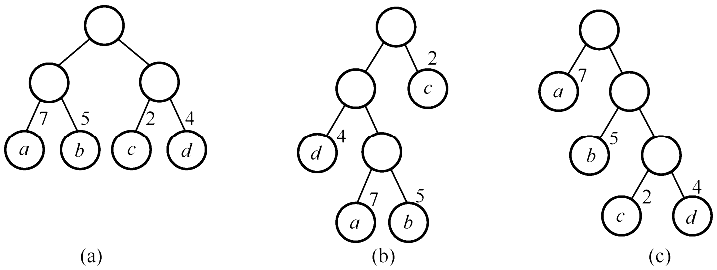


图 5.24　具有不同带权长度的二叉树

(a) WPL = 7×2 + 5×2 + 2×2 + 4×2 = 36。

(b) WPL = 4×2 + 7×3 + 5×3 + 2×1 = 46。

(c) WPL = 7×1 + 5×2 + 2×3 + 4×3 = 35。

其中，图 5.24(c)树的 WPL 最小。可以验证，它恰好为哈夫曼树。

2．哈夫曼树的构造

给定 n 个权值分别为 $w_1, w_2, \cdots, w_n$ 的结点，构造哈夫曼树的算法描述如下：

1）将这 n 个结点分别作为 n 棵仅含一个结点的二叉树，构成森林 F。

命题追踪　分析哈夫曼树的路径上权值序列的合法性（2010）

2）构造一个新结点，从 F 中选取两棵根结点权值最小的树作为新结点的左、右子树，并且将新结点的权值置为左、右子树上根结点的权值之和。公众号：小兔网盘 免费分享无水印PDF

3）从 F 中删除刚才选出的两棵树，同时将新得到的树加入 F 中。

4）重复步骤 2）和 3），直至 F 中只剩下一棵树为止。

命题追踪　哈夫曼树的性质（2010、2019）

从上述构造过程中可以看出哈夫曼树具有如下特点：

1）每个初始结点最终都成为叶结点，且权值越小的结点到根结点的路径长度越大。

2）构造过程中共新建了 $n-1$ 个结点（双分支结点），因此哈夫曼树的结点总数为 $2n-1$。

3）每次构造都选择 2 棵树作为新结点的孩子，因此哈夫曼树中不存在度为 1 的结点。

例如，权值{7, 5, 2, 4}的哈夫曼树的构造过程如图 5.25 所示。

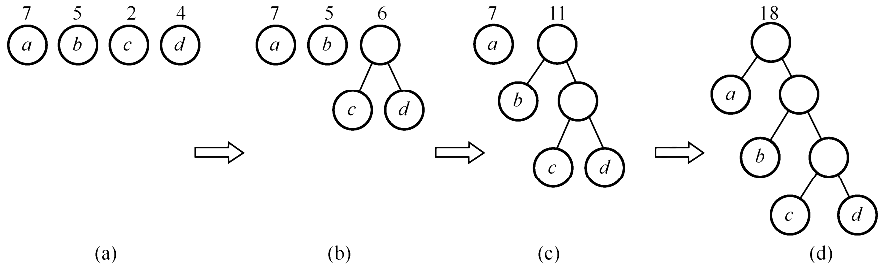


图 5.25　哈夫曼树的构造过程

3．哈夫曼编码

在数据通信中，若对每个字符用相等长度的二进制位表示，称这种编码方式为固定长度编码。若允许对不同字符用不等长的二进制位表示，则这种编码方式称为可变长度编码。可变长度编码比固定长度编码要好得多，其特点是对频率高的字符赋以短编码，而对频率较低的字符则赋以较长一些的编码，从而可以使字符的平均编码长度减短，起到压缩数据的效果。

命题追踪 ▶ 根据哈夫曼编码对编码序列进行译码（2017）

若没有一个编码是另一个编码的前缀，则称这样的编码为前缀编码。举例：设计字符 A, B 和 C 对应的编码 0, 10 和 110 是前缀编码。对前缀编码的解码很简单，因为没有一个编码是其他编码的前缀。所以识别出第一个编码，将它翻译为原字符，再对剩余的码串执行同样的解码操作。例如，码串 0010110 可被唯一地翻译为 A, A, B 和 C。另举反例：若再将字符 D 的编码设计为 11，此时 11 是 110 的前缀，则上述码串的后三位就无法唯一翻译。

命题追踪 ▶ 哈夫曼树的构造及相关的分析（2012、2018、2021、2023）

命题追踪 ▶ 前缀编码的分析及应用（2014、2020）

可以利用二叉树来设计二进制前缀编码。假设为 A, B, C, D 四个字符设计前缀编码，可以用图 5.26 所示的二叉树来表示，4 个叶结点分别表示 4 个字符，且约定左分支表示 0，右分支表示 1，从根到叶结点的路径上用分支标记组成的序列作为该叶结点字符的编码，可以证明如此得到的必为前缀编码。由图 5.26 得到字符 A, B, C, D 的前缀编码分别为 0, 10, 110, 111。

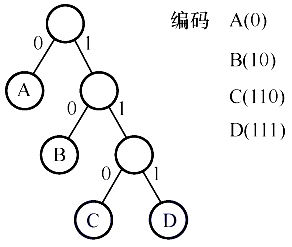


图 5.26　前缀编码示例

命题追踪 ▶ 哈夫曼编码和定长编码的差异（2022）

哈夫曼编码是一种非常有效的数据压缩编码。由哈夫曼树得到哈夫曼编码是很自然的过程。首先，将每个字符当作一个独立的结点，其权值为它出现的频度（或次数），构造出对应的哈夫曼树。然后，将从根到叶结点的路径上分支标记的字符串作为该字符的编码。图 5.27 所示为一个由哈夫曼树构造哈夫曼编码的示例，矩形方块表示字符及其出现的次数。

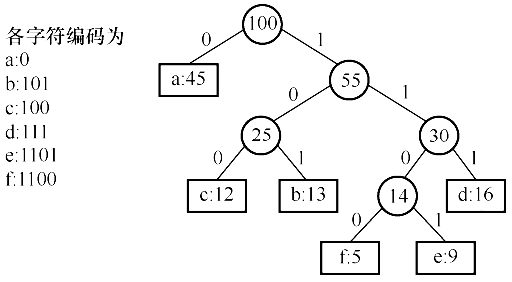


图 5.27　由哈夫曼树构造哈夫曼编码

这棵哈夫曼树的 WPL 为

$$\mathrm{WPL} = 1\times45 + 3\times(13 + 12 + 16) + 4\times(5 + 9) = 224$$

此处的 WPL 可视为最终编码得到二进制编码的长度，共 224 位。若采用 3 位固定长度编码，则得到的二进制编码长度为 300 位，因此哈夫曼编码共压缩了 25%的数据。利用哈夫曼树可以设计出总长度最短的二进制前缀编码。

注　意

左分支和右分支究竟是表示 0 还是表示 1 没有明确规定，因此构造出的哈夫曼树并不唯一，但各哈夫曼树的带权路径长度 WPL 相同且为最优。此外，如有若干权值相同的结点，则构造出的哈夫曼树更可能不同，但 WPL 必然相同且为最优。

5.5.2　并查集

1．并查集的概念

并查集是一种简单的集合表示，它支持以下 3 种操作：

1）`Initial(S)`：将集合 `S` 中的每个元素都初始化为只有一个单元素的子集合。

2）`Union(S,Root1,Root2)`：把集合 `S` 中的子集合 `Root2` 并入子集合 `Root1`。要求 `Root1` 和 `Root2` 互不相交，否则不执行合并。

3）`Find(S,x)`：查找集合 `S` 中单元素 `x` 所在的子集合，并返回该子集合的根结点。

2．并查集的存储结构

通常用树的双亲表示作为并查集的存储结构，每个子集合以一棵树表示。所有表示子集合的树，构成表示全集合的森林，存放在双亲表示数组内。通常用数组元素的下标代表元素名，用根结点的下标代表子集合名，根结点的双亲域为负数（可设置为该子集合元素数量的相反数）。

例如，若设有一个全集合为 $S=\{0,1,2,3,4,5,6,7,8,9\}$，初始化时每个元素自成一个单元素子集合，每个子集合的数组值为-1，如图 5.28 所示。

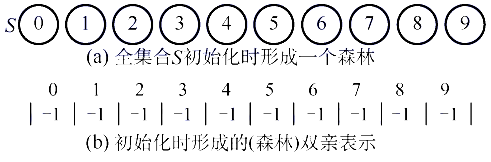


图 5.28　并查集的初始化

经过一段时间的计算后，这些子集合合并为 3 个更大的子集合，即 $S_1=\{0,6,7,8\}$，$S_2=\{1,4,9\}$，$S_3=\{2,3,5\}$，此时并查集的树形和存储结构如图 5.29 所示。

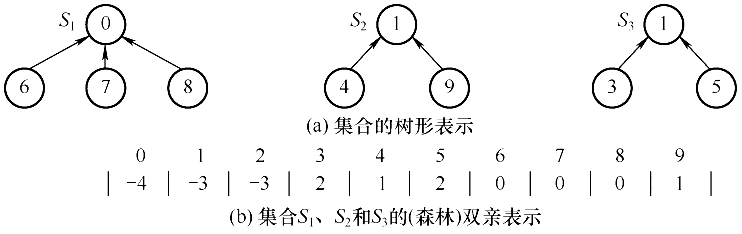


图 5.29　用树表示并查集

为了得到两个子集合的并，只需将其中一个子集合根结点的双亲指针指向另一个集合的根结点。因此，$S_1 \cup S_2$ 可以具有如图 5.30 所示的表示。

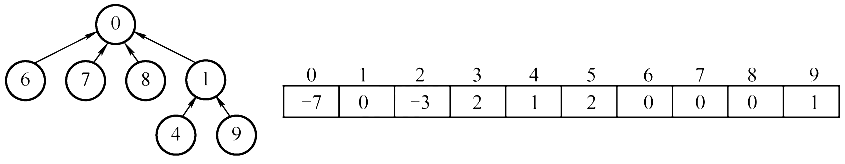


图 5.30　$S_1 \cup S_2$ 可能的表示方法

在采用树的双亲指针数组表示作为并查集的存储表示时，集合元素的编号从 0 到 SIZE-1。其中 SIZE 是最大元素的个数。

3．并查集的基本实现

并查集的结构定义如下：

```
#define SIZE 100
int     UFSets[SIZE];                    //集合元素数组（双亲指针数组）
```

下面是并查集主要运算的实现。

（1）并查集的初始化操作

```
void Initial(int S[]){                   //S 即并查集
    for(int i=0;i<SIZE;i++)              //每个自成单元素集合
        S[i]=-1;
}
```

（2）并查集的 Find 操作

在并查集 S 中查找并返回包含元素 x 的树的根。

```
int Find(int S[],int x){
    while(S[x]>=0)                       //循环寻找 x 的根
        x=S[x];
    return  x;                           //根的 S[]小于 0
}
```

判断两个元素是否属于同一集合，只需分别找到它们的根，再比较根是否相同即可。

（3）并查集的 Union 操作

求两个不相交子集合的并集。若将两个元素所在的集合合并为一个集合，则需要先找到两个元素的根，再令一棵子集树的根指向另一棵子集树的根。

```
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;             //要求 Root1 与 Root2 是不同的集合
    S[Root2]=Root1;                      //将根 Root2 连接到另一根 Root1 下面
}
```

Find 操作和 Union 操作的时间复杂度分别为 $O(d)$和 $O(1)$，其中 d 为树的深度。

4．并查集实现的优化

在极端情况下，n 个元素构成的集合树的深度为 n，则 Find 操作的最坏时间复杂度为 $O(n)$。改进的办法是：在做 Union 操作之前，首先判别子集中的成员数量，然后令成员少的根指向成员多的根，即把小树合并到大树，为此可令根结点的绝对值保存集合树中的成员数量。

（1）改进的 Union 操作

```
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]){               //Root2 结点数更少
        S[Root1]+=S[Root2];              //累加集合树的结点总数
        S[Root2]=Root1;                  //小树合并到大树
    }
    else{                                //Root1 结点数更少
        S[Root2]+=S[Root1];              //累加结点总数
        S[Root1]=Root2;                  //小树合并到大树
    }
}
```

采用这种方法构造得到的集合树，其深度不超过$\lfloor \log_2 n \rfloor + 1$。

随着子集逐对合并，集合树的深度越来越大，为了进一步减少确定元素所在集合的时间，还可进一步对上述 Find 操作进行优化，当所查元素 x 不在树的第二层时，在算法中增加一个压缩

路径的功能，即将从根到元素 x 路径上的所有元素都变成根的孩子。

（2）改进的 Find 操作

```
int Find(int S[],int x){
    int root=x;
    while(s[root]>=0)                    //循环找到根
        root=s[root];
    while(x!=root){                      //压缩路径
        int t=S[x];                      //t 指向 x 的父结点
        S[x]=root;                       //x 直接挂到根结点下面
        x=t;
    }
    return root;                         //返回根结点编号
}
```

通过 Find 操作的压缩路径优化后，可使集合树的深度不超过 $O(\alpha(n))$，其中 $\alpha(n)$是一个增长极其缓慢的函数，对于常见的正整数 n，通常 $\alpha(n)\leqslant 4$。

并查集应用的相关举例见本书配套课程。

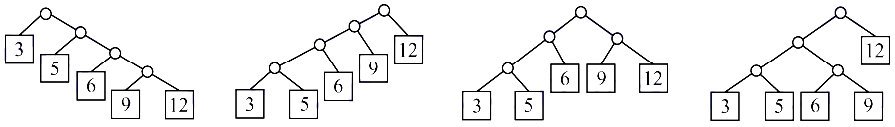
5.5.3 本节试题精选

一、单项选择题

01. 在有 n 个叶结点的哈夫曼树中，非叶结点的总数是（ ）。

A. $n-1$　　B. n　　C. $2n-1$　　D. $2n$

02. 给定整数集合{3, 5, 6, 9, 12}，与之对应的哈夫曼树是（ ）。



A.　　B.　　C.　　D.

03. 下列编码中，（ ）不是前缀码。

A. {00, 01, 10, 11}　　B. {0, 1, 00, 11}

C. {0, 10, 110, 111}　　D. {10, 110, 1110, 1111}

04. 设哈夫曼编码的长度不超过 4，若已对两个字符编码为 1 和 01，则还最多可对（ ）个字符编码。

A. 2　　B. 3　　C. 4　　D. 5

05. 一棵哈夫曼树共有 215 个结点，对其进行哈夫曼编码，共能得到（ ）个不同的码字。

A. 107　　B. 108　　C. 214　　D. 215

06. 设某哈夫曼树有 5 个叶结点，则该哈夫曼树的高度最高可以是（ ）。

A. 3　　B. 4　　C. 5　　D. 6

07. 以下对于哈夫曼树的说法中，错误的是（ ）

A. 用一组权值构造出的哈夫曼树可能不唯一，但带权路径长度唯一

B. 哈夫曼树具有最小的带权路径长度

C. 哈夫曼树中没有度为 1 的结点

D. 哈夫曼树中除了度为 1 的结点，还有度为 2 的结点和叶结点

08. 下列关于哈夫曼树的说法中，错误的是（ ）。

I. 哈夫曼树的总结点数不能是偶数

II. 哈夫曼树中度为 1 的结点数等于度为 2 和 0 的结点数之差

III. 哈夫曼树的带权路径长度等于其所有分支结点的权值之和

A. 仅 III　　B. I 和 II　　C. 仅 II　　D. I、II 和 III

09. 若度为 m 的哈夫曼树中，叶结点个数为 n，则非叶结点的个数为（　）。

A. $n-1$　　B. $\lfloor n/m \rfloor - 1$

C. $\lceil (n-1)/(m-1) \rceil$　　D. $\lceil n/(m-1) \rceil - 1$

10. 并查集的结构是一种（　）。

A. 二叉链表存储的二叉树　　B. 双亲表示法存储的树

C. 顺序存储的二叉树　　D. 孩子表示法存储的树

11. 并查集中最核心的两个操作是：①查找，查找两个元素是否属于同一个集合；②合并，若两个元素不属于同一个集合，且所在的两个集合互不相交，则合并这两个集合。假设初始长度为 10（0～9）的并查集，按 1-2、3-4、5-6、7-8、8-9、1-8、0-5、1-9 的顺序进行查找和合并操作，最终并查集共有（　）个集合。

A. 1　　B. 2　　C. 3　　D. 4

12. 下列关于并查集的说法中，正确的是（　）（注，本题涉及图的考点）。

A. 并查集不能检测图中是否存在环路的问题

B. 通过路径优化后的并查集在最坏情况下的高度仍是 $O(n)$

C. Find 操作返回集合中元素个数的相反数，它用来作为某个集合的标志

D. Union 操作时可根据当前集合的规模，将小集合合并到大集合中

13. 下列关于并查集的叙述中，（　）是错误的（注，本题涉及图的考点）。

A. 并查集是用双亲表示法存储的树

B. 并查集可用于实现克鲁斯卡尔算法

C. 并查集可用于判断无向图的连通性

D. 在长度为 n 的并查集中进行查找操作的时间复杂度为 $O(\log_2 n)$

14. 【2010 统考真题】n（$n \geq 2$）个权值均不相同的字符构成哈夫曼树，关于该树的叙述中，错误的是（　）。

A. 该树一定是一棵完全二叉树

B. 树中一定没有度为 1 的结点

C. 树中两个权值最小的结点一定是兄弟结点

D. 树中任意一个非叶结点的权值一定不小于下一层任意一个结点的权值

15. 【2014 统考真题】5 个字符有如下 4 种编码方案，不是前缀编码的是（　）。

A. 01,0000,0001,001,1　　B. 011,000,001,010,1

C. 000,001,010,011,100　　D. 0,100,110,1110,1100

16. 【2015 统考真题】下列选项给出的是从根分别到达两个叶结点路径上的权值序列，能属于同一棵哈夫曼树的是（　）。

A. 24, 10, 5 和 24, 10, 7　　B. 24,10, 5 和 24, 12, 7

C. 24, 10, 10 和 24, 14, 11　　D. 24, 10, 5 和 24, 14, 6

17. 【2017 统考真题】已知字符集{a, b, c, d, e, f, g, h}，若各字符的哈夫曼编码依次是 0100, 10, 0000, 0101, 001, 011, 11, 0001，则编码序列 0100011001001011110101 的译码结果是（　）。

A. a c g a b f h　　B. a d b a g b b　　C. a f b e a g d　　D. a f e e f g d

18. 【2018 统考真题】已知字符集{a, b, c, d, e, f}，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，

则对应字符集中各字符的哈夫曼编码可能是（ ）。

A. 00, 1011, 01, 1010, 11, 100　　B. 00, 100, 110, 000, 0010, 01

C. 10, 1011, 11, 0011, 00, 010　　D. 0011, 10, 11, 0010, 01, 000

19.【2019统考真题】对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼树共有115个结点，则 n 的值是（ ）。

A. 56　　B. 57　　C. 58　　D. 60

20.【2021统考真题】若某二叉树有5个叶结点，其权值分别为10, 12, 16, 21, 30，则其最小的带权路径长度（WPL）是（ ）。

A. 89　　B. 200　　C. 208　　D. 289

21.【2022统考真题】对任意给定的含 n（$n>2$）个字符的有限集 S，用二叉树表示 S 的哈夫曼编码集和定长编码集，分别得到二叉树 T_1 和 T_2。下列叙述中，正确的是（ ）。

A. T_1 与 T_2 的结点数相同　　B. T_1 的高度大于 T_2 的高度

C. 出现频次不同的字符在 T_1 中处于不同的层

D. 出现频次不同的字符在 T_2 中处于相同的层

22.【2023统考真题】在由6个字符组成的字符集 S 中，各字符出现的频次分别为3, 4, 5, 6, 8, 10，为 S 构造的哈夫曼编码的加权平均长度为（ ）。

A. 2.4　　B. 2.5　　C. 2.67　　D. 2.75

二、综合应用题

01. 设给定权集 $w=\{5, 7, 2, 3, 6, 8, 9\}$，试构造关于 w 的一棵哈夫曼树，并求其加权路径长度WPL。

02.【2012统考真题】设有6个有序表A, B, C, D, E, F，分别含有10, 35, 40, 50, 60和200个数据元素，各表中的元素按升序排列。要求通过5次两两合并，将6个表最终合并为1个升序表，并使最坏情况下比较的总次数达到最小。请回答下列问题：

1）给出完整的合并过程，并求出最坏情况下比较的总次数。

2）根据你的合并过程，描述 n（$n\geq 2$）个不等长升序表的合并策略，并说明理由。

03.【2020统考真题】若任意一个字符的编码都不是其他字符编码的前缀，则称这种编码具有前缀特性。现有某字符集（字符个数≥2）的不等长编码，每个字符的编码均为二进制的0、1序列，最长为 L 位，且具有前缀特性。请回答下列问题：

1）哪种数据结构适宜保存上述具有前缀特性的不等长编码？

2）基于你所设计的数据结构，简述从0/1串到字符串的译码过程。

3）简述判定某字符集的不等长编码是否具有前缀特性的过程。

5.5.4 答案与解析

一、单项选择题

01. A

由哈夫曼树的构造过程可知，哈夫曼树中只有度为0和2的结点。在非空二叉树中，有 $n_0=n_2+1$，所以 $n_2=n-1$。

【另解】n 个结点构造哈夫曼树需要 $n-1$ 次合并过程，每次合并新建一个分支结点，所以选择选项A。

02. C

首先，3 和 5 构造为一棵子树，其根权值为 8，然后该子树与 6 构造为一棵新子树，根权值为 14，再后 9 与 12 构造为一棵子树，最后两棵子树共同构造为一棵哈夫曼树。

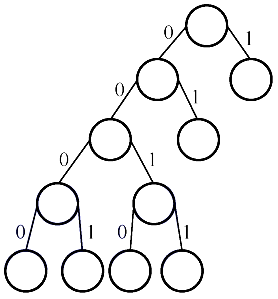
03．B

若没有一个编码是另一个编码的前缀，则称这样的编码为前缀编码。在选项 B 中，0 是 00 的前缀，1 是 11 的前缀。

04．C

在哈夫曼编码中，一个编码不能是任何其他编码的前缀。3 位编码可能是 001，对应的 4 位编码只能是 0000 和 0001。3 位编码也可能是 000，对应的 4 位编码只能是 0010 和 0011。若全采用 4 位编码，则可以为 0000, 0001, 0010 和 0011。题中问的是最多，所以选择选项 C。

【另解】若哈夫曼编码的长度只允许小于或等于 4，则哈夫曼树的高度最高是 5，已知一个字符编码为 1，另一个字符编码是 01，这说明第二层和第三层各有一个叶结点，为使得该树从第 3 层起能够对尽可能多的字符编码，余下的二叉树应该是满二叉树，如下图所示，底层可以有 4 个叶结点，最多可以再对 4 个字符编码。



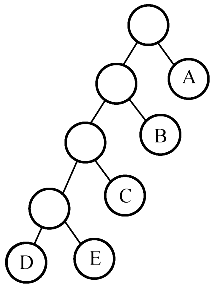
05．B

根据上题的结论，叶结点数为(215 + 1)/2 = 108，所以共有 108 个不同的码字。

【另解】在哈夫曼树中只有度为 0 和 2 的结点，结点总数 $n = n_0 + n_2$，且 $n_0 = n_2 + 1$，由题知 $n = 215$，$n_0 = 108$。

06．C

在哈夫曼树的构造中，每个初始结点最终都成为叶结点，5 个初始结点构造的哈夫曼树共新建 4 个双分支结点，4 个双分支结点所构成的高度最高的哈夫曼树如下图所示，其高度是 5。



07．D

在哈夫曼树的构造过程中，每次选根的权值最小的两棵树，一棵作为左子树，一棵作为右子树，生成新的二叉树，新的二叉树根的权值应为其左右两棵子树根结点权值的和。至于谁做左子树，谁做右子树，没有限制，所以构造的哈夫曼树是不唯一的，但其带权路径长度是最小的和唯一的。哈夫曼树只有度为 0 和 2 的结点，度为 0 的结点是外结点，带有权值，没有度为 1 的结点。

08. C

n 个初始结点构造的哈夫曼树共新建 $n-1$ 个双分支结点，因此哈夫曼树的结点总数是 $2n-1$，是个奇数，选项 I 正确。哈夫曼树中没有度为 1 的结点，选项 II 错误。哈夫曼的带权路径长度有两种计算方法：①所有叶结点的带权路径长度之和；②所有分支结点的权值之和，选项 III 正确。

09. C

一棵度为 m 的哈夫曼树应只有度为 0 和 m 的结点，设度为 m 的结点有 n_m 个，度为 0 的结点有 n_0 个，又设结点总数为 N，$N=n_0+n_m$。因有 N 个结点的哈夫曼树有 $N-1$ 条分支，则 $mn_m=N-1=n_m+n_0-1$，整理得 $(m-1)n_m=n_0-1$，$n_m=(n_0-1)/(m-1)$。

10. B

并查集的存储结构是用双亲表示法存储的树，主要是为了方便两个重要的操作。

11. C

初始时，0～9 各自成一个集合。查找 1-2 时，合并{1}和{2}；查找 3-4 时，合并{3}和{4}；查找 5-6 时，合并{5}和{6}；查找 7-8 时，合并{7}和{8}；查找 8-9 时，合并{7, 8}和{9}；查找 1-8 时，合并{1, 2}和{7, 8, 9}；查找 0-5 时，合并{0}和{5, 6}；查找 1-9 时，它们属于同一个集合。最终的集合为{0, 5, 6}、{1, 2, 7, 8, 9}和{3, 4}，因此答案选择选项 C。

12. D

依次探测图的各条边，用并查集检查该边依附的两个顶点是否已属于同一集合（两个顶点的根结点是否相同）。若是，则说明图中存在环路，选项 A 错误。经过路径优化后，并查集在最坏情况下的高度远小于 $O(n)$，选项 B 错误。`Find` 操作总返回当前根结点作为集合的标志，选项 C 错误。

13. D

在用并查集实现 Kruskal 算法求图的最小生成树时：判断是否加入一条边之前，先查找这条边关联的两个顶点是否属于同一个集合（判断加入这条边之后是否形成回路），若形成回路，则继续判断下一条边；若不形成回路，则将该边和边对应的顶点加入最小生成树 T，并继续判断下一条边，直到所有顶点都已加入最小生成树 T。选项 B 正确。用并查集判断无向图连通性的方法：遍历无向图的边，每遍历到一条边，就把这条边连接的两个顶点合并到同一个集合中，处理完所有边后，只要是相互连通的顶点都会被合并到同一个子集合中，相互不连通的顶点一定在不同的子集合中。选项 C 正确。未做路径优化的并查集在最坏情况下的高度为 n，此时查找操作的时间复杂度为 $O(n)$，时间复杂度通常指最坏情况下的时间复杂度。选项 D 错误。

14. A

哈夫曼树为带权路径长度最小的二叉树，不一定是完全二叉树。哈夫曼树中没有度为 1 的结点，选项 B 正确。构造哈夫曼树时，最先选取两个权值最小的结点作为左、右子树构造一棵新的二叉树，选项 C 正确。哈夫曼树中任意一个非叶结点的权值为其左、右子树根结点的权值之和，可知，哈夫曼树中任意一个非叶结点的权值一定不小于下一层任意一个结点的权值。

15. D

前缀编码的定义是在一个字符集中，任何一个字符的编码都不是另一个字符编码的前缀。选项 D 中的编码 110 是编码 1100 的前缀，违反了前缀编码的规则，所以选项 D 不是前缀编码。

16. D

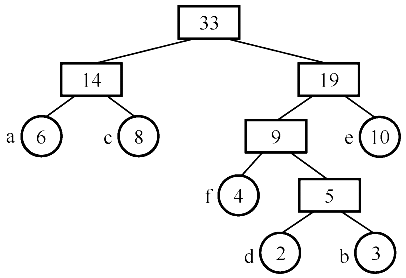
在哈夫曼树中，左右孩子权值之和为父结点权值。仅以分析选项 A 为例：若两个 10 分别属于两棵不同的子树，则根的权值不等于其孩子的权值和，不符；若两个 10 属同棵子树，则其权值不等于其两个孩子（叶结点）的权值和，不符。选项 B、C 选项的排除方法相同。

17．D

哈夫曼编码是前缀编码，各个编码的前缀不同，因此直接拿编码序列与哈夫曼编码一一比对即可。序列可分割为 0100 011 001 001 011 11 0101，译码结果是 afeefgd。选项 D 正确。

18．A

根据各字符出现的次数构造的哈夫曼树如下图所示。由图可知，a、c 和 e 的编码长度应该相同；a 和 c 的第 1 个编码应该相同，且与 e 的第 1 个编码不同；b 和 d 的前 3 个编码应该相同。

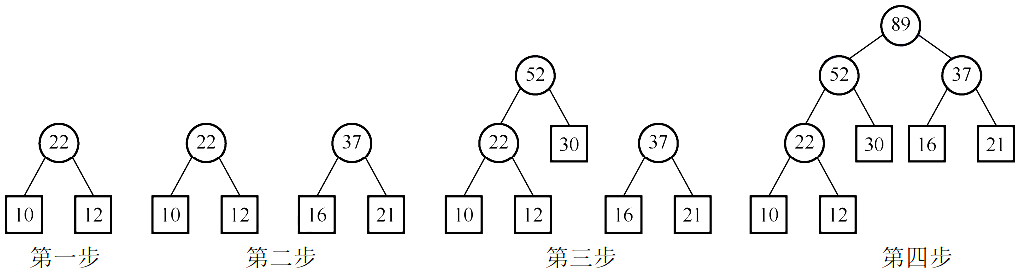


19．C

n 个符号构造成哈夫曼树的过程中，共新建了 $n-1$ 个结点（双分支结点），因此哈夫曼树的结点总数为 $2n-1=115$，n 的值为 58。

20．B

对于带权值的结点，构造出哈夫曼树的带权路径长度（WPL）最小，哈夫曼树的构造过程如下图所示。求得其 WPL = (10 + 12)×3 + (30 + 16 + 21)×2 = 200。



21．D

可以画一个简单的特例来证明。图 1 是满足条件的二叉树 T_1，图 2 是满足条件的二叉树 T_2，结点中有值表示这个结点是编码字符。T_1 和 T_2 的结点数不同，选项 A 错误。T_1 的高度等于 T_2 的高度，选项 B 错误。出现频次不同的字符在 T_1 中也可能处于相同的层，选项 C 错误。对于定长编码集，所有字符一定都在 T_2 中处于相同的层，而且都是叶结点。

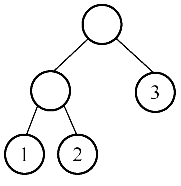


图 1

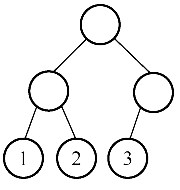
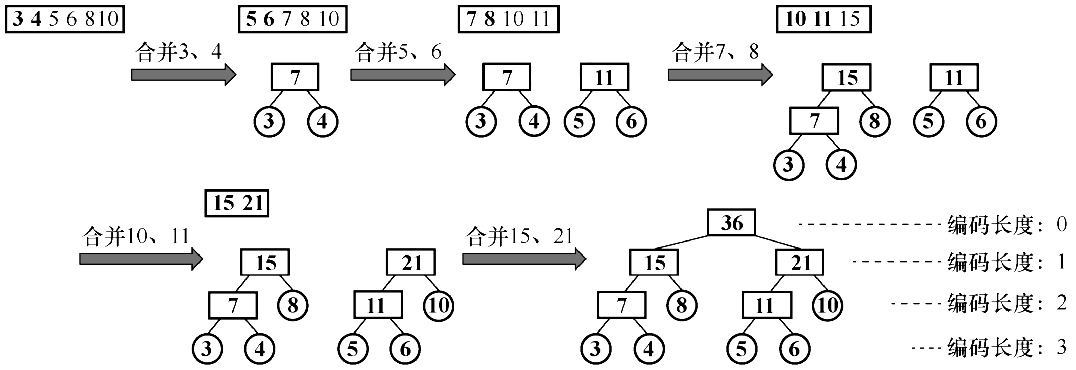


图 2

22．B

构建哈夫曼树的过程如下图所示。

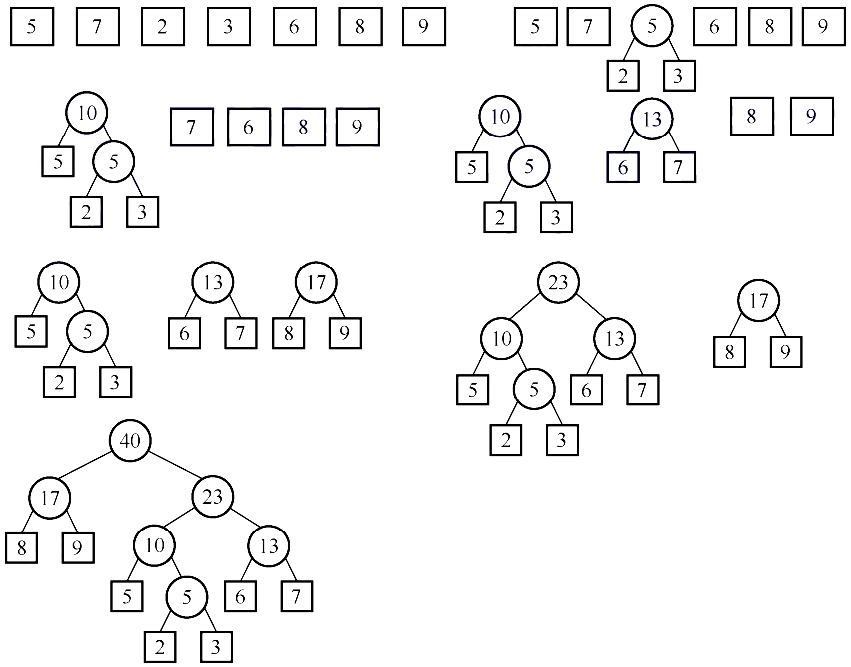


对叶结点的哈夫曼编码，共有 4 个长度为 3 的叶结点、2 个长度为 2 的叶结点，编码的加权平均长度为[(3+4+5+6)×3+(8+10)×2]/(3+4+5+6+8+10)=2.5。

二、综合应用题

01.【解答】

根据哈夫曼树的构造方法，每次从森林中选取两个根结点值最小的树合并成一棵树，将原先的两棵树作为左、右子树，且新根结点的值为左、右孩子关键字之和。构造过程如下图所示。



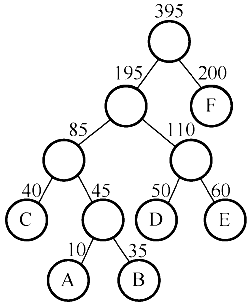
由构造出的哈夫曼树可得 WPL=(2+3)×4+(5+6+7)×3+(8+9)×2=108。

注 意

哈夫曼树并不唯一，但带权路径长度一定是相同的。

02.【解答】

1）最先合并的表中的元素在后续的每次合并中都会再次参与比较，因此求最小合并次数类似于求最小带权路径长度，此时可立即想到哈夫曼树。根据哈夫曼树的构造过程，每次选择表集合中长度最小的两个表进行合并。6 个表的合并顺序如下图所示。



根据图中的哈夫曼树，6 个序列的合并过程如下：

① 在表集合{10, 35, 40, 50, 60, 200}中，选择表 A 与表 B 合并，生成含 45 个元素的表 AB。

② 在表集合{40, 45, 50, 60, 200}中，将表 AB 与表 C 合并，生成含 85 个元素的表 ABC。

③ 在表集合{50, 60, 85, 200}中，表 D 与表 E 合并，生成含 110 个元素的表 DE。

④ 在表集合{85, 110, 200}中，表 ABC 与表 DE 合并，生成含 195 个元素的表 ABCDE。

⑤ 当前表集合为{195, 200}，表 ABCDE 与表 F 合并，生成含 395 个元素的表 ABCDEF。

因为合并两个长度分别为 m 和 n 的有序表，最坏情况下需要比较 $m+n-1$ 次，所以最坏情况下比较的总次数计算如下：

第 1 次合并：最多比较次数 = 10 + 35 – 1 = 44。

第 2 次合并：最多比较次数 = 40 + 45 – 1 = 84。

第 3 次合并：最多比较次数 = 50 + 60 – 1 = 109。

第 4 次合并：最多比较次数 = 85 + 110 – 1 = 194。

第 5 次合并：最多比较次数 = 195 + 200 – 1 = 394。

比较的总次数最多为 44 + 84 + 109 + 194 + 394 = 825。

2）各表的合并策略是：对多个有序表进行两两合并时，若表长不同，则最坏情况下总的比较次数依赖于表的合并次序。可以借助于哈夫曼树的构造思想，依次选择最短的两个表进行合并，此时可以获得最坏情况下的最佳合并效率。

03.【解答】

1）使用一棵二叉树保存字符集中各字符的编码，每个编码对应于从根开始到达某叶结点的一条路径，路径长度等于编码位数，路径到达的叶结点中保存该编码对应的字符。

2）从左至右依次扫描 0/1 串中的各位。从根开始，根据串中当前位沿当前结点的左子指针或右子指针下移，直到移动到叶结点时为止。输出叶结点中保存的字符。然后从根开始重复这个过程，直到扫描到 0/1 串结束，译码完成。

3）二叉树既可用于保存各字符的编码，又可用于检测编码是否具有前缀特性。判定编码是否具有前缀特性的过程，也是构建二叉树的过程。初始时，二叉树中仅含有根结点，其左子指针和右子指针均为空。

依次读入每个编码 C，建立/寻找从根开始对应于该编码的一条路径，过程如下：

对每个编码，从左至右扫描 C 的各位，根据 C 的当前位（0 或 1）沿结点的指针（左子指针或右子指针）向下移动。当遇到空指针时，创建新结点，让空指针指向该新结点并继续移动。沿指针移动的过程中，可能遇到三种情况：

① 若遇到了叶结点（非根），则表明不具有前缀特性，返回。

② 若在处理 C 的所有位的过程中，均没有创建新结点，则表明不具有前缀特性，返回。

③ 若在处理 C 的最后一个编码位时创建了新结点，则继续验证下一个编码。

若所有编码均通过验证，则编码具有前缀特性。

归纳总结

本章的内容较多，其中二叉树是极其重要的考查点。关于二叉树的有关操作，在 2014 年的统考中首次出现了线性表以外的算法设计题，需要引起读者的注意。

遍历是二叉树的各种操作的基础，统考时会考查遍历过程中对结点的各种其他操作，而且容易结合递归算法和利用栈或队列的非递归算法。读者需重点掌握各种遍历方法的代码书写，并学会在遍历的基础上，进行一些其他的相关操作。其中递归算法短小精悍，出现的概率较大，请读者不要掉以轻心，要做到对几种遍历方式的程序模板烂熟于心，并结合一定数量的习题，才可以在考试中快速地写出漂亮的代码。

二叉树遍历算法的递归程序：

```
void Track(BiTree *p){
    if(p!=NULL){
        //(1)
        Track(p->lchild);
        //(2)
        Track(p->rchild);
        //(3)
    }
}
```

访问函数 visit() 位于 (1)、(2)、(3) 的位置，分别对应于先序、中序、后序遍历。但对于具体题目来说，设计算法时要灵活应用。请读者认真练习下面的例题。

例题：设二叉树的存储结构为二叉链表，编写有关二叉树的递归算法。

1）统计二叉树中度为 1 的结点个数。

2）统计二叉树中度为 2 的结点个数。

3）统计二叉树中度为 0 的结点个数。

4）统计二叉树的高度。

5）统计二叉树的宽度。

6）从二叉树中删去所有叶结点。

7）计算指定结点*p 所在的层次。

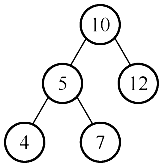
8）计算二叉树中各结点中的最大元素的值。

9）交换二叉树中每个结点的两个子女。

10）以先序次序输出一颗二叉树中所有结点的数据值及结点所在的层次。

思维拓展

输入一个整数 data 和一棵二元树。从树的根结点开始往下访问一直到叶结点，所经过的所有结点形成一条路径。打印出路径及与 data 相等的所有路径。例如，输入整数 22 和下图所示的二元树，则打印出两条路径 10, 12 和 10, 5, 7。



注　意

使用数组或栈保存访问的路径，并记录当前路径上所有元素的和 sum。若当前结点为叶结点，且当前结点值与 sum 的和等于 data，则满足条件，打印当前路径。然后递归返回到父结点，注意在递归返回之前要先减去当前结点元素的值。使用前序遍历操作的递归算法模板可以简化程序。

第 6 章

图

【考纲内容】

（一）图的基本概念

（二）图的存储及基本操作

邻接矩阵；邻接表；邻接多重表；十字链表

（三）图的遍历

深度优先搜索；广度优先搜索

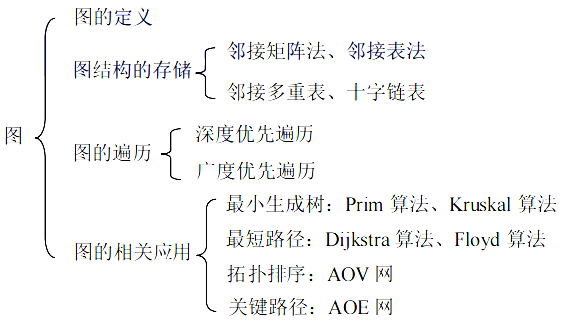
（四）图的基本应用

最小（代价）生成树；最短路径；拓扑排序；关键路径

扫一扫

视频讲解

【知识框架】



【复习提示】

图算法的难度较大，主要掌握深度优先搜索与广度优先搜索。掌握图的基本概念及基本性质、图的存储结构（邻接矩阵、邻接表、邻接多重表和十字链表）及特性、存储结构之间的转化、基于存储结构上的各种遍历操作和各种应用（拓扑排序、最小生成树、最短路径和关键路径）等。图的相关算法较多，通常只需掌握其基本思想和实现步骤，而实现代码不是重点。

6.1 图的基本概念

6.1.1 图的定义

图 G 由顶点集 V 和边集 E 组成，记为 $G=(V, E)$，其中 $V(G)$表示图 G 中顶点的有限非空集；$E(G)$表示图 G 中顶点之间的关系（边）集合。若 $V=\{v_1, v_2, \cdots, v_n\}$，则用$|V|$表示图 G 中顶点的个数，$E=\{(u, v) \mid u \in V, v \in V\}$，用$|E|$表示图 G 中边的条数。

注　意

线性表可以是空表，树可以是空树，但图不可以是空图。也就是说，图中不能一个顶点也没有，图的顶点集 V 一定非空，但边集 E 可以为空，此时图中只有顶点而没有边。

下面是图的一些基本概念及术语。

1．有向图

若 E 是有向边（也称弧）的有限集合，则图 G 为有向图。弧是顶点的有序对，记为<v, w>，其中 v, w 是顶点，v 称为弧尾，w 称为弧头，<v, w>称为从 v 到 w 的弧，也称 v 邻接到 w。

图 6.1(a)所示的有向图 G_1 可表示为

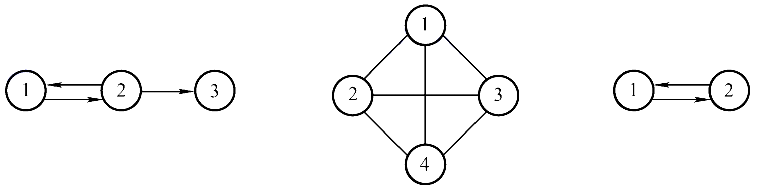
$$G_1 = (V_1, E_1)$$
$$V_1 = \{1, 2, 3\}$$
$$E_1 = \{<1, 2>, <2, 1>, <2, 3>\}$$

2．无向图

若 E 是无向边（简称边）的有限集合，则图 G 为无向图。边是顶点的无序对，记为(v, w)或(w, v)。可以说 w 和 v 互为邻接点。边(v, w)依附于 w 和 v，或称边(v, w)和 v, w 相关联。

图 6.1(b)所示的无向图 G_2 可表示为

$$G_2 = (V_2, E_2)$$
$$V_2 = \{1, 2, 3, 4\}$$
$$E_2 = \{(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)\}$$



(a) 有向图G_1　(b) 无向图G_2　(c) 有向完全图G_3

图 6.1　图的示例

3．简单图、多重图

一个图 G 若满足：①不存在重复边；②不存在顶点到自身的边，则称图 G 为简单图。图 6.1 中 G_1 和 G_2 均为简单图。若图 G 中某两个顶点之间的边数大于 1 条，又允许顶点通过一条边和自身关联，则称图 G 为多重图。多重图和简单图的定义是相对的。本书中仅讨论简单图。

4．顶点的度、入度和出度

命题追踪 ▶ 无向图中顶点和边的关系（2009、2017）

在无向图中，顶点 v 的度是指依附于顶点 v 的边的条数，记为 TD(v)。在图 6.1(b)中，每个顶点的度均为 3。无向图的全部顶点的度之和等于边数的 2 倍，因为每条边和两个顶点相关联。

在有向图中，顶点 v 的度分为入度和出度，入度是以顶点 v 为终点的有向边的数目，记为 ID(v)；而出度是以顶点 v 为起点的有向边的数目，记为 OD(v)。在图 6.1(a)中，顶点 2 的出度为 2、入度为 1。顶点 v 的度等于其入度与出度之和，即 TD(v) = ID(v) + OD(v)。有向图的全部顶点的入度之和与出度之和相等，并且等于边数，这是因为每条有向边都有一个起点和终点。

5．路径、路径长度和回路

顶点 v_p 到顶点 v_q 之间的一条路径是指顶点序列 $v_p, v_{i_1}, v_{i_2}, \cdots, v_{i_m}, v_q$，当然关联的边也可理解为路径的构成要素。路径上的边的数目称为路径长度。第一个顶点和最后一个顶点相同的路径称为回路或环。若一个图有 n 个顶点，且有大于 $n-1$ 条边，则此图一定有环。

6．简单路径、简单回路

命题追踪 ▶ 路径、回路、简单路径、简单回路的定义（2011）

在路径序列中，顶点不重复出现的路径称为简单路径。除第一个顶点和最后一个顶点外，其余顶点不重复出现的回路称为简单回路。

7．距离

从顶点 u 出发到顶点 v 的最短路径若存在，则此路径的长度称为从 u 到 v 的距离。若从 u 到 v 根本不存在路径，则记该距离为无穷（∞）。

8．子图

设有两个图 $G=(V, E)$ 和 $G'=(V', E')$，若 V' 是 V 的子集，且 E' 是 E 的子集，则称 G' 是 G 的子图。若有满足 $V(G')=V(G)$ 的子图 G'，则称其为 G 的生成子图。图 6.1 中 G_3 为 G_1 的子图。

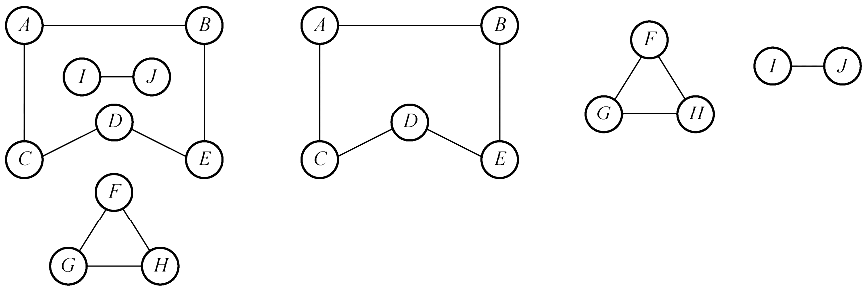
注　意

并非 V 和 E 的任何子集都能构成 G 的子图，因为这样的子集可能不是图，即 E 的子集中的某些边关联的顶点可能不在这个 V 的子集中。

9．连通、连通图和连通分量

命题追踪 ▶ 图的连通性与边和顶点的关系（2010、2022）

在无向图中，若从顶点 v 到顶点 w 有路径存在，则称 v 和 w 是连通的。若图 G 中任意两个顶点都是连通的，则称图 G 为连通图，否则称为非连通图。无向图中的极大连通子图称为连通分量，在图 6.2(a)中，图 G_4 有 3 个连通分量如图 6.2(b)所示。假设一个图有 n 个顶点，若边数小于 $n-1$，则此图必是非连通图；思考，若该图是非连通图，则最多可以有多少条边？[①]



(a) 无向图G_4　　(b) G_4的三个连通分量

图 6.2　无向图及其连通分量

10．强连通图、强连通分量

在有向图中，若有一对顶点 v 和 w，从 v 到 w 和从 w 到 v 之间都有路径，则称这两个顶点

① 非连通情况下边最多的情况：由 $n-1$ 个顶点构成一个完全图，此时再加入一个顶点则变成非连通图。

是强连通的。若图中任意一对顶点都是强连通的，则称此图为强连通图。有向图中的极大强连通子图称为有向图的强连通分量，图 G_1 的强连通分量如图 6.3 所示。思考，假设一个有向图有 n 个顶点，若该图是强连通图，则最少需要有多少条边？①

注　意

在无向图中讨论连通性，在有向图中讨论强连通性。

11．生成树、生成森林

连通图的生成树是包含图中全部顶点的一个极小连通子图。若图中顶点数为 n，则它的生成树含有 $n-1$ 条边。包含图中全部顶点的极小连通子图，只有生成树满足这个极小条件，对生成树而言，若砍去它的一条边，则会变成非连通图，若加上一条边则会形成一个回路。在非连通图中，连通分量的生成树构成了非连通图的生成森林。图 G_2 的一个生成树如图 6.4 所示。

图 6.3　图 G_1 的强连通分量

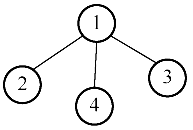


图 6.4　图 G_2 的一个生成树

注　意

区分极大连通子图和极小连通子图。极大连通子图要求子图必须连通，而且包含尽可能多的顶点和边；极小连通子图是既要保持子图连通又要使得边数最少的子图。

12．边的权、网和带权路径长度

在一个图中，每条边都可以标上具有某种含义的数值，该数值称为该边的权值。这种边上带有权值的图称为带权图，也称网。路径上所有边的权值之和，称为该路径的带权路径长度。

13．完全图（也称简单完全图）

对于无向图，$|E|$的取值范围为 0 到 $n(n-1)/2$，有 $n(n-1)/2$ 条边的无向图称为完全图，在完全图中任意两个顶点之间都存在边。对于有向图，$|E|$的取值范围为 0 到 $n(n-1)$，有 $n(n-1)$条弧的有向图称为有向完全图，在有向完全图中任意两个顶点之间都存在方向相反的两条弧。图 6.1 中 G_2 为无向完全图，而 G_3 为有向完全图。

14．稠密图、稀疏图

边数很少的图称为稀疏图，反之称为稠密图。稀疏和稠密本身是模糊的概念，稀疏图和稠密图常常是相对而言的。一般当图 G 满足$|E|<|V|\log_2|V|$时，可以将 G 视为稀疏图。

15．有向树

一个顶点的入度为 0、其余顶点的入度均为 1 的有向图，称为有向树。

6.1.2　本节试题精选

一、单项选择题

01. 图中有关路径的定义是（　）。

① 有向图强连通情况下边最少的情况：至少需要 n 条边，构成一个环路。

A. 由顶点和相邻顶点序偶构成的边所形成的序列

B. 由不同顶点所形成的序列

C. 由不同边所形成的序列

D. 上述定义都不是

02. 一个有 n 个顶点和 n 条边的无向图一定是（ ）。

A. 连通的　　B. 不连通的　　C. 无环的　　D. 有环的

03. 若从无向图的任意顶点出发进行一次深度优先搜索即可访问所有顶点，则该图一定是（ ）。

A. 强连通图　　B. 连通图　　C. 有回路　　D. 一棵树

04. 以下关于图的叙述中，正确的是（ ）。

A. 图与树的区别在于图的边数大于或等于顶点数

B. 假设有图 $G=\{V,\{E\}\}$，顶点集 $V'\subseteq V$，$E'\subseteq E$，则 V'和$\{E'\}$构成 G 的子图

C. 无向图的连通分量是指无向图中的极大连通子图

D. 图的遍历就是从图中某一顶点出发访遍图中其余顶点

05. 以下关于图的叙述中，正确的是（ ）。

A. 强连通有向图的任何顶点到其他所有顶点都有弧

B. 图的任意顶点的入度等于出度

C. 有向完全图一定是强连通有向图

D. 有向图的边集的子集和顶点集的子集都构成原有向图的子图

06. 一个有 28 条边的非连通无向图至少有（ ）个顶点。

A. 7　　B. 8　　C. 9　　D. 10

07. 对于一个有 n 个顶点的图：若是连通无向图，其边的个数至少为（ ）；若是强连通有向图，则其边的个数至少为（ ）。

A. $n-1,n$　　B. $n-1,n(n-1)$　　C. n,n　　D. $n,n(n-1)$

08. 无向图 G 有 23 条边，度为 4 的顶点有 5 个，度为 3 的顶点有 4 个，其余都是度为 2 的顶点，则图 G 有（ ）个顶点。

A. 11　　B. 12　　C. 15　　D. 16

09. 在有 n 个顶点的有向图中，顶点的度最大可达（ ）。

A. n　　B. $n-1$　　C. $2n$　　D. $2n-2$

10. 具有 6 个顶点的无向图，当有（ ）条边时能确保是一个连通图。

A. 8　　B. 9　　C. 10　　D. 11

11. 设有无向图 $G=(V,E)$和 $G'=(V'',E')$，若 G'是 G 的生成树，则下列不正确的是（ ）。

I. G'为 G 的连通分量

II. G'为 G 的无环子图

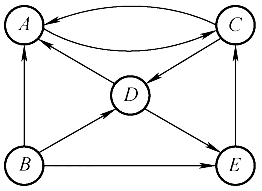
III. G'为 G 的极小连通子图且 $V''=V$

A. I、II　　B. 只有 III　　C. II、III　　D. 只有 I

12. 具有 51 个顶点和 21 条边的无向图的连通分量最多为（ ）。

A. 33　　B. 34　　C. 45　　D. 32

13. 在如下图所示的有向图中，共有（ ）个强连通分量。



A. 1　　B. 2　　C. 3　　D. 4

14. 若具有 n 个顶点的图是一个环，则它有（ ）棵生成树。

A. n^2　　B. n　　C. $n-1$　　D. 1

15. 若一个具有 n 个顶点、e 条边的无向图是一个森林，则该森林中必有（ ）棵树。

A. n　　B. e　　C. $n-e$　　D. 1

16. 【2009 统考真题】下列关于无向连通图特性的叙述中，正确的是（ ）。

I. 所有顶点的度之和为偶数

II. 边数大于顶点个数减 1

III. 至少有一个顶点的度为 1

A. 只有 I　　B. 只有 II　　C. I 和 II　　D. I 和 III

17. 【2010 统考真题】若无向图 $G=(V, E)$中含有 7 个顶点，要保证图 G 在任何情况下都是连通的，则需要的边数最少是（ ）。

A. 6　　B. 15　　C. 16　　D. 21

18. 【2017 统考真题】已知无向图 G 含有 16 条边，其中度为 4 的顶点个数为 3，度为 3 的顶点个数为 4，其他顶点的度均小于 3。图 G 所含的顶点个数至少是（ ）。

A. 10　　B. 11　　C. 13　　D. 15

19. 【2022 统考真题】对于无向图 $G=(V, E)$，下列选项中，正确的是（ ）。

A. 当$|V|>|E|$时，G 一定是连通的

B. 当$|V|<|E|$时，G 一定是连通的

C. 当$|V|=|E|-1$ 时，G 一定是不连通的

D. 当$|V|>|E|+1$ 时，G 一定是不连通的

二、综合应用题

01. 图 G 是一个非连通无向图，共有 28 条边，该图至少有多少个顶点？

6.1.3　答案与解析

一、单项选择题

01. A

本题是北京交通大学考研真题，不同教材对路径的定义可能略有不同，顶点之间关联的边也可理解为路径的构成要素。对于选项 B，路径的定义中并没有要求是不同顶点，比如简单回路的第一个顶点和最后一个顶点是可以相同的，此外选项 B 也没有说明这些顶点之间有边相联。

02. D

若一个无向图有 n 个顶点和 $n-1$ 条边，可以使它连通但没有环（生成树），但若再加一条边，在不考虑重边的情形下，则必然会构成环。

03. B

强连通图是有向图，与题意矛盾，选项 A 错误；对无向连通图做一次深度优先搜索，可以访问到该连通图的所有顶点，选项 B 正确；有回路的无向图不一定是连通图，因为回路不一定

包含图的所有结点，选项C错误；连通图可能是树，也可能存在环，选项D错误。

04．C

图与树的区别是逻辑上的区别，而不是边数的区别，图的边数也可能小于树的边数，选项A错误；若E'中的边对应的顶点不是V'的元素，V'和$\{E'\}$无法构成图，选项B错误；无向图的极大连通子图称为连通分量，选项C正确；图的遍历要求每个结点只能被访问一次，且若图非连通，则从某一顶点出发无法访问到其他全部顶点，选项D的说法不准确。

05．C

强连通有向图的任何顶点到其他所有顶点都有路径，但未必有弧；无向图任意顶点的入度等于出度，但有向图未必满足；若边集中的某条边对应的某个顶点不在对应的顶点集中，则有向图的边集的子集和顶点集的子集无法构成子图。

06．C

考查至少有多少个顶点的情形，我们考虑该非连通图最极端的情况，即它由一个完全图加一个独立的顶点构成，此时若再加一条边，则必然使图变成连通图。在$28 = n(n-1)/2 = 8\times7/2$条边的完全无向图中，总共有8个顶点，再加上1个不连通的顶点，共9个顶点。

07．A

对于连通无向图，边最少即构成一棵树的情形；对于强连通有向图，边最少即构成一个有向环的情形。

08．D

因为在具有n个顶点、e条边的无向图中，有$\sum_{i=1}^{n}\mathrm{TD}(v_i)=2e$，所以求得度为2的顶点数为7，从而共有16个顶点。

09．D

在有向图中，顶点的度等于入度与出度之和。n个顶点的有向图中，任意一个顶点最多还可以与其他$n-1$个顶点有一对指向相反的边相连。注意，数据结构中仅讨论简单图。

10．D

5个顶点构成一个完全无向图，需要$n(n-1)/2 = 10$条边；再加上1条边后，能保证第6个顶点必然与此完全无向图构成一个连通图，所以共需11条边。

11．D

一个连通图的生成树是一个极小连通子图，显然它是无环的，因此选项II、III正确。极大连通子图称为连通分量，G'连通但非连通分量。这里再补充一下“极大连通子图”：若图本来就是连通的，且每个子部分包含其本身的所有顶点和边，则它就是极大连通子图。

12．C

初始考虑只有51个顶点的无向图G，此时G中每个顶点都是连通分量，问题转化为向G中添加21条边，如何添加这21条边使得连通分量数目最多。若向两个不同的连通分量之间添加边，则连通分量数目会减1，所以应尽可能地将这21条边加入同一个连通分量且让其接近完全图，含有7个顶点的完全图有21条边，所以用7个顶点构成一个含有21条边的连通分量，剩下$51-7=44$个顶点对应44个连通分量，共有45个连通分量。

13．B

强连通分量是极大强连通子图，任意两个顶点之间有方向相反的两条路径。由定义不难得出，若一个顶点只有出边或入边，则该顶点必定单独构成一个连通分量。图中，顶点B只有出边，其他所有顶点都不可能有到顶点B的路径，所以顶点B单独构成一个强连通分量。在顶点A、C、D、E中，任意两个顶点之间都有方向相反的两条路径，所以可构成一个强连通分量。

14．B

n 个顶点的生成树是具有 $n-1$ 条边的极小连通子图，因为 n 个顶点构成的环共有 n 条边，去掉任意一条边就是一棵生成树，所以共有 n 种情况，所以可以有 n 棵不同的生成树。

15．C

n 个结点的树有 $n-1$ 条边，假设森林中有 x 棵树，将每棵树的根连到一个添加的结点，则成为一棵树，结点数是 $n+1$，边数是 $e+x$，从而可知 $x=n-e$。

【另解】设森林中有 x 棵树，则再用 $x-1$ 条边就可将所有的树连接成一棵树，此时边数 + 1 = 顶点数，即 $e+(x-1)+1=n$，所以 $x=n-e$。

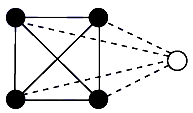
16．A

每条边都连接了两个顶点，在计算顶点的度之和时每条边都被计算了两次，所以所有顶点的度之和偶数。无向连通图对应的生成树也是无向连通图，但此时边数等于顶点数减 1，选项 II 错误。考虑 2 个或以上的顶点恰好构成一个环的情况，此时每个顶点的度都是 2，选项 III 错误。

17．C

题干要求无论如何分配边，都能使 7 个顶点连通，这不同于只要 6 条边两两相连就能构成一个连通图的情形。考虑最极端的情形，即图 G 的某 6 个顶点构成一个完全无向图，此时若再添加一条边，则都将连通第 7 个顶点，使该图变成一个连通图。所以最少边数 = 6×5/2 + 1 = 16。若边数 n 小于或等于 15，可以使这 n 条边仅连接图 G 中的某 6 个顶点，从而导致第 7 个顶点无法与这 6 个顶点构成连通图（不满足“在任何情况下”）。

为简单起见，以 5 个顶点为例，左边 4 个顶点和 4×3/2 = 6 条边构成一个完全图，此时若再添加一条边（可以是虚线中的任意一条），则能保证这 5 个顶点在任何情况下都是连通的，如下图所示。若边数小于 7，则不能保证 5 个顶点在任何情况下都是连通的。



18．B

无向图边数的 2 倍等于各顶点度数的总和。要求至少的顶点数，应使每个顶点的度取最大，而其他顶点的度均小于 3，因此可设它们的度都为 2，并设它们的数量是 x，列出方程 4×3 + 3×4 + $2x$ = 16×2，解得 $x=4$。因此至少包含 4 + 4 + 3 = 11 个顶点。

19．D

对于此类分析图的边数、顶点数与连通性问题，思路是寻找临界情况，在临界情况下任意增加或减少一条边，都会改变图的连通性。第一种临界情况如图 1 所示，此时若减少任意一条边，图就由连通变为不连通，即无向图连通的最小边数是 $|V|-1$，因此，当 $|E|<|V|-1$ 时，图一定不连通，选项 C 错误，选项 D 正确。第二种临界情况如图 2 所示，此时若增加任意一条边，则图就由不连通变为连通，即无向图不连通的最大边数是 $(|V|-1)(|V|-2)/2$（此时 $|V|-1$ 个顶点构成一个完全图），因此，仅当 $|E|\geqslant(|V|-1)(|V|-2)/2+1$ 时才能保证无向图一定连通，选项 A、B 错误。

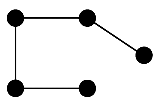


图 1

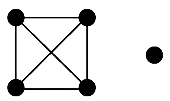


图 2

二、综合应用题

01．【解答】

图 G 是一个非连通无向图，当边数固定时，顶点数最少的情况是该图由两个连通子图构

成，且其中之一只含一个顶点，另一个为完全图。其中只含一个顶点的子图没有边，另一个完全图的边数为 $n(n-1)/2=28$，得 $n=8$。所以该图至少有 $1+8=9$ 个顶点。

6.2　图的存储及基本操作

图的存储必须要完整、准确地反映顶点集和边集的信息。根据不同图的结构和算法，采用不同的存储方式将对程序的效率产生相当大的影响，因此所选的存储结构应适合于待求解的问题。

6.2.1　邻接矩阵法

所谓邻接矩阵存储，是指用一个一维数组存储图中顶点的信息，用一个二维数组存储图中边的信息（各顶点之间的邻接关系），存储顶点之间邻接关系的二维数组称为邻接矩阵。

顶点数为 n 的图 $G=(V,E)$ 的邻接矩阵 $\boldsymbol{A}$ 是 $n\times n$ 的，将 G 的顶点编号为 $v_1,v_2,\cdots,v_n$，则

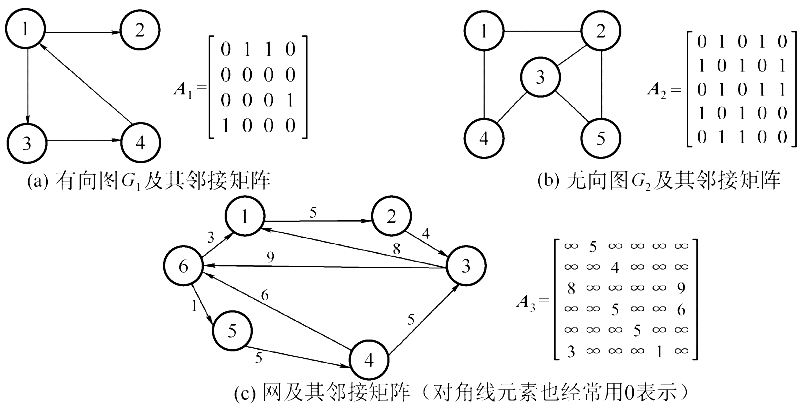
$$A[i][j]=\begin{cases}1, & (v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{是}E(G)\text{中的边}\\ 0, & (v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{不是}E(G)\text{中的边}\end{cases}$$

命题追踪 ▶ 图的邻接矩阵存储及相互转换（2011、2015、2018）

对带权图而言，若顶点 v_i 和 v_j 之间有边相连，则邻接矩阵中对应项存放着该边对应的权值，若顶点 V_i 和 V_j 不相连，则通常用 0 或∞来代表这两个顶点之间不存在边：

$$A[i][j]=\begin{cases}w_{ij}, & (v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{是}E(G)\text{中的边}\\ 0\text{或}\infty, & (v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{不是}E(G)\text{中的边}\end{cases}$$

有向图、无向图和网对应的邻接矩阵示例如图 6.5 所示。



(a) 有向图G_1及其邻接矩阵　　(b) 无向图G_2及其邻接矩阵

(c) 网及其邻接矩阵（对角线元素也经常用0表示）

图 6.5　有向图、无向图及网的邻接矩阵

命题追踪 ▶ （算法题）邻接矩阵的遍历及顶点的度的计算（2021、2023）

图的邻接矩阵存储结构定义如下：

```
#define MaxVertexNum 100                    //顶点数目的最大值
typedef char VertexType;                    //顶点对应的数据类型
typedef int EdgeType;                       //边对应的数据类型
typedef struct{
    VertexType vex[MaxVertexNum];           //顶点表
    EdgeType edge[MaxVertexNum][MaxVertexNum];//邻接矩阵，边表
```

```
    int vexnum,arcnum;                          //图的当前顶点数和边数
}MGraph;
```

注　意

① 在简单应用中，可直接用二维数组作为图的邻接矩阵（顶点信息等均可省略）。

② 当邻接矩阵的元素仅表示相应边是否存在时，EdgeType 可用值为 0 和 1 的枚举类型。

③ 无向图的邻接矩阵是对称矩阵，对规模特大的邻接矩阵可采用压缩存储。

④ 邻接矩阵表示法的空间复杂度为 $O(n^2)$，其中 n 为图的顶点数$|V|$。

命题追踪 ▶ 邻接矩阵的遍历的时间复杂度（2021）

图的邻接矩阵存储表示法具有以下特点：

① 无向图的邻接矩阵一定是一个对称矩阵（并且唯一）。因此，在实际存储邻接矩阵时只需存储上（或下）三角矩阵的元素。

命题追踪 ▶ 基于邻接矩阵的顶点的度的计算（2013、2021、2023）

② 对于无向图，邻接矩阵的第 i 行（或第 i 列）非零元素（或非∞元素）的个数正好是顶点 i 的度 $\mathrm{TD}(v_i)$。

③ 对于有向图，邻接矩阵的第 i 行非零元素（或非∞元素）的个数正好是顶点 i 的出度 $\mathrm{OD}(v_i)$；第 i 列非零元素（或非∞元素）的个数正好是顶点 i 的入度 $\mathrm{ID}(v_i)$。

④ 用邻接矩阵存储图，很容易确定图中任意两个顶点之间是否有边相连。但是，要确定图中有多少条边，则必须按行、按列对每个元素进行检测，所花费的时间代价很大。

⑤ 稠密图（边数较多的图）适合采用邻接矩阵的存储表示。

命题追踪 ▶ 计算 A^2 并说明 $A^n[i][j]$的含义（2015）

⑥ 设图 G 的邻接矩阵为 $\boldsymbol{A}$，$\boldsymbol{A}^n$ 的元素 $A^n[i][j]$等于由顶点 i 到顶点 j 的长度为 n 的路径的数目。该结论了解即可，证明方法可参考离散数学教材。

6.2.2　邻接表法

当一个图为稀疏图时，使用邻接矩阵法显然会浪费大量的存储空间，而图的邻接表法结合了顺序存储和链式存储方法，大大减少了这种不必要的浪费。

所谓邻接表，是指对图 G 中的每个顶点 v_i 建立一个单链表，第 i 个单链表中的结点表示依附于顶点 v_i 的边（对于有向图则是以顶点 v_i 为尾的弧），这个单链表就称为顶点 v_i 的边表（对于有向图则称为出边表）。边表的头指针和顶点的数据信息采用顺序存储，称为顶点表，所以在邻接表中存在两种结点：顶点表结点和边表结点，如图 6.6 所示。

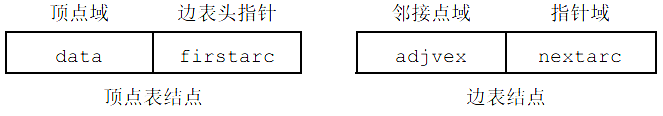
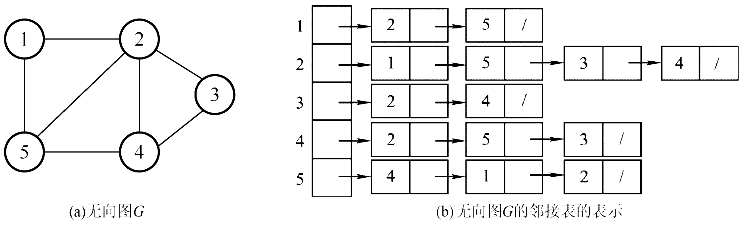


图 6.6　顶点表和边表结点结构

顶点表结点由两个域组成：顶点域（data）存储顶点 v_i 的相关信息，边表头指针域（firstarc）指向第一条边的边表结点。边表结点至少由两个域组成：邻接点域（adjvex）存储与头结点顶点 v_i 邻接的顶点编号，指针域（nextarc）指向下一条边的边表结点。

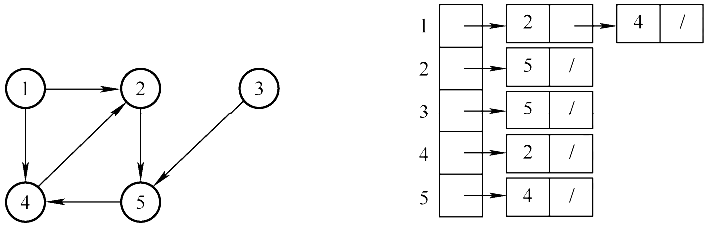
命题追踪 ▶ 图的邻接表存储的应用（2014）

无向图和有向图的邻接表的实例分别如图 6.7 和图 6.8 所示。



(a)无向图G　　(b)无向图G的邻接表的表示

图 6.7　无向图邻接表表示法实例



(a)有向图 G　　(b)有向图 G 的邻接表的表示

图 6.8　有向图邻接表表示法实例

图的邻接表存储结构定义如下：

```
#define MaxVertexNum 100          //图中顶点数目的最大值
typedef struct ArcNode{           //边表结点
    int adjvex;                   //该弧所指向的顶点的位置
    struct ArcNode *nextarc;      //指向下一条弧的指针
    //InfoType info;              //网的边权值
}ArcNode;
typedef struct VNode{             //顶点表结点
    VertexType data;              //顶点信息
    ArcNode *firstarc;            //指向第一条依附该顶点的弧的指针
}VNode,AdjList[MaxVertexNum];
typedef struct{
    AdjList vertices;             //邻接表
    int vexnum,arcnum;            //图的顶点数和弧数
}ALGraph;                         //ALGraph 是以邻接表存储的图类型
```

图的邻接表存储方法具有以下特点：

① 若 G 为无向图，则所需的存储空间为 $O(|V|+2|E|)$；若 G 为有向图，则所需的存储空间为 $O(|V|+|E|)$。前者的倍数 2 是因为在无向图中，每条边在邻接表中出现了两次。

命题追踪 ▶ 邻接矩阵法和邻接表法的适用性差异（2011）

② 对于稀疏图（边数较少的图），采用邻接表表示将极大地节省存储空间。

③ 在邻接表中，给定一个顶点，能很容易地找出它的所有邻边，因为只需要读取它的邻接表。在邻接矩阵中，相同的操作则需要扫描一行，花费的时间为 $O(n)$。但是，若要确定给定的两个顶点间是否存在边，则在邻接矩阵中可以立刻查到，而在邻接表中则需要在相应结点对应的边表中查找另一结点，效率较低。

④ 在无向图的邻接表中，求某个顶点的度只需计算其邻接表中的边表结点个数。在有向图的邻接表中，求某个顶点的出度只需计算其邻接表中的边表结点个数；但求某个顶点 x 的入度则需遍历全部的邻接表，统计邻接点（adjvex）域为 x 的边表结点个数。

⑤ 图的邻接表表示并不唯一，因为在每个顶点对应的边表中，各边结点的链接次序可以是任意的，它取决于建立邻接表的算法及边的输入次序。

6.2.3　十字链表

十字链表是有向图的一种链式存储结构。在十字链表中，有向图的每条弧用一个结点（弧结点）来表示，每个顶点也用一个结点（顶点结点）来表示。两种结点的结构如下所示。

弧结点

tailvex	headvex	hlink	tlink	(info)

顶点结点

data	firstin	firstout

弧结点中有 5 个域：tailvex 域和 headvex 域分别存放弧尾和弧头这两个顶点的编号；头链域 hlink 指向弧头相同的下一条弧；尾链域 tlink 指向弧尾相同的下一条弧；info 域存放该弧的相关信息。这样，弧头相同的弧在同一个链表上，弧尾相同的弧也在同一个链表上。

顶点结点中有 3 个域：data 域存放该顶点的数据信息，如顶点名称；firstin 域指向以该顶点为弧头的第一条弧；firstout 域指向以该顶点为弧尾的第一条弧。

图 6.9 为有向图的十字链表表示法。

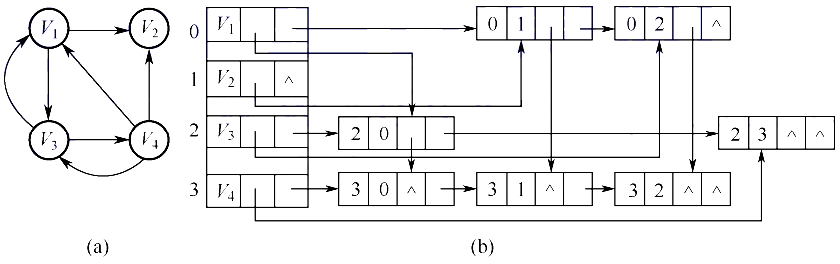


图 6.9　有向图的十字链表表示（弧结点省略 info 域）

注意，顶点结点之间是顺序存储的，弧结点省略了 info 域。

在十字链表中，既容易找到 V_i 为尾的弧，也容易找到 V_i 为头的弧，因而容易求得顶点的出度和入度。图的十字链表表示是不唯一的，但一个十字链表表示唯一确定一个图。

6.2.4　邻接多重表

邻接多重表是无向图的一种链式存储结构。在邻接表中，容易求得顶点和边的各种信息，但求两个顶点之间是否存在边而执行删除边等操作时，需要分别在两个顶点的边表中遍历，效率较低。与十字链表类似，在邻接多重表中，每条边用一个结点表示，其结构如下所示。

ivex	ilink	jvex	jlink	(info)

其中，ivex 域和 jvex 域存放该边依附的两个顶点的编号；ilink 域指向依附于顶点 ivex 的下一条边；jlink 域指向依附于顶点 jvex 的下一条边，info 域存放该边的相关信息。

每个顶点也用一个结点表示，它由如下所示的两个域组成。

data	firstedge

其中，data 域存放该顶点的相关信息，firstedge 域指向依附于该顶点的第一条边。

在邻接多重表中，所有依附于同一顶点的边串联在同一链表中，因为每条边依附于两个顶点，所以每个边结点同时链接在两个链表中。对无向图而言，其邻接多重表和邻接表的差别仅在于，同一条边在邻接表中用两个结点表示，而在邻接多重表中只有一个结点。

命题追踪 ▶ 图的邻接多重表表示的分析（2024）

图 6.10 为无向图的邻接多重表表示法。邻接多重表的各种基本操作的实现和邻接表类似。

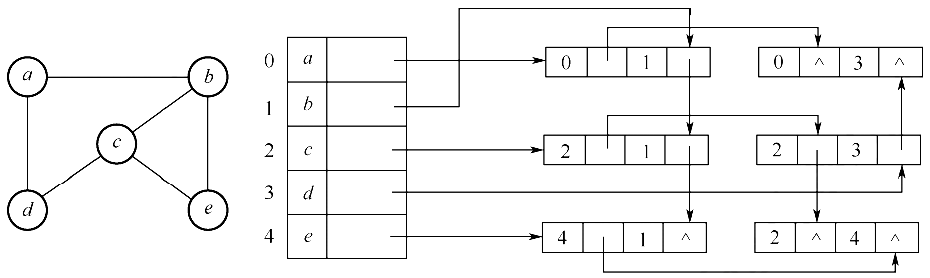


图 6.10　无向图的邻接多重表表示（边结点省略 info 域）

图的四种存储方式的总结如表 6.1 所示。

表 6.1　图的四种存储方式的总结

	邻接矩阵	邻接表	十字链表	邻接多重表
空间复杂度	$O(\|V\|^2)$	无向图：$O(\|V\|+2\|E\|)$ 有向图：$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$
找相邻边	遍历对应行或列的时间复杂度为 $O(\|V\|)$	找有向图的入度必须遍历整个邻接表	很方便	很方便
删除边或顶点	删除边很方便，删除顶点需要大量移动数据	无向图中删除边或顶点都不方便	很方便	很方便
适用于	稠密图	稀疏图和其他	只能存有向图	只能存无向图
表示方式	唯一	不唯一	不唯一	不唯一

6.2.5　图的基本操作

图的基本操作是独立于图的存储结构的。而对于不同的存储方式，操作算法的具体实现会有着不同的性能。在设计具体算法的实现时，应考虑采用何种存储方式的算法效率会更高。

图的基本操作主要包括（仅抽象地考虑，所以忽略各变量的类型）：

- Adjacent(G,x,y)：判断图 G 是否存在边<x,y>或(x,y)。
- Neighbors(G,x)：列出图 G 中与结点 x 邻接的边。
- InsertVertex(G,x)：在图 G 中插入顶点 x。
- DeleteVertex(G,x)：从图 G 中删除顶点 x。
- AddEdge(G,x,y)：若无向边(x,y)或有向边<x,y>不存在，则向图 G 中添加该边。
- RemoveEdge(G,x,y)：若无向边(x,y)或有向边<x,y>存在，则从图 G 中删除该边。
- FirstNeighbor(G,x)：求图 G 中顶点 x 的第一个邻接点，若有则返回顶点号。若 x 没有邻接点或图中不存在 x，则返回-1。
- NextNeighbor(G,x,y)：假设图 G 中顶点 y 是顶点 x 的一个邻接点，返回除 y 外顶点 x 的下一个邻接点的顶点号，若 y 是 x 的最后一个邻接点，则返回-1。

- `Get_edge_value(G,x,y)`：获取图 G 中边(x,y)或<x,y>对应的权值。
- `Set_edge_value(G,x,y,v)`：设置图 G 中边(x,y)或<x,y>对应的权值为 v。

此外，还有图的遍历算法：按照某种方式访问图中的每个顶点且仅访问一次。图的遍历算法包括深度优先遍历和广度优先遍历，具体见下一节的内容。

6.2.6　本节试题精选

一、单项选择题

01. 下列关于图的存储结构的说法中，错误的是（ ）。

A. 使用邻接矩阵存储一个图时，在不考虑压缩存储的情况下，所占用的存储空间大小只与图中的顶点数有关，与边数无关

B. 邻接表只用于有向图的存储，邻接矩阵适用于有向图和无向图

C. 若一个有向图的邻接矩阵的对角线以下的元素为 0，则该图的拓扑序列必定存在

D. 存储无向图的邻接矩阵是对称的，所以只需存储邻接矩阵的下（或上）三角部分

02. 若图的邻接矩阵中主对角线上的元素皆为 0，其余元素全为 1，则该图一定（ ）。

A. 是无向图　　B. 是有向图　　C. 是完全图　　D. 不是带权图

03. 在含有 n 个顶点和 e 条边的无向图的邻接矩阵中，零元素的个数为（ ）。

A. e　　B. $2e$　　C. n^2-e　　D. n^2-2e

04. 带权有向图 G 用邻接矩阵存储，则 v_i 的入度等于邻接矩阵中（ ）。

A. 第 i 行非∞的元素个数　　B. 第 i 列非∞的元素个数

C. 第 i 行非∞且非 0 的元素个数　　D. 第 i 列非∞且非 0 的元素个数

05. 一个有 n 个顶点的图用邻接矩阵 $\boldsymbol{A}$ 表示，若图为有向图，顶点 v_i 的入度是（ ）；若图为无向图，顶点 v_i 的度是（ ）。

A. $\sum_{i=1}^{n} A[i][j]$　　B. $\sum_{j=1}^{n} A[j][i]$

C. $\sum_{i=1}^{n} A[j][i]$　　D. $\sum_{j=1}^{n} A[j][i]$ 或 $\sum_{j=1}^{n} A[i][j]$

06. 从邻接矩阵 $\boldsymbol{A}=\begin{bmatrix}0&1&0\\1&0&1\\0&1&0\end{bmatrix}$ 可以看出，该图共有（①）个顶点；若是有向图，则该图共有（②）条弧；若是无向图，则共有（③）条边。

①A. 9　B. 3　C. 6　D. 1　E. 以上答案均不正确

②A. 5　B. 4　C. 3　D. 2　E. 以上答案均不正确

③A. 5　B. 4　C. 3　D. 2　E. 以上答案均不正确

07. 以下关于图的存储结构的叙述中，正确的是（ ）。

A. 一个图的邻接矩阵表示唯一，邻接表表示唯一

B. 一个图的邻接矩阵表示唯一，邻接表表示不唯一

C. 一个图的邻接矩阵表示不唯一，邻接表表示唯一

D. 一个图的邻接矩阵表示不唯一，邻接表表示不唯一

08. 矩阵 $\boldsymbol{A}$ 是有向图 G 的邻接矩阵，若矩阵 $\boldsymbol{A}^2$ 的某元素 $a_{i,j}^2=3$，则说明（ ）。

A. 从顶点 i 到 j 存在 3 条长度为 2 的路径

B. 从顶点 i 到 j 存在3条长度不超过2的路径

C. 从顶点 i 到 j 存在2条长度为3的路径

D. 从顶点 i 到 j 存在2条长度不超过3的路径

09. 用邻接表法存储图所用的空间大小（ ）。

A. 与图的顶点数和边数有关　　B. 只与图的边数有关

C. 只与图的顶点数有关　　D. 与边数的平方有关

10. 若邻接表中有奇数个边表结点，则（ ）。

A. 图中有奇数个结点　　B. 图中有偶数个结点

C. 图为无向图　　D. 图为有向图

11. 在有向图的邻接表存储结构中，顶点 v 在边表中出现的次数是（ ）。

A. 顶点 v 的度　　B. 顶点 v 的出度

C. 顶点 v 的入度　　D. 依附于顶点 v 的边数

12. n 个顶点的无向图的邻接表最多有（ ）个边表结点。

A. n^2　　B. $n(n-1)$　　C. $n(n+1)$　　D. $n(n-1)/2$

13. 设某无向图中有 n 个顶点和 e 条边，则建立该图的邻接表的时间复杂度是（ ）。

A. $O(n+e)$　　B. $O(n^2)$　　C. $O(ne)$　　D. $O(n^3)$

14. 假设有 n 个顶点、e 条边的有向图用邻接表表示，则删除与某个顶点 v 相关的所有边的时间复杂度为（ ）。

A. $O(n)$　　B. $O(e)$　　C. $O(n+e)$　　D. $O(ne)$

15. 设 n 个顶点、e 条边的有向图用邻接表表示，则某个顶点 v 的入度的时间复杂度为（ ）。

$O(n)$　　B. $O(e)$　　C. $O(n+e)$　　D. $O(ne)$

16. 对邻接表的叙述中，（ ）是正确的。

A. 无向图的邻接表中，第 i 个顶点的度为第 i 个链表中结点数的两倍

B. 邻接表比邻接矩阵的操作更简便

C. 邻接矩阵比邻接表的操作更简便

D. 求有向图结点的度，必须遍历整个邻接表

17. 邻接多重表是（ ）的存储结构。

A. 无向图　　B. 有向图

C. 无向图和有向图　　D. 都不是

18. 十字链表是（ ）的存储结构。

A. 无向图　　B. 有向图

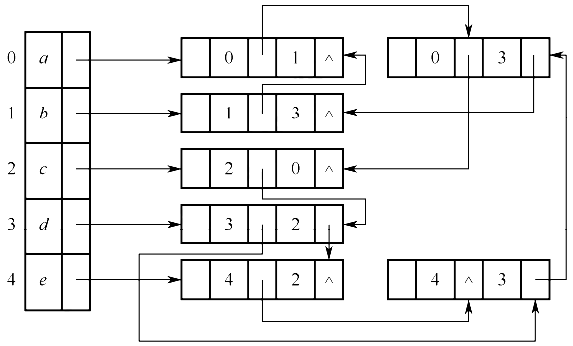
C. 无向图和有向图　　D. 都不是

19. 【2013统考真题】设图的邻接矩阵 $\boldsymbol{A}$ 如下所示，各顶点的度依次是（ ）。

$$\boldsymbol{A}=\begin{bmatrix}0&1&0&1\\0&0&1&1\\0&1&0&0\\1&0&0&0\end{bmatrix}$$

A. 1, 2, 1, 2　　B. 2, 2, 1, 1　　C. 3, 4, 2, 3　　D. 4, 4, 2, 2

20. 【2024统考真题】若无向图 $G=(V, E)$ 的邻接多重表如下图所示，则 G 中顶点 b 与 d 的度分别是（ ）。



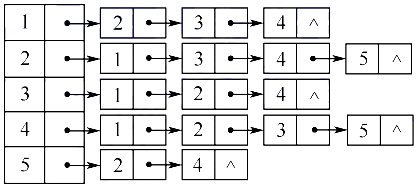
A. 0，2　　B. 2，4　　C. 2，5　　D. 3，4

二、综合应用题

01. 已知带权有向图 G 的邻接矩阵如下图所示，请画出该带权有向图 G。

0	15	2	12	∞	∞	∞
∞	0	∞	∞	6	∞	∞
∞	∞	0	∞	8	4	∞
∞	∞	∞	0	∞	∞	3
∞	∞	∞	∞	0	∞	9
∞	∞	∞	5	∞	0	10
∞	4	∞	∞	∞	∞	0

02. 设图 $G=(V, E)$以邻接表存储，如下图所示。画出其邻接矩阵存储及图 G。



03. 对 n 个顶点的无向图和有向图，分别采用邻接矩阵和邻接表表示时，试问：

1）如何判别图中有多少条边？

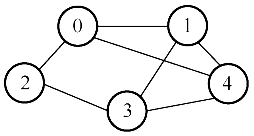
2）如何判别任意两个顶点 i 和 j 是否有边相连？

3）任意一个顶点的度是多少？

04. 如何对无环有向图中的顶点重新编号，使得该图的邻接矩阵中所有的 1 都集中到对角线以上？

05. 写出从图的邻接表表示转换成邻接矩阵表示的算法。

06. 【2015 统考真题】已知含有 5 个顶点的图 G 如下图所示。



请回答下列问题：

1）写出图 G 的邻接矩阵 $\boldsymbol{A}$（行、列下标从 0 开始）。

2）求 $\boldsymbol{A}^2$，矩阵 $\boldsymbol{A}^2$ 中位于 0 行 3 列元素值的含义是什么？

3）若已知具有 n（$n\geqslant 2$）个顶点的图的邻接矩阵为 $\boldsymbol{B}$，则 $\boldsymbol{B}^m$（$2\leqslant m\leqslant n$）中非零元素的含义是什么？

07.【2021统考真题】已知无向连通图 G 由顶点集 V 和边集 E 组成，$|E|>0$，当 G 中度为奇数的顶点个数为不大于 2 的偶数时，G 存在包含所有边且长度为$|E|$的路径（称为 EL 路径）。设图 G 采用邻接矩阵存储，类型定义如下：

```
typedef struct{                          //图的定义
    int  numVertices,numEdges;           //图中实际的顶点数和边数
    char VerticesList[MAXV];             //顶点表。MAXV 为已定义常量
    int Edge[MAXV][MAXV];                //邻接矩阵
  }MGraph;
```

请设计算法 int IsExistEL(MGraph G)，判断 G 是否存在 EL 路径，若存在，则返回 1，否则返回 0。要求：

1）给出算法的基本设计思想。

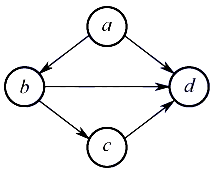
2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

08.【2023统考真题】已知有向图 G 采用邻接矩阵存储，类型定义如下：

```
typedef struct{                          //图的类型定义
    int numVertices,numEdges;            //图的顶点数和有向边数
    char VerticesList[MAXV];             //顶点表，MAXV 为已定义常量
    int Edge[MAXV][MAXV];                //邻接矩阵
}MGraph;
```

将图中出度大于入度的顶点称为 K 顶点。例如，在下图中，顶点 a 和 b 为 K 顶点。



请设计算法 int printVertices(MGraph G)，对给定的任意非空有向图 G，输出 G 中所有的 K 顶点，并返回 K 顶点的个数。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

6.2.7 答案与解析

一、单项选择题

01. B

n 个顶点的图，若采用邻接矩阵表示，不考虑压缩存储，则存储空间大小为 $O(n^2)$，选项 A 正确。邻接表可用于存储无向图，只是把每条边都视为两条方向相反的有向边，因此需要存储两次，选项 B 错误。因为邻接矩阵中对角线以下的元素全为 0，所以若存在$<i,j>$，则必有 $i<j$，由传递性可知图中路径的顶点编号是依次递增的，假设存在环 $k\rightarrow\cdots\rightarrow j\rightarrow k$，由题设可知 $k<j<k$，矛盾，所以不存在环，拓扑序列必定存在，选项 C 正确。选项 D 显然正确。

注 意

若邻接矩阵对角线以下（或以上）的元素全为 0，则图中必然不存在环，即拓扑序列一定存在，但这并不能说明拓扑序列是唯一的。

02. C

除主对角线上的元素外，其余元素全为 1，说明任意两个顶点之间都有边相连，因此该图一

定是完全图。

03．D

在无向图的邻接矩阵中，矩阵大小为 n^2，非零元素的个数为 $2e$，所以零元素的个数为 n^2-2e。读者应掌握此题的变体，即当无向图变为有向图时，能够求出零的个数和非零的个数。

04．D

带权有向图的邻接矩阵中，0 和∞表示的都不是有向边，而入度是由邻接矩阵的列中元素计算出来的；出度是由邻接矩阵的行中元素计算出来的。

05．B、D

有向图的入度是其第 i 列的非 0 元素之和，无向图的度是第 i 行或第 i 列的非 0 元素之和。

06．B、B、D

邻接矩阵的顶点数等于矩阵的行（列）数，有向图的边数等于矩阵中非零元素的个数，无向图的边数等于矩阵中非零元素个数的一半。

注　意

本题中所给的矩阵为对称矩阵，若不是对称矩阵，则必然不可能是无向图。

07．B

邻接矩阵表示唯一是因为图中边的信息在矩阵中有确定的位置，邻接表不唯一是因为邻接表的建立取决于读入边的顺序和边表中的插入算法。

08．A

设图 G 的邻接矩阵为 $\boldsymbol{A}$，$\boldsymbol{A}^n$ 的元素 $a^n_{i,j}$ 等于从顶点 i 到 j 的长度为 n 的路径的数目，因此 $a^2_{i,j}=3$ 表示从顶点 i 到 j 存在 3 条长度为 2 的路径。该结论记住即可。

09．A

邻接表存储时，顶点数 n 决定了顶点表的大小，边数 e 决定了边表结点的个数，且无向图的每条边存储两次，总存储空间为 $O(n+2e)$。而邻接矩阵只与图的顶点数有关，为 $O(n^2)$。

10．D

无向图采用邻接表表示时，每条边存储两次，所以其边表结点的个数为偶数。题中边表结点为奇数个，所以必然是有向图，且有奇数条边。

11．C

题中的边表是不包括顶点表的。因为任何顶点 u 对应的边表中存放的都是以 u 为起点的边所对应的另一个顶点 v。从而 v 在边表中出现的次数也就是它的入度。

12．B

最多有 $n(n-1)/2$ 条边，每条边在邻接表中存储两次，因此边表结点最多为 $n(n-1)$ 个。

13．A

建立图的邻接表需要遍历所有的顶点和边，每个顶点有一个顶点表结点，每条边需要创建一个边表结点并插入到相应的链表中。因此，共需 $n+2e$ 次操作，时间复杂度为 $O(n+e)$。

14．C

与顶点 v 相关的边包括出边和入边，对于出边，只需遍历 v 的顶点表结点和其指向的边表；对于入边，则需遍历整个边表。先删除出边：删除 v 的顶点表结点的单链表，出边数最多为 $n-1$，时间复杂度为 $O(n)$；再删除入边：扫描整个边表（扫描剩余全部顶点表结点及其指向的边表），删除所有的顶点 v 的入边，时间复杂度为 $O(n+e)$。因此总时间复杂度为 $O(n+e)$。

15. C

为了求顶点 v 的入度，只需要遍历邻接表中的所有边表，检查每条边是否指向顶点 v，这相当于遍历整个邻接表，因此算法的时间复杂度为 $O(n+e)$ 。

16. D

无向图的邻接表中，第 i 个顶点的度为第 i 个链表中的结点数，所以选项 A 错。邻接表和邻接矩阵对于不同的操作各有优势，选项 B 和 C 都不准确。有向图结点的度包括出度和入度，对于出度，需要遍历顶点表结点所对应的边表；对于入度，则需要遍历剩下的全部边表。

17. A

邻接多重表是无向图的存储结构。

18. B

十字链表是有向图的存储结构。

19. C

邻接矩阵 $\boldsymbol{A}$ 为非对称矩阵，说明图是有向图，度为入度与出度之和。各顶点的度是矩阵中此结点对应的行（对应出度）和列（对应入度）的非零元素之和。

20. B

在邻接多重表中，统计一个结点的入度、出度或度数（入度+出度）时，只需考虑所有的边结点。下图是邻接多重表的边结点的结构：

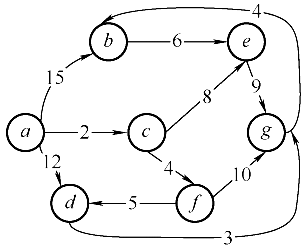
ivex	ilink	jvex	jlink	info

其中，`ivex` 表示弧头的编号，`ilink` 域指向下一个弧头相同的边结点，`jvex` 表示弧尾的编号，`jlink` 域指向下一个弧尾相同的边结点。根据定义可知：求 x 号结点的入度时，只需统计 `jvex` 域等于 x 的边结点个数；求 x 号结点的出度时，只需统计 `ivex` 域等于 x 的边结点个数；x 号结点的度数为入度和出度之和。顶点 b 的编号为 1，统计可知：入度为 1，出度为 1，度数为 2。顶点 d 的编号为 3，统计可知：入度为 1，出度为 3，度数为 4。

二、综合应用题

01.【解答】

带权有向图 G 如下图所示。



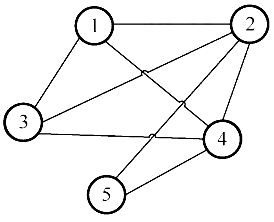
02.【解答】

其邻接矩阵存储如下所示。

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

在邻接表中，每条边存储了 2 次，在没有特殊说明时，通常默认其为无向图（当然，无向

图也可视为具有对边的有向图)。该邻接表对应的图 G 如下图所示。



03.【解答】

1）对于邻接矩阵表示的无向图，边数等于矩阵中 1 的个数除以 2；对于邻接表表示的无向图，边数等于边结点的个数除以 2。对于邻接矩阵表示的有向图，边数等于矩阵中 1 的个数；对于邻接表表示的有向图，边数等于边结点的个数。

2）在邻接矩阵表示的无向图或有向图中，对于任意两个顶点 i 和 j，邻接矩阵中 arcs[i][j] 或 arcs[j][i]为 1 表示有边相连，否则表示无边相连。在邻接表表示的无向图或有向图中，对于任意两个顶点 i 和 j，若从顶点表结点 i 出发找到编号为 j 的边表结点或从顶点表结点 j 出发找到编号为 i 的边表结点，表示有边相连；否则为无边相连。

3）对于邻接矩阵表示的无向图，顶点 i 的度等于第 i 行中 1 的个数；对于邻接矩阵表示的有向图，顶点 i 的出度等于第 i 行中 1 的个数；入度等于第 i 列中 1 的个数；度数等于它们的和。对于邻接表表示的无向图，顶点 i 的度等于顶点表结点 i 的单链表中边表结点的个数；对于邻接表表示的有向图，顶点 i 的出度等于顶点表结点 i 的单链表中边表结点的个数，顶点 i 的入度等于邻接表中所有编号为 i 的边表结点数；度数等于入度与出度之和。

04.【解答】

按各顶点的出度进行排序。n 个顶点的有向图，其顶点的最大出度是 $n-1$，最小出度为 0。这样排序后，出度最大的顶点编号为 1，出度最小的顶点编号为 n。之后，进行调整，即只要存在弧$<i, j>$，就不管顶点 j 的出度是否大于顶点 i 的出度，都把 i 编号在顶点 j 的编号之前，因为只有 $i \leqslant j$，弧$<i, j>$对应的 1 才能出现在邻接矩阵的上三角。

通过后面小节的学习，会发现采用拓扑排序并依次编号是一种更为简便的方法。

05.【解答】

算法思想：设图的顶点分别存储在数组 v[n]中。首先初始化邻接矩阵。遍历邻接表，在依次遍历顶点 v[i]的边链表时，修改邻接矩阵的第 i 行的元素值。若链表边结点的值为 j，则置 arcs[i][j]=1。遍历完邻接表时，整个转换过程结束。此算法对于无向图、有向图均适用。

算法的实现如下：

```
void  Convert(ALGraph &G,int arcs[M][N]){
//此算法将邻接表方式表示的图 G 转换为邻接矩阵 arcs
    for(i=0;i<n;i++){              //依次遍历各顶点表结点为头的边链表
        p=(G->v[i]).firstarc;      //取出顶点 i 的第一条出边
        while(p!=NULL){            //遍历边链表
            arcs[i][p->adjvex]=1;
            p=p->nextarc;          //取下一条出边
        }                          //while
    }                              //for
}
```

06.【解答】

考查图的邻接矩阵的性质。

1）图 G 的邻接矩阵 $\boldsymbol{A}$ 如下：

$$\boldsymbol{A}=\begin{bmatrix}0&1&1&0&1\\1&0&0&1&1\\1&0&0&1&0\\0&1&1&0&1\\1&1&0&1&0\end{bmatrix}$$

2）$\boldsymbol{A}^2$ 如下：

$$\boldsymbol{A}^2=\begin{bmatrix}3&1&0&3&1\\1&3&2&1&2\\0&2&2&0&2\\3&1&0&3&1\\1&2&2&1&3\end{bmatrix}$$

0 行 3 列的元素值 3 表示从顶点 0 到顶点 3 之间长度为 2 的路径共有 3 条。

3）$\boldsymbol{B}^m$（$2\leqslant m\leqslant n$）中位于 i 行 j 列（$0\leqslant i,\ j\leqslant n-1$）的非零元素的含义是，图中从顶点 i 到顶点 j 的长度为 m 的路径条数。

07.【解答】

1）算法的基本设计思想：

本算法题属于送分题，题干已经告诉我们算法的思想。对于采用邻接矩阵存储的无向图，在邻接矩阵的每一行（列）中，非零元素的个数为本行（列）对应顶点的度。可以依次计算连通图 G 中各顶点的度，并记录度为奇数的顶点个数，若个数为 0 或 2，则返回 1，否则返回 0。

2）算法实现

```
int IsExistEL(MGraph G){
    //采用邻接矩阵存储，判断图是否存在 EL 路径
    int degree,i,j,count=0;
    for(i=0;i<G.numVertices;i++){
        degree=0;
        for(j=0;j<G.numVertices;j++)
            degree+=G.Edge[i][j];          //依次计算各个顶点的度
        if(degree%2!=0)
            count++;      //对度为奇数的顶点计数
    }
    if(count==0||count==2)
        return 1;              //存在 EL 路径，返回 1
    else
        return 0;              //不存在 EL 路径，返回 0
}
```

3）时间复杂度和空间复杂度

算法需要遍历整个邻接矩阵，所以时间复杂度是 $O(n^2)$，空间复杂度是 $O(1)$。

08.【解答】

1）算法的基本设计思想：

采用邻接矩阵表示有向图时，一行中 1 的个数为该行对应顶点的出度，一列中 1 的个数为该列对应顶点的入度。使用一个初值为零的计数器记录 K 顶点的个数。对图 G 的每个顶点，根据邻接矩阵计算其出度 outdegree 和入度 indegree。若 outdegree-indegree>0，则输出该顶点且计数器加 1。最后返回计数器的值。

2）用 C 语言描述的算法：

```
int printVertices(MGraph G){
//采用邻接矩阵存储，输出 K 顶点，返回个数
    int indegree,outdegree,k,m,count=0;
    for(k=0;k<G.numVertices;k++){
```

```
            indegree=outdegree=0;
            for(m=0;m<G.numVertices;m++)      //计算顶点的出度
                outdegree+=G.Edge[k][m];
            for(m=0;m<G.numVertices;m++)      //计算顶点的入度
                indegree+=G.Edge[m][k];
            if(outdegree>indegree){
                printf("%c",G.VerticesList[k]);
                count++;
            }
        }
        return count;                         //返回 K 顶点的个数
    }
```

6.3　图的遍历

图的遍历是指从图中的某一顶点出发，按照某种搜索方法沿着图中的边对图中的所有顶点访问一次，且仅访问一次。注意到树是一种特殊的图，所以树的遍历实际上也可视为一种特殊的图的遍历。图的遍历算法是求解图的连通性问题、拓扑排序和求关键路径等算法的基础。

图的遍历比树的遍历要复杂得多，因为图的任意一个顶点都可能和其余的顶点相邻接，所以在访问某个顶点后，可能沿着某条路径搜索又回到该顶点。为避免同一顶点被访问多次，在遍历图的过程中，必须记下每个已访问过的顶点，为此可以设一个辅助数组 visited[]来标记顶点是否被访问过。图的遍历算法主要有两种：广度优先搜索和深度优先搜索。

6.3.1　广度优先搜索

广度优先搜索（Breadth-First-Search, BFS）类似于树的层序遍历。基本思想是：首先访问起始顶点 v，接着由 v 出发，依次访问 v 的各个未访问过的邻接顶点 $w_1, w_2, \cdots, w_i$，然后依次访问 $w_1, w_2, \cdots, w_i$ 的所有未被访问过的邻接顶点；再从这些访问过的顶点出发，访问它们所有未被访问过的邻接顶点，直至图中所有顶点都被访问过为止。若此时图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作为始点，重复上述过程，直至图中所有顶点都被访问到为止。Dijkstra 单源最短路径算法和 Prim 最小生成树算法也应用了类似的思想。

换句话说，广度优先搜索遍历图的过程是以 v 为起始点，由近至远依次访问和 v 有路径相通且路径长度为 1, 2,… 的顶点。广度优先搜索是一种分层的查找过程，每向前走一步可能访问一批顶点，不像深度优先搜索那样有往回退的情况，因此它不是一个递归的算法。为了实现逐层的访问，算法必须借助一个辅助队列，以记忆正在访问的顶点的下一层顶点。

广度优先搜索算法的伪代码如下：

```
bool visited[MAX_VERTEX_NUM];          //访问标记数组
void BFSTraverse(Graph G){             //对图 G 进行广度优先遍历
    for(i=0;i<G.vexnum;++i)
        visited[i]=FALSE;              //访问标记数组初始化
    InitQueue(Q);                      //初始化辅助队列 Q
    for(i=0;i<G.vexnum;++i)            //从 0 号顶点开始遍历
        if(!visited[i])                //对每个连通分量调用一次 BFS()
            BFS(G,i);                  //若 vi 未访问过，从 vi 开始调用 BFS()
}
```

用邻接表实现广度优先搜索的算法如下：

```
void BFS(ALGraph G,int i){
```

```
        visit(i);                          //访问初始顶点 i
        visited[i]=TRUE;                   //对 i 做已访问标记
        EnQueue(Q,i);                      //顶点 i 入队
        while(!IsEmpty(Q)){
            DeQueue(Q,v);                  //队首顶点 v 出队
            for(p=G.vertices[v].firstarc;p;p=p->nextarc){ //检测 v 的所有邻接点
                w=p->adjvex;
                if(visited[w]==FALSE){
                    visit(w);              // w 为 v 的尚未访问的邻接点，访问 w
                    visited[w]=TRUE;       //对 w 做已访问标记
                    EnQueue(Q,w);          //顶点 w 入队
                }
            }
        }
    }
```

用邻接矩阵实现广度优先搜索的算法如下：

```
    void BFS(MGraph G,int i){
        visit(i);                          //访问初始顶点 i
        visited[i]=TRUE;                   //对 i 做已访问标记
        EnQueue(Q,i);                      //顶点 i 入队
        while(!IsEmpty(Q)){
            DeQueue(Q,v);                  //队首顶点 v 出队
            for(w=0;w<G.vexnum;w++)        //检测 v 的所有邻接点
                if(visited[w]==FALSE&&G.edge[v][w]==1){
                    visit(w);              // w 为 v 的尚未访问的邻接点，访问 w
                    visited[w]=TRUE;       //对 w 做已访问标记
                    EnQueue(Q,w);          //顶点 w 入队
                }
        }
    }
```

辅助数组 visited[] 标志顶点是否被访问过，其初始状态为 FALSE。在图的遍历过程中，一旦某个顶点 v_i 被访问，就立即置 visited[i] 为 TRUE，防止它被多次访问。

命题追踪 ▶ 广度优先遍历的过程（2013）

下面通过实例演示广度优先搜索的过程，给定图 G 如图 6.11 所示。假设从顶点 a 开始访问，a 先入队。此时队列非空，取出队头元素 a，因为 b, c 与 a 邻接且未被访问过，于是依次访问 b, c，并将 b, c 依次入队。队列非空，取出队头元素 b，依次访问与 b 邻接且未被访问的顶点 d, e，并将 d, e 入队（注意：a 与 b 也邻接，但 a 已置访问标记，所以不再重复访问）。此时队列非空，取出队头元素 c，访问与 c 邻接且未被访问的顶点 f, g，并将 f, g 入队。此时，取出队头元素 d，但与 d 邻接且未被访问的顶点为空，所以不进行任何操作。继续取出队头元素 e，将 h 入队列……最终取出队头元素 h 后，队列为空，从而循环自动跳出。遍历结果为 $abcdefgh$。

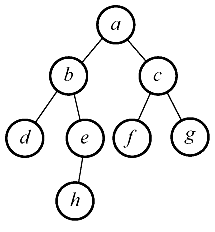


图 6.11　一个无向图 G

从上例不难看出，图的广度优先搜索的过程与二叉树的层序遍历是完全一致的，这也说明了图的广度优先搜索遍历算法是二叉树的层次遍历算法的扩展。

1．BFS 算法的性能分析

无论是邻接表还是邻接矩阵的存储方式，BFS 算法都需要借助一个辅助队列 Q，n 个顶点均需入队一次，在最坏的情况下，空间复杂度为 $O(|V|)$。

命题追踪 ▶ 基于邻接表存储的 BFS 的效率（2012）

遍历图的过程实质上是对每个顶点查找其邻接点的过程，耗费的时间取决于所采用的存储结构。采用邻接表存储时，每个顶点均需搜索（或入队）一次，时间复杂度为 $O(|V|)$，在搜索每个顶点的邻接点时，每条边至少访问一次，时间复杂度为 $O(|E|)$，总的时间复杂度为 $O(|V| + |E|)$。采用邻接矩阵存储时，查找每个顶点的邻接点所需的时间为 $O(|V|)$，总时间复杂度为 $O(|V|^2)$。

2．BFS 算法求解单源最短路径问题

若图 $G = (V, E)$为非带权图，定义从顶点 u 到顶点 v 的最短路径 $d(u, v)$为从 u 到 v 的任何路径中最少的边数；若从 u 到 v 没有通路，则 $d(u, v) = \infty$。

使用 BFS，我们可以求解一个满足上述定义的非带权图的单源最短路径问题，这是由广度优先搜索总是按照距离由近到远来遍历图中每个顶点的性质决定的。

BFS 算法求解单源最短路径问题的算法如下：

```
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从 u 到 i 结点的最短路径
    for(i=0;i<G.vexnum;++i)
        d[i]=∞;                          //初始化路径长度
    visited[u]=TRUE; d[u]=0;
    EnQueue(Q,u);
    while(!isEmpty(Q)){                  //BFS 算法主过程
        DeQueue(Q,u);                    //队头元素 u 出队
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w))
            if(!visited[w]){             //w 为 u 的尚未访问的邻接顶点
                visited[w]=TRUE;         //设已访问标记
                d[w]=d[u]+1;             //路径长度加 1
                EnQueue(Q,w);            //顶点 w 入队
            }
    }
}
```

3．广度优先生成树

在广度遍历的过程中，我们可以得到一棵遍历树，称为广度优先生成树，如图 6.12 所示。需要注意的是，同一个图的邻接矩阵存储表示是唯一的，所以其广度优先生成树也是唯一的，但因为邻接表存储表示不唯一，所以其广度优先生成树也是不唯一的。

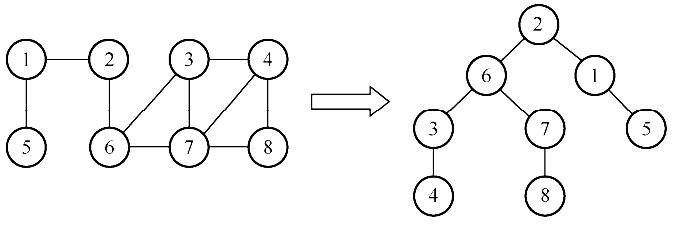


图 6.12　图的广度优先生成树

6.3.2 深度优先搜索

与广度优先搜索不同，深度优先搜索（Depth-First-Search，DFS）类似于树的先序遍历。如其名称中所暗含的意思一样，这种搜索算法所遵循的策略是尽可能“深”地搜索一个图。

它的基本思想如下：首先访问图中某一起始顶点 v，然后由 v 出发，访问与 v 邻接且未被访问的任意一个顶点 w_1，再访问与 w_1 邻接且未被访问的任意一个顶点 w_2……重复上述过程。当不能再继续向下访问时，依次退回到最近被访问的顶点，若它还有邻接顶点未被访问过，则从该点开始继续上述搜索过程，直至图中所有顶点均被访问过为止。

一般情况下，其递归形式的算法十分简洁，算法过程如下：

```
bool visited[MAX_VERTEX_NUM];          //访问标记数组
void DFSTraverse(Graph G){             //对图 G 进行深度优先遍历
    for(i=0;i<G.vexnum;i++)
        visited[i]=FALSE;              //初始化已访问标记数组
    for(i=0;i<G.vexnum;i++)            //本代码中是从 v0 开始遍历
        if(!visited[i])                //对尚未访问的顶点调用 DFS()
            DFS(G,i);
}
```

用邻接表实现深度优先搜索的算法如下：

```
void DFS(ALGraph G,int i){
    visit(i);                          //访问初始顶点 i
    visited[i]=TRUE;                   //对 i 做已访问标记
    for(p=G.vertices[i].firstarc;p;p=p->nextarc){ //检测 i 的所有邻接点
        j=p->adjvex;
        if(visited[j]==FALSE)
            DFS(G,j);                  // j 为 i 的尚未访问的邻接点，递归访问 j
    }
}
```

用邻接矩阵实现深度优先搜索的算法如下：

```
void DFS(MGraph G,int i){
    visit(i);                          //访问初始顶点 i
    visited[i]=TRUE;                   //对 i 做已访问标记
    for(j=0;j<G.vexnum;j++){           //检测 i 的所有邻接点
        if(visited[j]==FALSE&&G.edge[i][j]==1)
            DFS(G,j);                  // j 为 i 的尚未访问的邻接点，递归访问 j
    }
}
```

命题追踪 ▶ 深度优先遍历的过程（2015、2016）

以图 6.11 的无向图为例，深度优先搜索的过程：首先访问 a，并置 a 访问标记；然后访问与 a 邻接且未被访问的顶点 b，置 b 访问标记；然后访问与 b 邻接且未被访问的顶点 d，置 d 访问标记。此时 d 已没有未被访问过的邻接点，所以返回上一个访问的顶点 b，访问与其邻接且未被访问的顶点 e，置 e 访问标记，以此类推，直至图中所有顶点都被访问一次。遍历结果为 *abdehcfg*。

注 意

图的邻接矩阵表示是唯一的，但对邻接表来说，若边的输入次序不同，则生成的邻接表也不同。因此，对同样一个图，基于邻接矩阵的遍历得到的 DFS 序列和 BFS 序列是唯一的，基于邻接表的遍历得到的 DFS 序列和 BFS 序列是不唯一的。

1．DFS 算法的性能分析

DFS 算法是一个递归算法，需要借助一个递归工作栈，所以其空间复杂度为 $O(|V|)$。

遍历图的过程实质上是通过边查找邻接点的过程，因此两种遍历方式的时间复杂度都相同，不同之处仅在于对顶点访问顺序的不同。采用邻接矩阵存储时，总时间复杂度为 $O(|V|^2)$。采用邻接表存储时，总的时间复杂度为 $O(|V|+|E|)$。

2．深度优先的生成树和生成森林

与广度优先搜索一样，深度优先搜索也会产生一棵深度优先生成树。当然，这是有条件的，即对连通图调用 DFS 才能产生深度优先生成树，否则产生的将是深度优先生成森林，如图 6.13 所示。与 BFS 类似，基于邻接表存储的深度优先生成树是不唯一的。

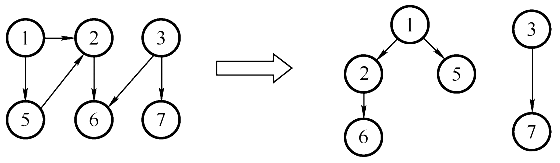


图 6.13　图的深度优先生成森林

6.3.3　图的遍历与图的连通性

图的遍历算法可以用来判断图的连通性。对于无向图来说，若无向图是连通的，则从任意一个结点出发，仅需一次遍历就能够访问图中的所有顶点；若无向图是非连通的，则从某一个顶点出发，一次遍历只能访问到该顶点所在连通分量的所有顶点，而对于图中其他连通分量的顶点，则无法通过这次遍历访问。对于有向图来说，若从初始顶点到图中的每个顶点都有路径，则能够访问到图中的所有顶点，否则不能访问到所有顶点。

因此，在 `BFSTraverse()` 或 `DFSTraverse()` 中添加了第二个 `for` 循环，再选取初始点，继续进行遍历，以防止一次无法遍历图的所有顶点。对于无向图，上述两个函数调用 `BFS(G,i)` 或 `DFS(G,i)` 的次数等于该图的连通分量数；而对于有向图则不是这样，因为一个连通的有向图分为强连通的和非强连通的，它的连通子图也分为强连通分量和非强连通分量，非强连通分量一次调用 `BFS(G,i)` 或 `DFS(G,i)` 不一定能访问到该子图的所有顶点，如图 6.14 所示。

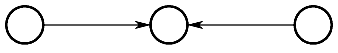


图 6.14　有向图的非强连通分量

6.3.4　本节试题精选

一、单项选择题

01. 下列关于广度优先算法的说法中，正确的是（　）。

I. 当各边的权值相等时，广度优先算法可以解决单源最短路径问题

II. 当各边的权值不等时，广度优先算法可用来解决单源最短路径问题

III. 广度优先遍历算法类似于树中的后序遍历算法

IV. 实现图的广度优先算法时，使用的数据结构是队列

A. I、IV　　B. II、III、IV　　C. II、IV　　D. I、III、IV

02. 下列关于图的说法中，错误的是（　）。

I. 对一个无向图进行深度优先遍历时，得到的深度优先遍历序列是唯一的

II. 若有向图不存在回路，即使不用访问标志位，同一结点也不会被访问两次

III. 采用深度优先遍历或拓扑排序算法可以判断一个有向图中是否有环（回路）

IV. 对任何非强连通图必须2次或以上调用广度优先遍历算法才可访问所有的顶点

A. I、II、III　　B. II、III　　C. I、II　　D. I、II、IV

03. 对于一个非连通无向图 G，采用深度优先遍历访问所有顶点，在 DFSTraverse 函数（见考点讲解 DFS 部分）中调用 DFS 的次数正好等于（　）。

A. 顶点数　　B. 边数　　C. 连通分量数　　D. 不确定

04. 对一个有 n 个顶点、e 条边的图采用邻接表表示时，进行 DFS 遍历的时间复杂度为（　），空间复杂度为（　）；进行 BFS 遍历的时间复杂度为（　），空间复杂度为（　）。

A. $O(n)$　　B. $O(e)$　　C. $O(n+e)$　　D. $O(1)$

05. 图的广度优先遍历算法中使用队列作为其辅助数据结构，那么在算法执行过程中，每个顶点的入队次数最多为（　）。

A. 1　　B. 2　　C. 3　　D. 4

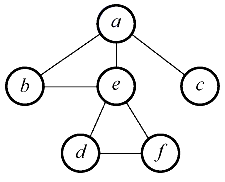
06. 对有 n 个顶点、e 条边的图采用邻接矩阵表示时，进行 DFS 遍历的时间复杂度为（　），进行 BFS 遍历的时间复杂度为（　）。

A. $O(n^2)$　　B. $O(e)$　　C. $O(n+e)$　　D. $O(e^2)$

07. 无向图 $G=(V, E)$，其中 $V=\{a, b, c, d, e, f\}$，$E=\{(a, b), (a, e), (a, c), (b, e), (c, f), (f, d), (e, d)\}$，对该图从 a 开始进行深度优先遍历，得到的顶点序列正确的是（　）。

A. a, b, e, c, d, f　　B. a, c, f, e, b, d　　C. a, e, b, c, f, d　　D. a, e, d, f, c, b

08. 如下图所示，在下面的5个序列中，符合深度优先遍历的序列个数是（　）。



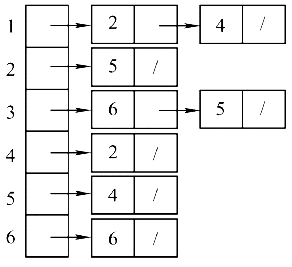
1. $aebfdc$　2. $acfdeb$　3. $aedfcb$　4. $aefdbc$　5. $aecfdb$

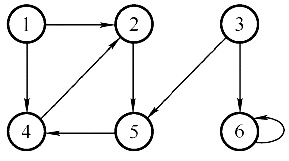
A. 5　　B. 4　　C. 3　　D. 2

09. 用邻接表存储的图的深度优先遍历算法类似于树的（　），而其广度优先遍历算法类似于树的（　）。

A. 中序遍历　　B. 先序遍历　　C. 后序遍历　　D. 按层次遍历

10. 一个有向图 G 的邻接表存储如下图所示，从顶点1出发，对图 G 调用深度优先遍历所得顶点序列是（　）；按广度优先遍历所得顶点序列是（　）。





A. 125436　　B. 124536　　C. 124563　　D. 362514

11. 无向图 $G=(V,E)$，其中 $V=\{a,b,c,d,e,f\}$，$E=\{(a,b),(a,e),(a,c),(b,e),(c,f),(f,d),(e,d)\}$。对该图进行深度优先遍历，不能得到的序列是（　）。

A. *acfdeb*　　B. *aebdfc*　　C. *aedfcb*　　D. *abecdf*

12. 判断有向图中是否存在回路，除拓扑排序外，还可以利用（　）。（注：涉及下节内容）

A. 求关键路径的方法　　B. 求最短路径的 Dijkstra 算法

C. 深度优先遍历算法　　D. 广度优先遍历算法

13. 设无向图 $G=(V,E)$和 $G'=(V',E')$，若 G'是 G 的生成树，则下列说法错误的是（　）。

A. G'为 G 的子图　　B. G'为 G 的连通分量

C. G'为 G 的极小连通子图且 $V=V'$　　D. G'是 G 的一个无环子图

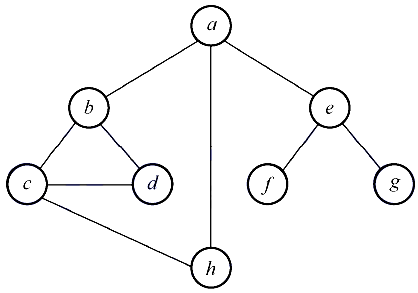
14. 图的广度优先生成树的树高比深度优先生成树的树高（　）。

A. 小或相等　　B. 小　　C. 大或相等　　D. 大

15. 【2012 统考真题】对有 n 个顶点、e 条边且使用邻接表存储的有向图进行广度优先遍历，其算法的时间复杂度是（　）。

A. $O(n)$　　B. $O(e)$　　C. $O(n+e)$　　D. $O(ne)$

16. 【2013 统考真题】下列选项中，不是如下无向图的广度优先遍历序列的是（　）。



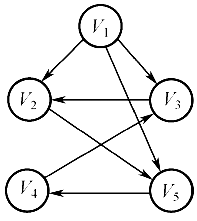
A. *h, c, a, b, d, e, g, f*　　B. *e, a, f, g, b, h, c, d*

C. *d, b, c, a, h, e, f, g*　　D. *a, b, c, d, h, e, f, g*

17. 【2015 统考真题】设有向图 $G=(V,E)$，顶点集 $V=\{V_0,V_1,V_2,V_3\}$，边集 $E=\{<v_0,v_1>,<v_0,v_2>,<v_0,v_3>,<v_1,v_3>\}$。若从顶点 V_0 开始对图进行深度优先遍历，则可能得到的不同遍历序列个数是（　）。

A. 2　　B. 3　　C. 4　　D. 5

18. 【2016 统考真题】下列选项中，不是下图深度优先搜索序列的是（　）。

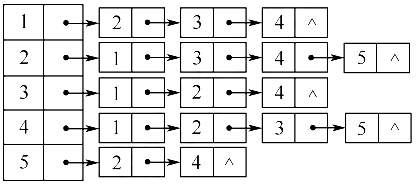


A. V_1, V_5, V_4, V_3, V_2　　B. V_1, V_3, V_2, V_5, V_4

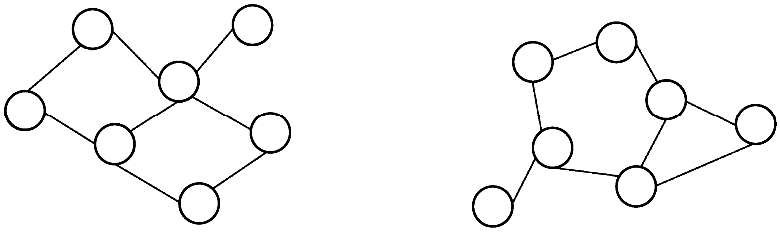
C. V_1, V_2, V_5, V_4, V_3　　D. V_1, V_2, V_3, V_4, V_5

二、综合应用题

01. 图 $G=(V,E)$以邻接表存储，如下图所示，试画出图 G 的深度优先生成树和广度优先生成树（假设从结点 1 开始遍历）。



02. 给定一个连通无向图，采用邻接表存储，将图的所有顶点分别染成红色或蓝色，若存在一种染色方法使图中每条边的两个顶点的颜色都不相同，则称这个图能被二分。



1）判断上面两个无向图是否能被二分，若能被二分，则请标出每个顶点的颜色。

2）请设计一种算法用来判断图是否能被二分，仅用语言描述算法的思想即可。

3）给出你设计的算法的时间复杂度和空间复杂度。

03. 试设计一个算法，判断一个无向图 G 是否为一棵树。若是一棵树，则算法返回 `true`，否则返回 `false`。

04. 分别采用基于深度优先遍历和广度优先遍历算法判别以邻接表方式存储的有向图中是否存在由顶点 v_i 到顶点 v_j 的路径（$i \neq j$）。注意，算法中涉及的图的基本操作必须在此存储结构上实现。

05. 假设图用邻接表表示，设计一个算法，输出从顶点 V_i 到顶点 V_j 的所有简单路径。

6.3.5 答案与解析

一、单项选择题

01. A

广度优先搜索以起始结点为中心，一层一层地向外层扩展遍历图的顶点，因此无法考虑到边权值，只适合求边权值相等的图的单源最短路径。广度优先搜索相当于树的层序遍历，选项 III 错误。广度优先搜索需要用到队列，深度优先搜索需要用到栈，选项 IV 正确。

02. D

图的深度优先遍历序列通常是不唯一的，选项 I 错误。图 1 是一个不存在回路的有向图，从顶点 1 开始执行广度优先遍历，若不设置访问标志位，则会重复访问顶点 3，选项 II 错误。深度优先遍历（见本节后面习题的解析）或拓扑排序算法可以判断有向图中是否有环，选项 III 正确。图 2 是一个非强连通图，但从顶点 1 开始调用一次广度优先遍历算法就可访问所有顶点，选项 IV 错误。

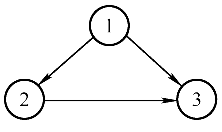


图 1

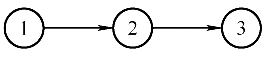


图 2

03. C

DFS（或 BFS）可用来计算无向图的连通分量数，因为一次遍历必然会将一个连通图中的所有顶点都访问到，所以计算图的连通分量数正好是 DFSTraverse()中 DFS 被调用的次数。

04．C、A、C、A

深度优先遍历时，每个顶点表结点和每个边表结点均查找一次，每个顶点递归调用一次，需要借助一个递归工作栈；而广度优先遍历时，也是每个顶点表结点和每个边表结点均查找一次，需要借助一个辅助队列。因此，时间复杂度都是 $O(n+e)$，空间复杂度都是 $O(n)$。

05．A

在图的广度优先遍历算法中，每个顶点被访问后立即做访问标记并入队。若队列不空，则队首顶点出队，若该顶点的邻接顶点未被访问，则访问之，做访问标记并入队；若被访问过，则跳过，如此反复，直至队空。因此，在广度优先遍历过程中，每个顶点最多入队一次。

06．A、A

采用邻接矩阵表示时，查找一个顶点所有出边的时间复杂度为 $O(n)$，共有 n 个顶点，所以时间复杂度均为 $O(n^2)$。

07．D

画出草图后，此类题可以根据边的邻接关系快速排除错误选项。以选项 A 为例，在遍历到 e 之后，应该访问与 e 邻接但未被访问的结点，(e, c)显然不在边集中。

08．D

仅 1 和 4 正确。以 2 为例，遍历到 c 之后，与 c 邻接且未被访问的结点为空集，所以应为 a 的邻接点 b 或 e 入栈。以 3 为例，因为遍历要按栈退回，所以是先 b 后 c，而不能先 c 后 b。

09．B、D

图的深度优先搜索类似于树的先根遍历，即先访问结点，再递归向外层结点遍历，都采用回溯算法。图的广度优先搜索类似于树的层序遍历，即一层一层向外层扩展遍历，都需要采用队列来辅助算法的实现。

10．A、B

DFS 序列产生的路径为<1, 2>, <2, 5>, <5, 4>, <3, 6>；BFS 序列产生的路径为<1, 2>, <1, 4>, <2, 5>, <3, 6>。

11．D

画出 V 和 E 对应的图 G，然后根据搜索算法求解。

注　意

为什么本题序列是不唯一的，而上题序列却是唯一的呢？

因为上题给出了具体的存储结构，此时就必须按照算法的过程来执行，每个顶点的邻接点的顺序已固定，但本题中每个顶点的邻接点的顺序是非固定的。

12．C

利用深度优先遍历可以判断图 G 中是否存在回路。

对于无向图来说，若深度优先遍历过程中遇到了回边，则必定存在环；对于有向图来说，这条回边可能是指向深度优先森林中另一棵生成树上的顶点的弧；但是，从有向图的某个顶点 v 出发进行深度优先遍历时，若在 DFS(v)结束之前出现一条从顶点 u 到顶点 v 的回边，且 u 在生成树上是 v 的子孙，则有向图必定存在包含顶点 v 和顶点 u 的环。

13．B

连通分量是无向图的极大连通子图，其中极大的含义是将依附于连通分量中顶点的所有边都加上，所以连通分量中可能存在回路，这样就不是生成树了。

注 意

极大连通子图是无向图（不一定连通）的连通分量，极小连通子图是连通无向图的生成树。极小和极大是在满足连通的前提下，针对边的数目而言的。极大连通子图包含连通分量的全部边；极小连通子图（生成树）包含连通图的全部顶点，且使其连通的边数最少。

14．A

对于无向图的广度优先搜索生成树，起点到其他顶点的路径是图中对应的最短路径，即所有生成树中树高最小。此外，深度优先总是尽可能“深”地搜索图，因此其路径也尽可能长，所以深度优先生成树的树高总是大于或等于广度优先生成树的树高。

15．C

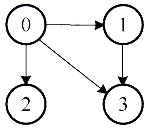
广度优先遍历需要借助队列实现。采用邻接表存储方式对图进行广度优先遍历时，每个顶点均需入队一次（顶点表遍历），所以时间复杂度为 $O(n)$，在搜索所有顶点的邻接点的过程中，每条边至少访问一次（出边表遍历），所以时间复杂度为 $O(e)$，算法总的时间复杂度为 $O(n+e)$。

16．D

只要掌握 DFS 和 BFS 的遍历过程，便能轻易解决。逐个代入，手工模拟，选项 D 是深度优先遍历，而不是广度优先遍历。

17．D

画出该有向图，如下图所示。采用图的深度优先遍历，共有 5 种可能：$<v_0, v_1, v_3, v_2>$，$<v_0, v_2, v_3, v_1>$，$<v_0, v_2, v_1, v_3>$，$<v_0, v_3, v_2, v_1>$，$<v_0, v_3, v_1, v_2>$。



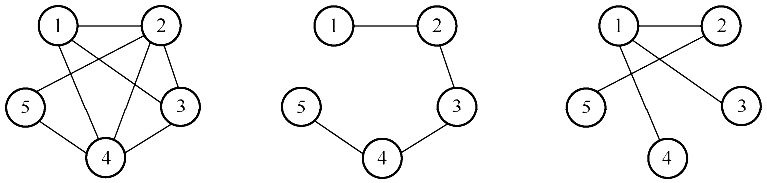
18．D

按深度优先遍历的策略进行遍历。对于选项 A：先访问 V_1，然后访问与 V_1 邻接且未被访问的任意一个顶点（满足的有 V_2, V_3 和 V_5），此时访问 V_5，然后从 V_5 出发，访问与 V_5 邻接且未被访问的任意一个顶点（满足的只有 V_4），然后从 V_4 出发，访问与 V_4 邻接且未被访问的任意一个顶点（满足的只有 V_3），然后从 V_3 出发，访问与 V_3 邻接且未被访问的任意一个顶点（满足的只有 V_2），结束遍历。选项 B 和 C 的分析方法与 A 相同。对于选项 D，首先访问 V_1，然后从 V_1 出发，访问与 V_1 邻接且未被访问的任意一个顶点（满足的有 V_2, V_3 和 V_5），然后从 V_2 出发，访问与 V_2 邻接且未被访问的任意一个顶点（满足的只有 V_5），按规则本应该访问 V_5，但选项 D 却访问了 V_3，错误。

二、综合应用题

01.【解答】

根据 G 的邻接表不难画出图(a)。



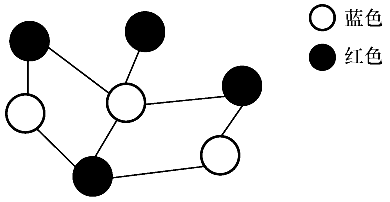
(a) 图G　　(b) 深度优先生成树　　(c) 广度优先生成树

1）采用深度优先遍历。深度优先搜索总是尽可能“深”地搜索图，根据存储结构可知深度优先搜索的路径次序为(1, 2), (2, 3), (3, 4), (4, 5)，深度优先生成树如图(b)所示。需要注意的是，当存储结构固定时，生成树的树形也就固定了，比如不能先搜索(1, 3)。

2）采用广度优先遍历。广度优先搜索总是尽可能“广”地搜索图，一层一层地向外扩展，根据存储结构可知广度优先搜索的路径次序为(1, 2), (1, 3), (1, 4), (2, 5)，广度优先生成树如图(c)所示。

02.【解答】

1）右图不能被二分，左图能被二分，染色情况如下图所示。



2）从任意一个结点开始，将其染成红色，并从该结点开始对整个图进行遍历，在遍历过程中，若当前遍历的结点 a 有一条边指向 b，则可能出现三种情况：①b 未被染色，将它染成与结点 a 不同的颜色，并且继续遍历与 b 相连的结点；②a 与 b 的颜色相同，说明该图不能被二分，直接返回；③ a 与 b 的颜色不同，跳过 b 点。

3）上述遍历无论是使用深度优先还是使用广度优先，时间复杂度都为 $O(n + m)$，其中的 n 和 m 分别是顶点数和边数。需要一个数组来存储各结点的颜色及是否已访问，空间复杂度为 $O(n)$。

03.【解答】

一个无向图 G 是一棵树的条件是，G 必须是无回路的连通图或有 $n-1$ 条边的连通图。这里采用后者作为判断条件。对连通的判定，可以用能否一次遍历全部顶点来实现。可以采用深度优先搜索算法在遍历图的过程中统计可能访问到的顶点个数和边的条数，若一次遍历就能访问到 n 个顶点和 $n-1$ 条边，则可断定此图是一棵树。算法实现如下：

```
bool isTree(Graph& G){
    for(i=1;i<=G.vexnum;i++)
        visited[i]=FALSE;              //访问标记 visited[]初始化
    int Vnum=0,Enum=0;                 //记录顶点数和边数
    DFS(G,1,Vnum,Enum,visited);
    if(Vnum==G.vexnum&&Enum==2*(G.vexnum-1))
        return  true;                  //符合树的条件
    else
        return  false;                 //不符合树的条件
}
void DFS(Graph& G,int v,int& Vnum,int& Enum,int visited[]){
//深度优先遍历图G，统计访问过的顶点数和边数，通过 Vnum 和 Enum 返回
    visited[v]=TRUE;Vnum++;            //作访问标记，顶点计数
    int w=FirstNeighbor(G,v);          //取 v 的第一个邻接顶点
    while(w!=-1){                      //当邻接顶点存在
        Enum++;                        //边存在，边计数
        if(!visited[w])                //当该邻接顶点未访问过
            DFS(G,w,Vnum,Enum,visited);
        w=NextNeighbor(G,v,w);
    }
}
```

04.【解答】

两个不同的遍历算法都采用从顶点 v_i 出发，依次遍历图中每个顶点，直到搜索到顶点 v_j，若能够搜索到 v_j，则说明存在由顶点 v_i 到顶点 v_j 的路径。

深度优先遍历算法的实现如下：

```
int visited[MAXSIZE]={0};     //访问标记数组
void DFS(ALGraph G,int i,int j,bool &can_reach){
//深度优先判断有向图 G 中顶点 vi 到顶点 vj 是否有路径，用 can_reach 来标识
    if(i==j){
        can_reach=true;
        return;                //i 就是 j
    }
    visited[i]=1;              //置访问标记
    for(int p=FirstNeighbor(G,i);p>=0;p=NextNeighbor(G,i,p))
        if(!visited[p]&&!can_reach)     //递归检测邻接点
            DFS(G,p,j,can_reach);
}
```

广度优先遍历算法的实现如下：

```
int visited[MAXSIZE]={0};          //访问标记数组
int BFS(ALGraph G,int i,int j){
//广度优先判断有向图 G 中顶点 vi 到顶点 vj 是否有路径，若是，则返回 1，否则返回 0
    InitQueue(Q); EnQueue(Q,i);  //顶点 i 入队
    while(!isEmpty(Q)){          //非空循环
        DeQueue(Q,i);            //队头顶点出队
        visited[i]=1;            //置访问标记
        if(i==j)  return 1;
        for(int p=FirstNeighbor(G,i);p;p=NextNeighbor(G,i,p)){
                                 //检查所有邻接点
            if(p==j)             //若 p==j，则查找成功
                return 1;
            if(!visited[p]){     //否则，顶点 p 入队
                EnQueue(Q,p);
                visited[p]=1;
            }
        }
    }
 return  0;
}
```

本题也可以这样解答：调用以 i 为参数的 DFS(G,i) 或 BFS(G,i)，执行结束后判断 visited[j] 是否为 TRUE，若是，则说明 v_j 已被遍历，图中必存在由 v_i 到 v_j 的路径。但此种解法每次都耗费最坏时间复杂度对应的时间，需要遍历与 v_i 连通的所有顶点。

05.【解答】

本题采用基于递归的深度优先遍历算法，从结点 u 出发，递归深度优先遍历图中结点，若访问到结点 v，则输出该搜索路径上的结点。为此，设置一个 path 数组来存放路径上的结点（初始为空），d 表示路径长度（初始为-1）。查找从顶点 u 到 v 的简单路径过程说明如下（假设查找函数名为 FindPath()）：

1）FindPath(G,u,v,path,d):d++;path[d]=u；若找到 u 的未访问过的相邻结点 u1，则继续下去，否则置 visited[u]=0 并返回。

2）FindPath(G,u1,v,path,d):d++;path[d]=u1；若找到 u1 的未访问过的相邻结点 u2，则继续下去，否则置 visited[u1]=0。

3）以此类推，继续上述递归过程，直到 ui=v，输出 path。

算法实现如下：

```
 void FindPath(AGraph *G,int u,int v,int path[],int d){
```

```
    int w;
    ArcNode *p;
    d++;                              //路径长度增 1
    path[d]=u;                        //将当前顶点添加到路径中
    visited[u]=1;                     //置已访问标记
    if(u==v)                          //找到一条路径则输出
        print(path[]);                //输出路径上的结点
    p=G->adjlist[u]. firstarc;        //p 指向 u 的第一个相邻点
    while(p!=NULL){
        w=p->adjvex;                  //若顶点 w 未访问，递归访问它
        if(visited[w]==0)
            FindPath(G,w,v,path,d);
        p=p->nextarc;                 //p 指向 u 的下一个相邻点
    }
    visited[u]=0;                     //恢复环境，使该顶点可重新使用
}
```

6.4　图的应用

本节是历年考查的重点。图的应用主要包括：最小生成（代价）树、最短路径、拓扑排序和关键路径。一般而言，这部分内容直接以算法设计题形式考查的可能性偏小，而更多的是结合图的实例来考查算法的具体操作过程，读者必须学会手工模拟给定图的各个算法的执行过程。此外，还需掌握对给定模型建立相应的图去解决问题的方法。

6.4.1　最小生成树

一个连通图的生成树包含图的所有顶点，并且只含尽可能少的边。对于生成树来说，若砍去一条边，则会使生成树变成非连通图；若增加一条边，则会在图中形成一条回路。

对于一个带权连通无向图 G，生成树不同，每棵树的权（树中所有边的权值之和）也可能不同。权值之和最小的那棵生成树称为 G 的最小生成树（Minimum-Spanning-Tree, MST）。

命题追踪 ▶ 最小生成树的性质（2012、2017）　公众号：小兔网盘　免费分享无水印PDF

不难看出，最小生成树具有如下性质：

1）若图 G 中存在权值相同的边，则 G 的最小生成树可能不唯一，即最小生成树的树形不唯一。当图 G 中的各边权值互不相等时，G 的最小生成树是唯一的；若无向连通图 G 的边数比顶点数少 1，即 G 本身是一棵树时，则 G 的最小生成树就是它本身。

2）虽然最小生成树不唯一，但其对应的边的权值之和总是唯一的，而且是最小的。

3）最小生成树的边数为顶点数减 1。

命题追踪 ▶ 最小生成树中某顶点到其他顶点是否具有最短路径的分析（2023）

注　意

最小生成树中所有边的权值之和最小，但不能保证任意两个顶点之间的路径是最短路径。如下图所示，最小生成树中 A 到 C 的路径长度为 5，但图中 A 到 C 的最短路径长度为 4。

构造最小生成树有多种算法，但大多数算法都利用了最小生成树的下列性质：假设 $G=(V, E)$ 是一个带权连通无向图，U 是顶点集 V 的一个非空子集。若(u, v)是一条具有最小权值的边，其中 $u\in U$，$v\in V-U$，则必存在一棵包含边(u, v)的最小生成树。

基于该性质的最小生成树算法主要有 Prim 算法和 Kruskal 算法，它们都基于贪心算法的策略。对这两种算法应主要掌握算法的本质含义和基本思想，并能动手模拟算法的实现步骤。

下面介绍一个通用的最小生成树算法：

```
GENERIC_MST(G){
    T=NULL;
    while T 未形成一棵生成树;
        do 找到一条最小代价边(u,v)并且加入 T 后不会产生回路;
            T=T∪(u,v);
}
```

通用算法每次加入一条边以逐渐形成一棵生成树，下面介绍两种实现上述通用算法的途径。

1．Prim 算法

Prim（普里姆）算法的执行非常类似于寻找图的最短路径的 Dijkstra 算法（见下一节）。

命题追踪 ▶ Prim 算法构造最小生成树的实例（2015、2017、2018）

Prim 算法构造最小生成树的过程如图 6.15 所示。初始时从图中任取一顶点（如顶点 1）加入树 T，此时树中只含有一个顶点，之后选择一个与当前 T 中顶点集合距离最近的顶点，并将该顶点和相应的边加入 T，每次操作后 T 中的顶点数和边数都增 1。以此类推，直至图中所有的顶点都并入 T，得到的 T 就是最小生成树。此时 T 中必然有 $n-1$ 条边。

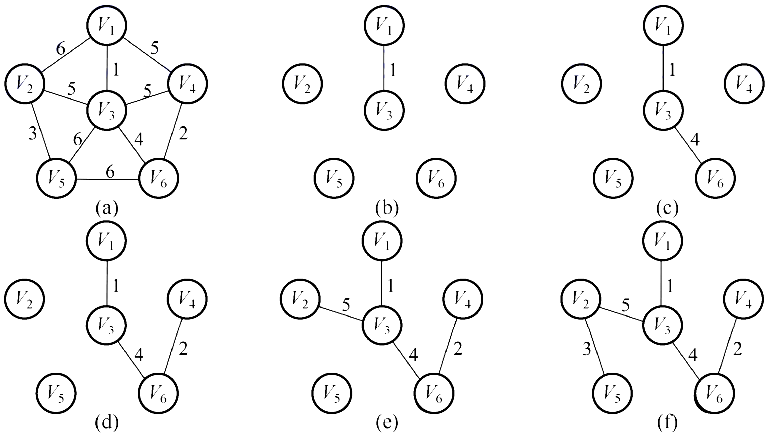


图 6.15　Prim 算法构造最小生成树的过程

Prim 算法的步骤如下：

假设 $G=\{V, E\}$是连通图，其最小生成树 $T=(U, E_T)$，E_T是最小生成树中边的集合。

初始化：向空树 $T=(U, E_T)$中添加图 $G=(V, E)$的任意一个顶点 u_0，使 $U=\{u_0\}$，$E_T=\varnothing$。

循环（重复下列操作直至 $U=V$）：从图 G 中选择满足$\{(u, v)|u\in U, v\in V-U\}$且具有最小权值的边$(u, v)$，加入树 T，置 $U=U\cup\{v\}$，$E_T=E_T\cup\{(u, v)\}$。

Prim 算法的简单实现如下：

```
void Prim(G,T){
    T=∅;                          //初始化空树
    U={w};                        //添加任意一个顶点 w
```

```
        while((V-U)!=∅){          //若树中不含全部顶点
            设(u,v)是使 u∈U 与 v∈(V-U),且权值最小的边;
            T=T∪{(u,v)};          //边归入树
            U=U∪{v};              //顶点归入树
        }
    }
```

Prim 算法的时间复杂度为 $O(|V|^2)$，不依赖于$|E|$，因此它适用于求解边稠密的图的最小生成树。虽然采用其他方法能改进 Prim 算法的时间复杂度，但增加了实现的复杂性。

2．Kruskal 算法

与 Prim 算法从顶点开始扩展最小生成树不同，Kruskal 算法是一种按权值的递增次序选择合适的边来构造最小生成树的方法。

命题追踪 ▶ Kruskal 算法构造最小生成树的实例（2015、2018、2020）

Kruskal 算法构造最小生成树的过程如图 6.16 所示。初始时为只有 n 个顶点而无边的非连通图 $T=\{V, \{\}\}$，每个顶点自成一个连通分量。然后按照边的权值由小到大的顺序，不断选取当前未被选取过且权值最小的边，若该边依附的顶点落在 T 中不同的连通分量上（使用并查集判断这两个顶点是否属于同一棵集合树），则将此边加入 T，否则舍弃此边而选择下一条权值最小的边。以此类推，直至 T 中所有顶点都在一个连通分量上。

Kruskal 算法的步骤如下：

假设 $G=(V, E)$是连通图，其最小生成树 $T=(U, E_T)$。

初始化：$U=V, E_T=\varnothing$。即每个顶点构成一棵独立的树，T此时是一个仅含$|V|$个顶点的森林。

循环（重复直至 T 是一棵树）：按 G 的边的权值递增顺序依次从 $E-E_T$ 中选择一条边，若这条边加入 T 后不构成回路，则将其加入 E_T，否则舍弃，直到 E_T中含有 $n-1$ 条边。

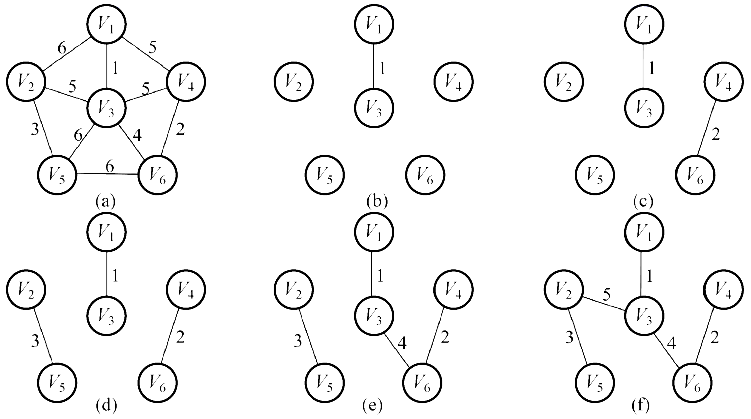


图 6.16　Kruskal 算法构造最小生成树的过程

Kruskal 算法的简单实现如下：

```
void Kruskal(V,T){
    T=V;                          //初始化树 T，仅含顶点
    numS=n;                       //连通分量数
    while(numS>1){                //若连通分量数大于 1
        从 E 中取出权值最小的边(v,u);
        if(v 和 u 属于 T 中不同的连通分量){
        T=T∪{(v,u)};              //将此边加入生成树中
        numS--;                   //连通分量数减 1
```

```
            }
        }
    }
```

根据图的相关性质，若一条边连接了两棵不同树中的顶点，则对这两棵树来说，它必定是连通的，将这条边加入森林中，完成两棵树的合并，直到整个森林合并成一棵树。

在 Kruskal 算法中，最坏情况需要对$|E|$条边各扫描一次。通常采用堆（见第 7 章）来存放边的集合，每次选择最小权值的边需要 $O(\log_2|E|)$的时间；每次使用并查集来快速判断两个顶点是否属于一个集合所需的时间为 $O(\alpha(|V|))$，$\alpha(|V|)$的增长极其缓慢，可视为常数。算法的总时间复杂度为 $O(|E|\log_2|E|)$，不依赖于$|V|$，因此 Kruskal 算法适合于边稀疏而顶点较多的图。

6.4.2 最短路径

命题追踪 ▶ 最短路径的分析与举例以及相关的算法（2009、2023）

6.3 节所述的广度优先搜索查找最短路径只是对无权图而言的。当图是带权图时，把从一个顶点 v_0 到图中其余任意一个顶点 v_i 的一条路径所经过边上的权值之和，定义为该路径的带权路径长度，把带权路径长度最短的那条路径（可能不止一条）称为最短路径。

求解最短路径的算法通常都依赖于一种性质，即两点之间的最短路径也包含了路径上其他顶点间的最短路径。带权有向图 G 的最短路径问题一般可分为两类：一是单源最短路径，即求图中某一顶点到其他各顶点的最短路径，可通过经典的 Dijkstra（迪杰斯特拉）算法求解；二是求每对顶点间的最短路径，可通过 Floyd（弗洛伊德）算法来求解。

1．Dijkstra 算法求单源最短路径问题

Dijkstra 算法设置一个集合 S 记录已求得的最短路径的顶点，初始时把源点 v_0 放入 S，集合 S 每并入一个新顶点 v_i，都要修改源点 v_0 到集合 $V-S$ 中顶点当前的最短路径长度值（这里可能不太好理解？没关系，继续往下看，相信会逐步理解）。

在构造的过程中还设置了三个辅助数组：

- `final[]`：标记各顶点是否已找到最短路径，即是否归入集合 S。
- `dist[]`：记录从源点 v_0 到其他各顶点当前的最短路径长度，它的初始值为：若从 v_0 到 v_i 有弧，则 `dist[i]`为弧上的权值；否则置 `dist[i]`为∞。
- `path[]`：`path[i]`表示从源点到顶点 i 之间的最短路径的前驱结点。在算法结束时，可根据其值追溯得到源点 v_0 到顶点 v_i 的最短路径。

假设从顶点 0 出发，即 $v_0=0$，集合 S 最初只包含顶点 0，邻接矩阵 `arcs` 表示带权有向图，`arcs[i][j]`表示有向边$<i,j>$的权值，若不存在有向边$<i,j>$，则 `arcs[i][j]`为∞。

Dijkstra 算法的步骤如下（不考虑对 `path[]`的操作）：

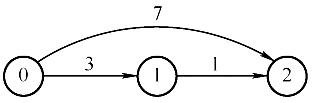
1）初始化：集合 S 初始为{0}，`dist[]`的初始值 `dist[i]=arcs[0][i]`，$i=1,2,\cdots,n-1$。

2）从顶点集合 $V-S$ 中选出 v_j，满足 `dist[j]=Min{dist[i]|` $v_i\in V-S$`}`，v_j 就是当前求得的一条从 v_0 出发的最短路径的终点，令 $S=S\cup\{j\}$。

3）修改从 v_0 出发到集合 $V-S$ 上任意一个顶点 v_k 可达的最短路径长度：若 `dist[j]+arcs[j][k]<dist[k]`，则更新 `dist[k]=dist[j]+arcs[j][k]`。

4）重复 2）～3）操作共 $n-1$ 次，直到所有的顶点都包含在集合 S 中。

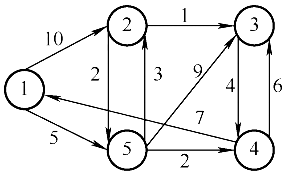
步骤 3）也就是开头留下的疑问，每当一个顶点加入集合 S 后，可能需要修改源点 v_0 到集合 $V-S$ 中可达顶点当前的最短路径长度，下面举一简单例子证明。如下图所示，源点为 v_0，初始时 $S=\{v_0\}$，`dist[1]=3`，`dist[2]=7`，当将 v_1 并入集合 S 后，`dist[2]`需要更新为 4。



思考：Dijkstra 算法与 Prim 算法有何异同之处？[①]

命题追踪 ▶ Dijkstra 算法求解最短路径的实例（2012、2014、2016、2021）

例如，对图 6.17 中的图应用 Dijkstra 算法求从顶点 1 出发至其余顶点的最短路径的过程，如表 6.2 所示。算法执行过程的说明如下。



每轮得到的最短路径如下：
第1轮：1→5，路径距离为5
第2轮：1→5→4，路径距离为7
第3轮：1→5→2，路径距离为8
第4轮：1→5→2→3，路径距离为9

图 6.17　应用 Dijkstra 算法图

表 6.2　从 v_1 到各终点的 dist 值和最短路径的求解过程

顶点	第 1 轮	第 2 轮	第 3 轮	第 4 轮
2	10 $v_1 \rightarrow v_2$	8 $v_1 \rightarrow v_5 \rightarrow v_2$	**8** **$v_1 \rightarrow v_5 \rightarrow v_2$**	
3	∞	14 $v_1 \rightarrow v_5 \rightarrow v_3$	13 $v_1 \rightarrow v_5 \rightarrow v_4 \rightarrow v_3$	**9** **$v_1 \rightarrow v_5 \rightarrow v_2 \rightarrow v_3$**
4	∞	**7** **$v_1 \rightarrow v_5 \rightarrow v_4$**		
5	**5** **$v_1 \rightarrow v_5$**			
集合 S	**{1, 5}**	**{1, 5, 4}**	**{1, 5, 4, 2}**	**{1, 5, 4, 2, 3}**

初始化：集合 S 初始为 $\{v_1\}$，v_1 可达 v_2 和 v_5，v_1 不可达 v_3 和 v_4，因此 dist[]数组各元素的初始值依次设置为 dist[2]=10，dist[3]=∞，dist[4]=∞，dist[5]=5。

第 1 轮：选出最小值 dist[5]，将顶点 v_5 并入集合 S，即此时已找到 v_1 到 v_5 的最短路径。当 v_5 加入 S 后，从 v_1 到集合 $V-S$ 中可达顶点的最短路径长度可能会产生变化。因此需要更新 dist[]数组。v_5 可达 v_2，因 $v_1 \rightarrow v_5 \rightarrow v_2$ 的距离 8 比 dist[2]=10 小，更新 dist[2]=8；v_5 可达 v_3，$v_1 \rightarrow v_5 \rightarrow v_3$ 的距离 14，更新 dist[3]=14；v_5 可达 v_4，$v_1 \rightarrow v_5 \rightarrow v_4$ 的距离 7，更新 dist[4]=7。

第 2 轮：选出最小值 dist[4]，将顶点 v_4 并入集合 S。继续更新 dist[]数组。v_4 不可达 v_2，dist[2]不变；v_4 可达 v_3，$v_1 \rightarrow v_5 \rightarrow v_4 \rightarrow v_3$ 的距离 13 比 dist[3]小，故更新 dist[3]=13。

第 3 轮：选出最小值 dist[2]，将顶点 v_2 并入集合 S。继续更新 dist[]数组。v_2 可达 v_3，$v_1 \rightarrow v_5 \rightarrow v_2 \rightarrow v_3$ 的距离 9 比 dist[3]小，更新 dist[3]=9。

第 4 轮：选出唯一最小值 dist[3]，将顶点 v_3 并入集合 S，此时全部顶点都已包含在 S 中。

显然，Dijkstra 算法也是基于贪心策略的。

① Dijkstra 算法的流程、操作与 Prim 算法的都很相似，都基于贪心策略。区别在于，i. 目的不同：Dijkstra 算法的目的是构建单源点的最短路径树；Prim 算法的目的是构建最小生成树。ii. 算法思路略有不同：Prim 算法从一个点开始，每次选择权值最小的边，将其连接到已构建的生成树上，直至所有顶点都已加入；而 Dijkstra 算法每次找出到源点距离最近且未归入集合的点，并把它归入集合，同时以这个点为基础更新从源点到其他所有顶点的距离。iii. 适用的图不同：Prim 算法只能用于带权无向图；Dijkstra 算法可用于带权有向图或带权无向图。

使用邻接矩阵表示时，时间复杂度为 $O(|V|^2)$。使用带权的邻接表表示时，虽然修改 dist[] 的时间可以减少，但因为在 dist[] 中选择最小分量的时间不变，所以时间复杂度仍为 $O(|V|^2)$。

人们可能只希望找到从源点到某个特定顶点的最短路径，但这个问题和求解源点到其他所有顶点的最短路径一样复杂，时间复杂度也为 $O(|V|^2)$。

注意，边上带有负权值时，Dijkstra 算法并不适用。若允许边上带有负权值，则在与集合 S（已求得最短路径的顶点集，归入 S 内的顶点的最短路径不再变更）内某顶点（记为 a）以负边相连的顶点（记为 b）确定其最短路径时，其最短路径长度加上这条负边的权值结果可能小于 a 原先确定的最短路径长度，而此时 a 在 Dijkstra 算法下是无法更新的。例如，对于图 6.18 所示的带权有向图，利用 Dijkstra 算法不一定能得到正确的结果。

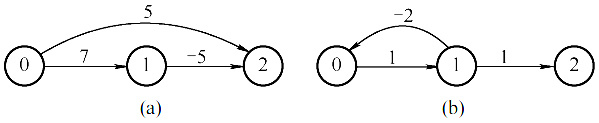


图 6.18　边上带有负权值的有向带权图

2．Floyd 算法求各顶点之间最短路径问题

求所有顶点之间的最短路径问题描述如下：已知一个各边权值均大于 0 的带权有向图，对任意两个顶点 $v_i \neq v_j$，要求求出 v_i 与 v_j 之间的最短路径和最短路径长度。

Floyd 算法的基本思想是：递推产生一个 n 阶方阵序列 $A^{(-1)}, A^{(0)}, \cdots, A^{(k)}, \cdots, A^{(n-1)}$，其中 $A^{(k)}[i][j]$表示从顶点 v_i 到顶点 v_j 的路径长度，k 表示绕行第 k 个顶点的运算步骤。初始时，对于任意两个顶点 v_i 和 v_j，若它们之间存在边，则以此边上的权值作为它们之间的最短路径长度；若它们之间不存在有向边，则以 ∞ 作为它们之间的最短路径长度。以后逐步尝试在原路径中加入顶点 k（$k = 0, 1, \cdots, n-1$）作为中间顶点。若增加中间顶点后，得到的路径比原来的路径长度减少了，则以此新路径代替原路径。算法描述如下：

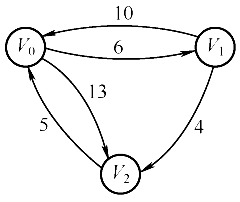
定义一个 n 阶方阵序列 $A^{(-1)}, A^{(0)}, \cdots, A^{(n-1)}$，其中，

$$A^{(-1)}[i][j] = \text{arcs}[i][j]$$

$$A^{(k)}[i][j] = \text{Min}\{A^{(k-1)}[i][j],\quad A^{(k-1)}[i][k] + A^{(k-1)}[k][j]\},\ k = 0, 1, \cdots, n-1$$

式中，$A^{(0)}[i][j]$是从顶点 v_i 到 v_j、中间顶点是 v_0 的最短路径的长度，$A^{(k)}[i][j]$是从顶点 v_i 到 v_j、中间顶点的序号不大于 k 的最短路径的长度。Floyd 算法是一个迭代的过程，每迭代一次，在从 v_i 到 v_j 的最短路径上就多考虑了一个顶点；经过 n 次迭代后，所得到的 $A^{(n-1)}[i][j]$就是 v_i 到 v_j 的最短路径长度，即方阵 $A^{(n-1)}$中就保存了任意一对顶点之间的最短路径长度。

图 6.19 所示为带权有向图 G 及其邻接矩阵。应用 Floyd 算法求所有顶点之间的最短路径长度的过程如表 6.3 所示。算法执行过程的说明如下。



(a) 有向图 G

$$\begin{bmatrix} 0 & 6 & 13 \\ 10 & 0 & 4 \\ 5 & \infty & 0 \end{bmatrix}$$

(b) G 的邻接矩阵

图 6.19　带权有向图 G 及其邻接矩阵

初始化：方阵 $A^{(-1)}[i][j]=\text{arcs}[i][j]$。

第 1 轮：将 v_0 作为中间顶点，对于所有顶点对 $\{i,j\}$，若有 $A^{-1}[i][j]>A^{-1}[i][0]+A^{-1}[0][j]$，则将 $A^{-1}[i][j]$ 更新为 $A^{-1}[i][0]+A^{-1}[0][j]$。有 $A^{-1}[2][1]>A^{-1}[2][0]+A^{-1}[0][1]=11$，更新 $A^{-1}[2][1]=11$，更新后的方阵标记为 A^0。

第 2 轮：将 v_1 作为中间顶点，继续检测全部顶点对 $\{i,j\}$。有 $A^0[0][2]>A^0[0][1]+A^0[1][2]=10$，更新 $A^0[0][2]=10$，更新后的方阵标记为 A^1。

第 3 轮：将 v_2 作为中间顶点，继续检测全部顶点对 $\{i,j\}$。有 $A^1[1][0]>A^1[1][2]+A^1[2][0]=9$，更新 $A^1[1][0]=9$，更新后的方阵标记为 A^2。此时 A^2 中保存的就是任意顶点对的最短路径长度。

表 6.3　Floyd 算法的执行过程

A	$A^{(-1)}$			$A^{(0)}$			$A^{(1)}$			$A^{(2)}$		
	V_0	V_1	V_2	V_0	V_1	V_2	V_0	V_1	V_2	V_0	V_1	V_2
V_0	0	6	13	0	6	13	0	6	**10**	0	6	10
V_1	10	0	4	10	0	4	10	0	4	**9**	0	4
V_2	5	∞	0	5	**11**	0	5	11	0	5	11	0

Floyd 算法的时间复杂度为 $O(|V|^3)$。不过其代码很紧凑，且并不包含其他复杂的数据结构，因此隐含的常数系数是很小的，即使对于中等规模的输入来说，它仍然是相当有效的。

Floyd 算法允许图中有带负权值的边，但不允许包含总权值为负的回路。Floyd 算法同样适用于带权无向图，因为带权无向图可视为权值相同往返二重边的有向图。

也可以用单源最短路径算法来解决每对顶点之间的最短路径问题。轮流将每个顶点作为源点，并且在所有边权值均非负时，运行一次 Dijkstra 算法，其时间复杂度为 $O(|V|^2)\cdot|V|=O(|V|^3)$。

BFS 算法、Dijkstra 算法和 Floyd 算法求最短路径的总结如表 6.4 所示。

表 6.4　BFS 算法、Dijkstra 算法和 Floyd 算法求最短路径的总结

	BFS 算法	Dijkstra 算法	Floyd 算法
用途	求单源最短路径	求单源最短路径	求各顶点之间的最短路径
无权图	适用	适用	适用
带权图	不适用	适用	适用
带负权值的图	不适用	不适用	适用
带负权回路的图	不适用	不适用	不适用
时间复杂度	$O(\|V\|^2)$或 $O(\|V\|+\|E\|)$	$O(\|V\|^2)$	$O(\|V\|^3)$

6.4.3　有向无环图描述表达式

有向无环图：若一个有向图中不存在环，则称为有向无环图，简称 DAG 图。

命题追踪 ▶ 构建表达式的有向无环图（2019）

有向无环图是描述含有公共子式的表达式的有效工具。例如表达式

$$((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)$$

可以用上一章描述的二叉树来表示，如图 6.20 所示。仔细观察该表达式，可发现有一些相同的子表达式$(c+d)$和$(c+d)*e$，而在二叉树中，它们也重复出现。若利用有向无环图，则可实现对相同子式的共享，从而节省存储空间，图 6.21 所示为该表达式的有向无环图表示。

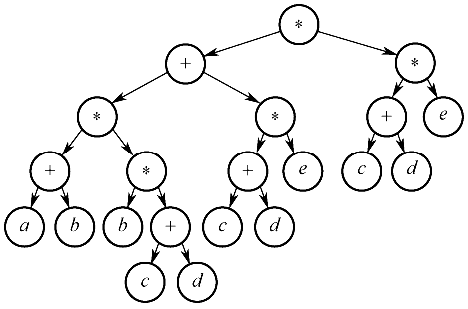


图 6.20 二叉树描述表达式

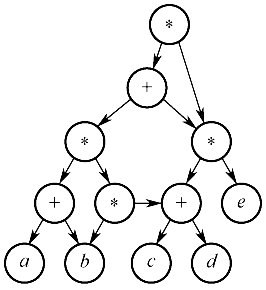


图 6.21 有向无环图描述表达式

用有向无环图描述表达式的解题方法举例见本书配套课程。

注 意

在表达式的有向无环图表示中，不可能出现重复的操作数顶点。

6.4.4 拓扑排序

AOV 网：若用有向无环图表示一个工程，其顶点表示活动，用有向边<V_i, V_j>表示活动 V_i 必须先于活动 V_j 进行的这样一种关系，则将这种有向图称为顶点表示活动的网络，简称 AOV 网。在 AOV 网中，活动 V_i 是活动 V_j 的直接前驱，V_j 是 V_i 的直接后继，这种前驱和后继关系具有传递性，且任何活动 V_i 不能以它自己作为自己的前驱或后继。

拓扑排序：在图论中，由一个有向无环图的顶点组成的序列，当且仅当满足下列条件时，称为该图的一个拓扑排序：

1）每个顶点出现且只出现一次。

2）若顶点 A 在序列中排在顶点 B 的前面，则在图中不存在从 B 到 A 的路径。

或定义为：拓扑排序是对有向无环图的顶点的一种排序，它使得若存在一条从顶点 A 到顶点 B 的路径，则在排序中 B 出现在 A 的后面。每个 AOV 网都有一个或多个拓扑排序序列。

命题追踪 ▶ 拓扑排序和回路的关系（2011）

对一个 AOV 网进行拓扑排序的算法有很多，下面介绍比较常用的一种方法的步骤：

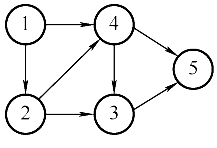
① 从 AOV 网中选择一个没有前驱（入度为 0）的顶点并输出。

② 从网中删除该顶点和所有以它为起点的有向边。

③ 重复①和②直到当前的 AOV 网为空或当前网中不存在无前驱的顶点为止。后一种情况说明有向图中必然存在环。

命题追踪 ▶ 拓扑排序的实例（2010、2014、2018、2021）

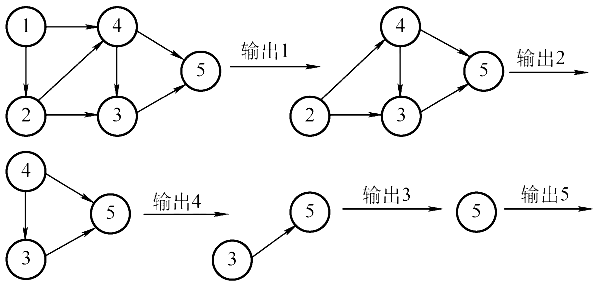
图 6.22 所示为拓扑排序过程的示例。每轮选择一个入度为 0 的顶点并输出，然后删除该顶点和所有以它为起点的有向边，最后得到拓扑排序的结果为{1, 2, 4, 3, 5}。



结点号	1	2	3	4	5
初始入度	0	1	2	2	2
第 1 轮		0	2	1	2
第 2 轮			1	0	2
第 3 轮			0		1
第 4 轮					0
第 5 轮					

(a)

图 6.22 有向无环图的拓扑排序过程



(b)

图 6.22　有向无环图的拓扑排序过程（续）

命题追踪 ▶ （算法题）拓扑排序的实现（2024）

拓扑排序算法（基于邻接表存储）的实现如下：

```
bool TopologicalSort(Graph G){
    InitStack(S);                    //初始化栈，存储入度为 0 的顶点
    int i;
    for(i=0;i<G.vexnum;i++)
        if(indegree[i]==0)
            Push(S,i);               //将所有入度为 0 的顶点入栈
    int count=0;                     //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){              //栈不空，则存在入度为 0 的顶点
        Pop(S,i);                    //栈顶元素出栈
        print[count++]=i;            //输出顶点 i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
        //将所有 i 指向的顶点的入度减 1，并且将入度减为 0 的顶点压入栈 S
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);           //入度为 0，则入栈
        }
    }
    if(count<G.vexnum)
        return  false;               //排序失败，有向图中有回路
    else
        return  true;                //拓扑排序成功
}
```

命题追踪 ▶ 不同存储方式下的拓扑排序的效率（2016）

因为输出每个顶点的同时还要删除以它为起点的边，所以采用邻接表存储时拓扑排序的时间复杂度为 $O(|V|+|E|)$，采用邻接矩阵存储时拓扑排序的时间复杂度为 $O(|V|^2)$。

命题追踪 ▶ DFS 实现拓扑排序的思想（2020）

此外，利用上一节的深度优先遍历也可以实现拓扑排序，下面简单描述其思路，具体代码见本节后的习题。对于有向无环图 G 中的任意结点 u, v，它们之间的关系必然是下列三种之一：

1）若 u 是 v 的祖先，则在调用 DFS 访问 u 之前，必然已对 v 进行了 DFS 访问，即 v 的 DFS 结束时间先于 u 的 DFS 结束时间。从而可考虑在 DFS 函数中设置一个时间标记，在 DFS 调用结束时，对各顶点计时。因此，祖先的结束时间必然大于子孙的结束时间。

2）若 u 是 v 的子孙，则 v 为 u 的祖先，按上述思路，v 的结束时间大于 u 的结束时间。

3）若 u 和 v 没有路径关系，则 u 和 v 在拓扑序列的关系任意。

于是，按结束时间从大到小排列，就可以得到一个拓扑排序序列。

对一个AOV网，若采用下列步骤进行排序，则称之为逆拓扑排序：

① 从AOV网中选择一个没有后继（出度为0）的顶点并输出。

② 从网中删除该顶点和所有以它为终点的有向边。

③ 重复①和②直到当前的AOV网为空。

用拓扑排序算法处理AOV网时，应注意以下问题：

1）入度为零的顶点，即没有前驱活动的或前驱活动都已经完成的顶点，工程可以从这个顶点所代表的活动开始或继续。

命题追踪 ▶ 拓扑排序序列的存在性和唯一性分析（2011、2024）

2）拓扑排序的结果可能不唯一。不少人误认为AOV网的各顶点为线性序列是拓扑序列唯一的充要条件，而它其实只是充分非必要条件。拓扑序列是否唯一的判断条件是在每次输出顶点时，检测入度为0的顶点是否唯一，若每次都唯一，则说明拓扑序列唯一。

3）AOV网中各顶点的地位平等，每个顶点编号是人为的，因此可以按拓扑排序的结果重新编号，生成AOV网的新的邻接存储矩阵，这种邻接矩阵可以是三角矩阵；但对于一般的图来说，若其邻接矩阵是三角矩阵，则存在拓扑序列；反之则不一定成立。

6.4.5 关键路径

在带权有向图中，以顶点表示事件，以有向边表示活动，以边上的权值表示完成该活动的开销（如完成活动所需的时间），称之为用边表示活动的网络，简称AOE网。AOE网和AOV网都是有向无环图，不同之处在于它们的边和顶点所代表的含义是不同的，AOE网中的边有权值；而AOV网中的边无权值，仅表示顶点之间的前后关系。

AOE网具有以下两个性质：

① 只有在某顶点所代表的事件发生后，从该顶点出发的各有向边所代表的活动才能开始；

② 只有在进入某顶点的各有向边所代表的活动都已结束时，该顶点所代表的事件才能发生。

在AOE网中仅有一个入度为0的顶点，称为开始顶点（源点），它表示整个工程的开始；也仅有一个出度为0的顶点，称为结束顶点（汇点），它表示整个工程的结束。

命题追踪 ▶ 关键路径的性质（2020）

在AOE网中，有些活动是可以并行进行的。从源点到汇点的有向路径可能有多条，并且这些路径长度可能不同。完成不同路径上的活动所需的时间虽然不同，但是只有所有路径上的活动都已完成，整个工程才能算结束。因此，从源点到汇点的所有路径中，具有最大路径长度的路径称为关键路径，而把关键路径上的活动称为关键活动。

完成整个工程的最短时间就是关键路径的长度，即关键路径上各活动花费开销的总和。这是因为关键活动影响了整个工程的时间，即若关键活动不能按时完成，则整个工程的完成时间就会延长。因此，只要找到了关键活动，就找到了关键路径，也就可以得出最短完成时间。

下面给出在寻找关键活动时所用到的几个参量的定义。

1．事件v_k的最早发生时间$v_e(k)$

指从源点v_1到顶点v_k的最长路径长度。事件v_k的最早发生时间决定了所有从v_k开始的活动能够开工的最早时间。可用下面的递推公式来计算：

v_e(源点)$=0$

$v_e(k)=\mathrm{Max}\{v_e(j)+\mathrm{Weight}(v_j, v_k)\}$，$v_k$为$v_j$的任意后继，$\mathrm{Weight}(v_j, v_k)$表示$<v_j, v_k>$上的权值

注　意

计算 $v_e()$值时，按从前往后的顺序进行，可以在拓扑排序的基础上计算：

① 初始时，令 $v_e[1...n]=0$。

② 输出一个入度为 0 的顶点 v_j 时，计算它所有直接后继顶点 v_k 的最早发生时间，若 $v_e[j]+\text{Weight}(v_j, v_k)>v_e[k]$，则 $v_e[k]=v_e[j]+\text{Weight}(v_j, v_k)$。以此类推，直至输出全部顶点。

2．事件 v_k 的最迟发生时间 $v_l(k)$

指在不推迟整个工程完成的前提下，即保证它的后继事件 v_j 在其最迟发生时间 $v_l(j)$能够发生时，该事件最迟必须发生的时间。可用下面的递推公式来计算：

$v_l(\text{汇点})=v_e(\text{汇点})$

$v_l(k)=\text{Min}\{v_l(j)-\text{Weight}(v_k, v_j)\}$，$v_k$ 为 v_j 的任意前驱

注　意

计算 $v_l(k)$值时，按从后往前的顺序进行，可以在逆拓扑排序的基础上计算。增设一个栈以记录拓扑序列，拓扑排序结束后从栈顶至栈底便为逆拓扑有序序列。过程如下：

① 初始时，令 $v_l[1...n]=v_e[n]$。

② 栈顶顶点 v_j 出栈，计算其所有直接前驱顶点 v_k 的最迟发生时间，若 $v_l[j]-\text{Weight}(v_k, v_j)<v_l[k]$，则 $v_l[k]=v_l[j]-\text{Weight}(v_k, v_j)$。以此类推，直至输出全部栈中顶点。

3．活动 a_i 的最早开始时间 $e(i)$

指该活动弧的起点所表示的事件的最早发生时间。若边<v_k, v_j>表示活动 a_i，则有 $e(i)=v_e(k)$。

4．活动 a_i 的最迟开始时间 $l(i)$

指该活动弧的终点所表示事件的最迟发生时间与该活动所需时间之差。若边<v_k, v_j>表示活动 a_i，则有 $l(i)=v_l(j)-\text{Weight}(v_k, v_j)$。

5．一个活动 a_i 的最迟开始时间 $l(i)$和其最早开始时间 $e(i)$的差额 $d(i)=l(i)-e(i)$

指该活动完成的时间余量，即在不增加完成整个工程所需总时间的情况下，活动 a_i 可以拖延的时间。若一个活动的时间余量为零，则说明该活动必须要如期完成，否则就会拖延整个工程的进度，所以称 $l(i)-e(i)=0$ 即 $l(i)=e(i)$的活动 a_i 是关键活动。

命题追踪　▶　求关键路径的实例（2019、2022）

求关键路径的算法步骤如下：

① 从源点出发，令 $v_e(\text{源点})=0$，按拓扑有序求其余顶点的最早发生时间 $v_e()$。

② 从汇点出发，令 $v_l(\text{汇点})=v_e(\text{汇点})$，按逆拓扑有序求其余顶点的最迟发生时间 $v_l()$。

③ 根据各顶点的 $v_e()$值求所有弧的最早开始时间 $e()$。

④ 根据各顶点的 $v_l()$值求所有弧的最迟开始时间 $l()$。

⑤ 求 AOE 网中所有活动的差额 $d()$，找出所有 $d()=0$ 的活动构成关键路径。

图 6.23 所示为求解关键路径的过程，简单说明如下：

① 求 $v_e()$：初始 $v_e(1)=0$，在拓扑排序输出顶点过程中，求得 $v_e(2)=3$，$v_e(3)=2$，$v_e(4)=\max\{v_e(2)+2, v_e(3)+4\}=\max\{5, 6\}=6$，$v_e(5)=6$，$v_e(6)=\max\{v_e(5)+1, v_e(4)+2, v_e(3)+3\}=\max\{7, 8, 5\}=8$。

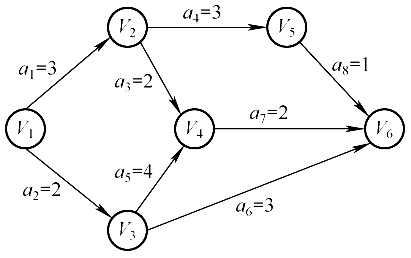
若这是一道选择题，根据上述求 $v_e()$的过程就已经能知道关键路径。

② 求 $v_l()$：初始 $v_l(6)=8$，在逆拓扑排序出栈过程中，求得 $v_l(5)=7$，$v_l(4)=6$，$v_l(3)=\min\{v_l(4)-4, v_l(6)-3\}=\min\{2,5\}=2$，$v_l(2)=\min\{v_l(5)-3, v_l(4)-2\}=\min\{4,4\}=4$，$v_l(1)$必然为 0 而无须再求。

③ 弧的最早开始时间 $e()$等于该弧的起点的顶点的 $v_e()$，结果如下表。

④ 弧的最迟开始时间 $l(i)$等于该弧的终点的顶点的 $v_l()$减去该弧持续的时间，结果如下表。

⑤ 根据 $l(i)-e(i)=0$ 的关键活动，得到的关键路径为(v_1, v_3, v_4, v_6)。



	v_1	v_2	v_3	v_4	v_5	v_6
$v_e(i)$	0	3	2	6	6	8
$v_l(i)$	0	4	2	6	7	8

	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
$e(i)$	0	0	3	3	2	2	6	6
$l(i)$	1	0	4	4	2	5	6	7
$l(i)-e(i)$	1	0	1	1	0	3	0	1

图 6.23　求解关键路径的过程

命题追踪 ▶ 缩短工期的相关分析（2013）

对于关键路径，需要注意以下几点：

1）关键路径上的所有活动都是关键活动，它是决定整个工程的关键因素，因此可以通过加快关键活动来缩短整个工程的工期。但也不能任意缩短关键活动，因为一旦缩短到一定的程度，该关键活动就可能会变成非关键活动。

2）网中的关键路径并不唯一，且对于有几条关键路径的网，只提高一条关键路径上的关键活动速度并不能缩短整个工程的工期，只有加快那些包括在所有关键路径上的关键活动才能达到缩短工期的目的。

各种图算法在采用邻接矩阵或邻接表存储时的时间复杂度如表 6.5 所示。

表 6.5　采用不同存储结构时各种图算法的时间复杂度

	Dijkstra	Floyd	Prim	Kruskal	DFS	BFS	拓扑排序	关键路径
邻接矩阵	$O(n^2)$	$O(n^3)$	$O(n^2)$	-	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$
邻接表	-	-	-	$O(e\log_2 e)$	$O(n+e)$	$O(n+e)$	$O(n+e)$	$O(n+e)$

6.4.6　本节试题精选

一、单项选择题

01. 任何一个无向连通图的最小生成树（　）。

A. 有一棵或多棵　　B. 只有一棵

C. 一定有多棵　　D. 可能不存在

02. 用 Prim 算法和 Kruskal 算法构造图的最小生成树，所得到的最小生成树（　）。

A. 相同　　B. 不相同

C. 可能相同，可能不同　　D. 无法比较

03. 下列关于图的生成树和最小生成树的叙述中，正确的是（　）。

A. 只要无向连通图中没有权值相同的边，则其最小生成树唯一

B. 只要无向图中有权值相同的边，则其最小生成树一定不唯一

C. 从 n 个顶点的连通图中选取 $n-1$ 条权值最小的边，即可构成最小生成树

D. 设连通图 G 含有 n 个顶点，则含有 n 个顶点、$n-1$ 条边的子图一定是 G 的生成树

04. 设有 n 个顶点的无向连通图的最小生成树不唯一，则下列说法中正确的是（ ）。

A. 图的边数一定大于 $n-1$　　B. 图的权值最小的边一定有多条

C. 图的最小生成树的代价不一定相等　　D. 图的各条边的权值不相等

05. 用 Prim 算法求一个带权连通图的最小生成树，在算法执行的某个时刻，已选取的顶点集合 $U=\{1, 2, 3\}$，已选取的边集合 TE = {(1, 2), (2, 3)}，要选取下一条权值最小的边，应当从（ ）组中选取。

A. {(1, 4), (3, 4), (3, 5), (2, 5)}　　B. {(3, 4), (3, 5), (4, 5), (1, 4)}

C. {(1, 2), (2, 3), (3, 5)}　　D. {(4, 5), (1, 3), (3, 5)}

06. 用 Kruskal 算法求一个带权连通图的最小生成树，在算法执行的某个时刻，已选取的边集合 TE = {(1, 2), (2, 3), (3, 5)}，要选取下一条权值最小的边，不可能选取的边是（ ）。

A. (3, 6)　　B. (2, 4)　　C. (1, 3)　　D. (1, 4)

07. 下列关于图的最短路径的相关叙述中，正确的是（ ）。

A. 最短路径一定是简单路径

B. Dijkstra 算法不适合求有回路的带权图的最短路径

C. Dijkstra 算法不适合求任意两个顶点的最短路径

D. Floyd 算法求两个顶点的最短路径时，$path_{k-1}$ 一定是 $path_k$ 的子集

08. 下列关于图的最短路径的相关叙述中，正确的是（ ）。

I. Dijkstra 算法求单源最短路径不允许边的权为负

II. Dijkstra 算法求每对顶点间的最短路径的时间复杂度是 $O(n^2)$

III. Floyd 算法求每对顶点间的最短路径允许边的权为负，但不允许含有负权的回路

A. I、II 和 III　　B. 仅 I　　C. I 和 III　　D. II 和 III

09. 已知带权连通无向图 $G=(V, E)$，其中 $V=\{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$，$E=\{(v_1, v_2)10, (v_1, v_3)2, (v_3, v_4)2, (v_3, v_6)11, (v_2, v_5)1, (v_4, v_5)4, (v_4, v_6)6, (v_5, v_7)7, (v_6, v_7)3\}$（注：顶点偶对括号外的数据表示边上的权值），从源点 v_1 到顶点 v_7 的最短路径上经过的顶点序列是（ ）。

A. v_1, v_2, v_5, v_7　　B. v_1, v_3, v_4, v_6, v_7

C. v_1, v_3, v_4, v_5, v_7　　D. $v_1, v_2, v_5, v_4, v_6, v_7$

10. 用 Dijkstra 算法求一个带权有向图的从顶点 0 出发的最短路径，在算法执行的某个时刻，已求得的最短路径的顶点集合 $S=\{0, 2, 3, 4\}$，下一个选取的目标顶点是顶点 1，则可能修改的最短路径是（ ）。

A. 从顶点 0 到顶点 3 的最短路径　　B. 从顶点 0 到顶点 2 的最短路径

C. 从顶点 2 到顶点 4 的最短路径　　D. 从顶点 0 到顶点 1 的最短路径

11. 下面的（ ）方法可以判断出一个有向图是否有环（回路）。

I. 深度优先遍历　II. 拓扑排序　III. 求最短路径　IV. 求关键路径

A. I、II、IV　　B. I、III、IV　　C. I、II、III　　D. 全部可以

12. 在有向图 G 的拓扑序列中，若顶点 v_i 在顶点 v_j 之前，则不可能出现的情形是（ ）。

A. G 中有弧$<v_i, v_j>$　　B. G 中有一条从 v_i 到 v_j 的路径

C. G中没有弧$<v_i, v_j>$　　D. G中有一条从v_j到v_i的路径

13. 下列关于拓扑排序的说法中，错误的是（ ）。

I. 若某有向图存在环路，则该有向图一定不存在拓扑排序

II. 在拓扑排序算法中为暂存入度为零的顶点，可以使用栈，也可以使用队列

III. 若有向图的拓扑有序序列唯一，则图中每个顶点的入度和出度最多为1

IV. 若有向图的拓扑有序序列唯一，则图中入度为0和出度为0的顶点都仅有1个

A. I、III、IV　　B. III、IV　　C. II、IV　　D. III

14. 下列关于拓扑排序的说法中，正确的是（ ）。

I. 顶点数大于1的强连通图不能进行拓扑排序

II. 在一个有向图的拓扑序列中，若顶点a在顶点b之前，则图中必有一条弧$<a, b>$

III. 若有向无环图的拓扑序列唯一，则可以唯一确定该图

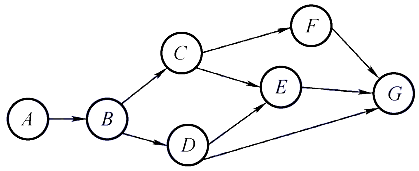
A. I和II　　B. I、II和III　　C. 仅I　　D. I和III

15. 若一个有向图的顶点不能排成一个拓扑序列，则判定该有向图（ ）。

A. 含有多个出度为0的顶点　　B. 是个强连通图

C. 含有多个入度为0的顶点　　D. 含有顶点数大于1的强连通分量

16. 下图所示有向图的所有拓扑序列共有（ ）个。



A. 4　　B. 6　　C. 5　　D. 7

17. 已知有向图$G=(V, E)$，其中$V=\{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$，$E=\{<v_1, v_2>, <v_1, v_3>, <v_1, v_4>, <v_2, v_5>, <v_3, v_5>, <v_3, v_6>, <v_5, v_7>, <v_6, v_7>, <v_4, v_6>\}$，$G$的拓扑序列是（ ）。

A. $\{v_1, v_3, v_4, v_6, v_2, v_5, v_7\}$　　B. $\{v_1, v_3, v_2, v_6, v_4, v_5, v_7\}$

C. $\{v_1, v_3, v_4, v_5, v_2, v_6, v_7\}$　　D. $\{v_1, v_2, v_5, v_3, v_4, v_6, v_7\}$

18. 下列哪种图的邻接矩阵是对称矩阵？（ ）

A. 有向网　　B. 无向图　　C. AOV网　　D. AOE网

19. 若一个有向图具有有序的拓扑排序序列，则它的邻接矩阵必定为（ ）。

A. 对称　　B. 稀疏　　C. 三角　　D. 一般

20. 用DFS算法遍历一个无环有向图，并在DFS算法退栈返回时输出相应的顶点，则输出的顶点序列是（ ）。

A. 逆拓扑有序　　B. 拓扑有序　　C. 无序的　　D. 无法确定

21. 下列关于图的说法中，正确的是（ ）。

I. 有向图中顶点V的度等于其邻接矩阵中第V行中1的个数

II. 无向图的邻接矩阵一定是对称矩阵，有向图的邻接矩阵一定是非对称矩阵

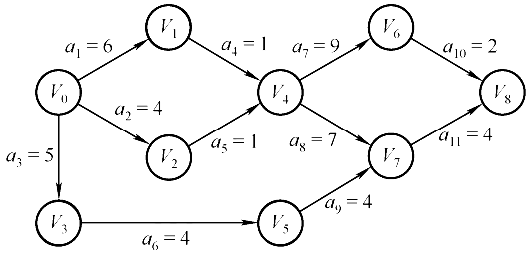
III. 在带权图G的最小生成树G_1中，某条边的权值可能会超过未选边的权值

IV. 若有向无环图的拓扑序列唯一，则可以唯一确定该图

A. I、II和III　　B. III和IV　　C. III　　D. IV

22. 下图所示的AOE网中，关键路径长度为（ ）。

A. 16　　B. 17　　C. 18　　D. 19



23. 若某带权图为 $G=(V, E)$，其中 $V=\{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8, v_9, v_{10}\}$，$E=\{<v_1, v_2>5, <v_1, v_3>6, <v_2, v_5>3, <v_3, v_5>6, <v_3, v_4>3, <v_4, v_5>3, <v_4, v_7>1, <v_4, v_8>4, <v_5, v_6>4, <v_5, v_7>2, <v_6, v_{10}>4, <v_7, v_9>5, <v_8, v_9>2, <v_9, v_{10}>2\}$（注：边括号外的数据表示边上的权值），则 G 的关键路径的长度为（　）。

A. 19　　B. 20　　C. 21　　D. 22

24. 下面关于求关键路径的说法中，不正确的是（　）。

A. 求关键路径是以拓扑排序为基础的

B. 一个事件的最早发生时间与以该事件为始的弧的活动的最早开始时间相同

C. 一个事件的最迟发生时间是以该事件为尾的弧的活动的最迟开始时间与该活动的持续时间的差

D. 任何一个活动的持续时间的改变可能会影响关键路径的改变

25. 下列关于 AOE 网的关键路径的说法中，正确的是（　）。

I. 改变网上某一关键路径上的任意一个关键活动后，必将产生不同的关键路径

II. 关键路径上活动的时间延长多少，整个工期也就随之延长多少

III. 缩短关键路径上任意一个关键活动的持续时间可缩短关键路径长度

IV. 缩短所有关键路径上共有的任意一个关键活动的持续时间可缩短关键路径长度

V. 缩短多条关键路径上共有的任意一个关键活动的持续时间可缩短关键路径长度

A. II 和 V　　B. I、II 和 IV　　C. II 和 IV　　D. I 和 IV

26. 在求 AOE 网的关键路径时，若该有向图用邻接矩阵表示且第 i 列值全为∞，则（　）。

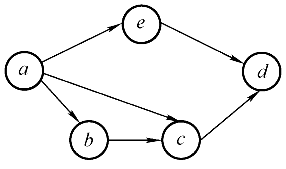
A. 若关键路径存在，第 i 个顶点一定是起点

B. 若关键路径存在，第 i 个顶点一定是终点

C. 关键路径不存在

D. 该有向图对应的无向图存在多个连通分量

27. 【2010 统考真题】对下图进行拓扑排序，可得不同拓扑序列的个数是（　）。



A. 4　　B. 3　　C. 2　　D. 1

28. 【2012 统考真题】下列关于最小生成树的叙述中，正确的是（　）。

I. 最小生成树的代价唯一

II. 所有权值最小的边一定会出现在所有的最小生成树中

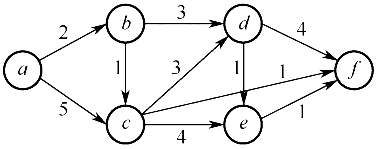
III. 使用 Prim 算法从不同顶点开始得到的最小生成树一定相同

IV. 使用 Prim 算法和 Kruskal 算法得到的最小生成树总不相同

A. 仅 I　　B. 仅 II　　C. 仅 I、III　　D. 仅 II、IV

29. 【2012 统考真题】对下图所示的有向带权图，若采用 Dijkstra 算法求从源点 a 到其他各

顶点的最短路径，则得到的第一条最短路径的目标顶点是 b，第二条最短路径的目标顶点是 c，后续得到的其余各最短路径的目标顶点依次是（ ）。



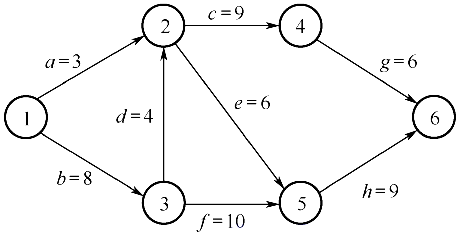
A. d, e, f　　B. e, d, f　　C. f, d, e　　D. f, e, d

30. 【2012 统考真题】若用邻接矩阵存储有向图，矩阵中主对角线以下的元素均为零，则关于该图拓扑序列的结论是（ ）。

A. 存在，且唯一　　B. 存在，且不唯一

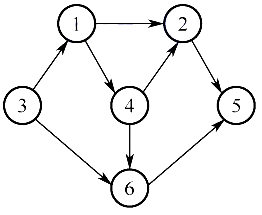
C. 存在，可能不唯一　　D. 无法确定是否存在

31. 【2013 统考真题】下列 AOE 网表示一项包含 8 个活动的工程。通过同时加快若干活动的进度可缩短整个工程的工期。在下列选项中，加快其进度就可缩短工程工期的是（ ）。



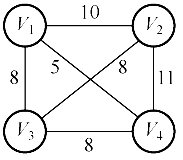
A. c 和 e　　B. d 和 c　　C. f 和 d　　D. f 和 h

32. 【2014 统考真题】对下图所示的有向图进行拓扑排序，得到的拓扑序列可能是（ ）。



A. 3, 1, 2, 4, 5, 6　　B. 3, 1, 2, 4, 6, 5　　C. 3, 1, 4, 2, 5, 6　　D. 3, 1, 4, 2, 6, 5

33. 【2015 统考真题】求下面的带权图的最小（代价）生成树时，可能是 Kruskal 算法第 2 次选中但不是 Prim 算法（从 V_4 开始）第 2 次选中的边是（ ）。



A. (V_1, V_3)　　B. (V_1, V_4)　　C. (V_2, V_3)　　D. (V_3, V_4)

34. 【2011 统考真题】下列关于图的叙述中，正确的是（ ）。

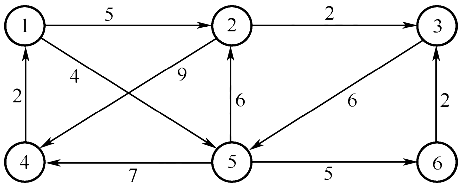
I. 回路是简单路径

II. 存储稀疏图，用邻接矩阵比邻接表更省空间

III. 若有向图中存在拓扑序列，则该图不存在回路

A. 仅 II　　B. 仅 I、II　　C. 仅 III　　D. 仅 I、III

35. 【2016 统考真题】使用 Dijkstra 算法求下图中从顶点 1 到其他各顶点的最短路径，依次得到的各最短路径的目标顶点是（ ）。

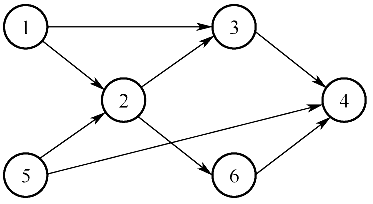


A．5, 2, 3, 4, 6　　B．5, 2, 3, 6, 4　　C．5, 2, 4, 3, 6　　D．5, 2, 6, 3, 4

36. 【2016 统考真题】若对 n 个顶点、e 条弧的有向图采用邻接表存储，则拓扑排序算法的时间复杂度是（　）。

A．$O(n)$　　B．$O(n+e)$　　C．$O(n^2)$　　D．$O(ne)$

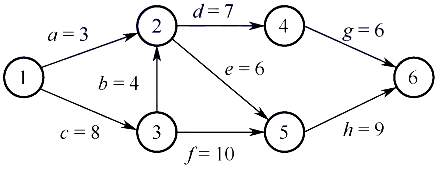
37. 【2018 统考真题】下列选项中，不是如下有向图的拓扑序列的是（　）。



A．1, 5, 2, 3, 6, 4　　B．5, 1, 2, 6, 3, 4

C．5, 1, 2, 3, 6, 4　　D．5, 2, 1, 6, 3, 4

38. 【2019 统考真题】下图所示的 AOE 网表示一项包含 8 个活动的工程。活动 d 的最早开始时间和最迟开始时间分别是（　）。

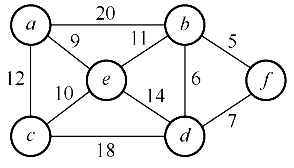


A．3 和 7　　B．12 和 12　　C．12 和 14　　D．15 和 15

39. 【2019 统考真题】用有向无环图描述表达式 $(x+y)((x+y)/x)$，需要的顶点个数至少是（　）。

A．5　　B．6　　C．8　　D．9

40. 【2020 统考真题】已知无向图 G 如下所示，使用 Kruskal 算法求图 G 的最小生成树，加到最小生成树中的边依次是（　）。



A．$(b,f),(b,d),(a,e),(c,e),(b,e)$　　B．$(b,f),(b,d),(b,e),(a,e),(c,e)$

C．$(a,e),(b,e),(c,e),(b,d),(b,f)$　　D．$(a,e),(c,e),(b,e),(b,f),(b,d)$

41. 【2020 统考真题】修改递归方式实现的图的深度优先搜索（DFS）算法，将输出（访问）顶点信息的语句移到退出递归前（执行输出语句后立刻退出递归）。采用修改后的算法遍历有向无环图 G，若输出结果中包含 G 中的全部顶点，则输出的顶点序列是 G 的（　）。

A．拓扑有序序列　　B．逆拓扑有序序列

C. 广度优先搜索序列　　D. 深度优先搜索序列

42.【2020统考真题】若使用AOE网估算工程进度，则下列叙述中正确的是（ ）。

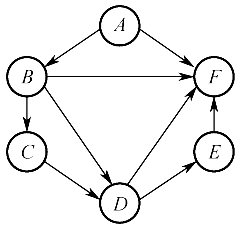
A. 关键路径是从源点到汇点边数最多的一条路径

B. 关键路径是从源点到汇点路径长度最长的路径

C. 增加任意一个关键活动的时间不会延长工程的工期

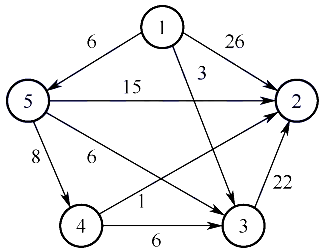
D. 缩短任意一个关键活动的时间将会缩短工程的工期

43.【2021统考真题】给定如下有向图，该图的拓扑有序序列的个数是（ ）。



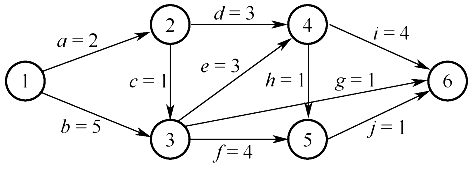
A. 1　　B. 2　　C. 3　　D. 4

44.【2021统考真题】使用Dijkstra算法求下图中从顶点1到其余各顶点的最短路径，将当前找到的从顶点1到顶点2, 3, 4, 5的最短路径长度保存在数组dist中，求出第二条最短路径后，dist中的内容更新为（ ）。



A. 26, 3, 14, 6　　B. 25, 3, 14, 6　　C. 21, 3, 14, 6　　D. 15, 3, 14, 6

45.【2022统考真题】下图是一个有10个活动的AOE网，时间余量最大的活动是（ ）。



A. c　　B. g　　C. h　　D. j

46.【2023统考真题】已知无向连通图G中各边的权值均为1。在下列算法中，一定能够求出图G中从某顶点到其余各顶点最短路径的是（ ）。

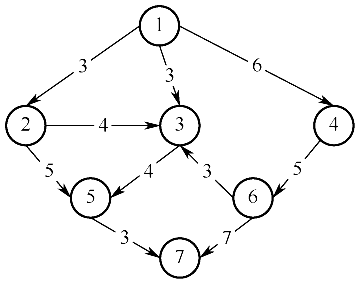
I. Prim算法　II. Kruskal算法　III. 图的广度优先搜索算法

A. 仅I　　B. 仅III　　C. 仅I、II　　D. I、II、III

二、综合应用题

01. 下面是一种称为“破圈法”的求解最小生成树的方法：所谓“破圈法”，是指“任取一圈，去掉圈上权最大的边”，反复执行这一步骤，直到没有圈为止。试判断这种方法是否正确。若正确，说明理由；若不正确，举出反例（注：圈就是回路）。

02. 已知有向图如下图所示。



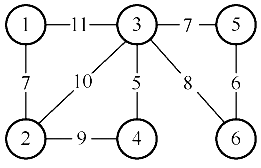
1）写出该图的邻接矩阵表示并据此给出从顶点 1 出发的深度优先遍历序列。

2）求该有向图的强连通分量的数目。

3）给出该图的任意两个拓扑序列。

4）若将该图视为无向图，分别用 Prim 算法和 Kruskal 算法求最小生成树。

03. 对下图所示的无向图，按照 Dijkstra 算法，写出从顶点 1 到其他各个顶点的最短路径和最短路径长度（顺序不能颠倒）。



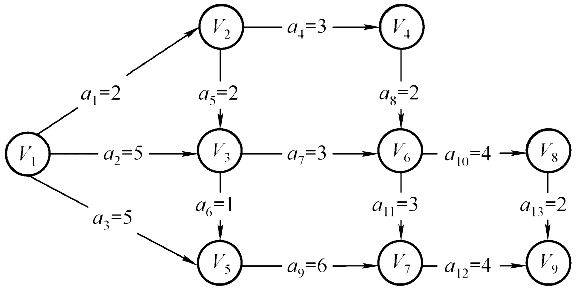
04. 下图所示为一个用 AOE 网表示的工程。

1）画出此图的邻接表表示。

2）完成此工程至少需要多少时间？

3）指出关键路径。

4）哪些活动加速可以缩短完成工程所需的时间？



05. 下表给出了某工程各工序之间的优先关系和各工序所需的时间（其中“—”表示无先驱工序），请完成以下各题：

1）画出相应的 AOE 网。

2）列出各事件的最早发生时间和最迟发生时间。

3）求出关键路径并指明完成该工程所需的最短时间。

工序代号	A	B	C	D	E	F	G	H
所需时间	3	2	2	3	4	3	2	1
先驱工序	—	—	A	A	B	A	C、E	D

06. 一连通无向图，边非负权值，问用 Dijkstra 最短路径算法能否给出一棵生成树，该树是否一定是最小生成树？说明理由。

07. 试编写利用 DFS 实现有向无环图拓扑排序的算法。

08. 【2009统考真题】带权图（权值非负，表示边连接的两顶点间的距离）的最短路径问题是找出从初始顶点到目标顶点之间的一条最短路径。假设从初始顶点到目标顶点之间存在路径，现有一种解决该问题的方法：

① 设最短路径初始时仅包含初始顶点，令当前顶点 u 为初始顶点。

② 选择离 u 最近且尚未在最短路径中的一个顶点 v，加入最短路径，修改当前顶点 $u=v$。

③ 重复步骤②，直到 u 是目标顶点时为止。

请问上述方法能否求得最短路径？若该方法可行，请证明；否则，请举例说明。

09. 【2011统考真题】已知有6个顶点（顶点编号为0～5）的有向带权图 G，其邻接矩阵 A 为上三角矩阵，按行为主序（行优先）保存在如下的一维数组中。

4	6	∞	∞	∞	5	∞	∞	∞	4	3	∞	∞	3	3

要求：

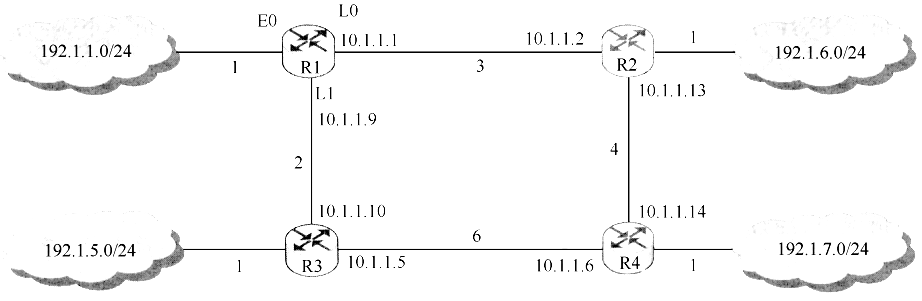
1）写出图 G 的邻接矩阵 A。

2）画出有向带权图 G。

3）求图 G 的关键路径，并计算该关键路径的长度。

10. 【2014统考真题】某网络中的路由器运行OSPF路由协议，下表是路由器R1维护的主要链路状态信息（LSI），R1构造的网络拓扑图（见下图）是根据题下表及R1的接口名构造出来的网络拓扑。

		R1的LSI	R2的LSI	R3的LSI	R4的LSI	备　注
Router ID		10.1.1.1	10.1.1.2	10.1.1.5	10.1.1.6	标识路由器的IP地址
Link1	ID	10.1.1.2	10.1.1.1	10.1.1.6	10.1.1.5	所连路由器的Router ID
	IP	10.1.1.1	10.1.1.2	10.1.1.5	10.1.1.6	Link1的本地IP地址
	Metric	3	3	6	6	Link1的费用
Link2	ID	10.1.1.5	10.1.1.6	10.1.1.1	10.1.1.2	所连路由器的Router ID
	IP	10.1.1.9	10.1.1.13	10.1.1.10	10.1.1.14	Link2的本地IP地址
	Metric	2	4	2	4	Link2的费用
Net1	Prefix	192.1.1.0/24	192.1.6.0/24	192.1.5.0/24	192.1.7.0/24	直连网络Net1的网络前缀
	Metric	1	1	1	1	到达直连网络Net1的费用



请回答下列问题。

1）本题中的网络可抽象为数据结构中的哪种逻辑结构？

2）针对表中的内容，设计合理的链式存储结构，以保存表中的链路状态信息（LSI）。要求给出链式存储结构的数据类型定义，并画出对应表的链式存储结构示意图（示意图中可仅以ID标识结点）。

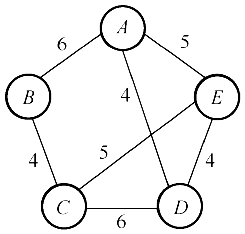
3）按照 Dijkstra 算法的策略，依次给出 R1 到达子网 192.1.x.x 的最短路径及费用。

11. 【2017 统考真题】使用 Prim 算法求带权连通图的最小（代价）生成树（MST）。请回答下列问题：

1）对下列图 G，从顶点 A 开始求 G 的 MST，依次给出按算法选出的边。

2）图 G 的 MST 是唯一的吗？

3）对任意的带权连通图，满足什么条件时，其 MST 是唯一的？



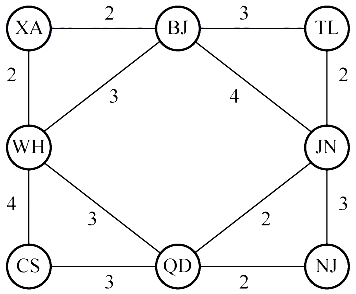
12. 【2018 统考真题】拟建设一个光通信骨干网络连通 BJ、CS、XA、QD、JN、NJ、TL 和 WH 等 8 个城市，下图中无向边上的权值表示两个城市之间备选光缆的铺设费用。

请回答下列问题：

1）仅从铺设费用角度出发，给出所有可能的最经济的光缆铺设方案（用带权图表示），并计算相应方案的总费用。

2）该图可采用图的哪种存储结构？给出求解问题 1）所用的算法名称。

3）假设每个城市采用一个路由器按 1）中得到的最经济方案组网，主机 H1 直接连接 TL 的路由器，主机 H2 直接连接 BJ 的路由器。若 H1 向 H2 发送一个 TTL = 5 的 IP 分组，则 H2 是否可以收到该 IP 分组？



13. 【2024 统考真题】2023 年 10 月 26 日，神舟十七号载人飞船发射取得圆满成功，再次彰显了中国航天事业的辉煌成就。载人航天工程是包含众多子工程的复杂系统工程，为了保证工程的有序开展，需要明确各子工程的前导子工程，以协调各子工程的实施。该问题可以简化、抽象为有向图的拓扑序列问题。已知有向图 G 采用邻接矩阵存储，类型定义如下。

```
typedef struct                          //图的类型定义
{
    int numVertices, numEdges;          //图的顶点数和有向边数
    char VerticesList[MAXV];            //顶点表，MAXV 为已定义常量
    int Edge[MAXV][MAXV];               //邻接矩阵
} MGraph;
```

请设计算法：`int uniquely(MGraqh G)`，判定 `G` 是否存在唯一的拓扑序列，若是，则返回 1，否则返回 0。要求如下。

1）给出算法的基本设计思想。